$ kubectl get interview-readiness

Kubernetes & EKS Production Interview Handbook

259 scenario-based questions for Staff / Lead SRE & Platform Engineering interviews

7 DOMAINS · 21 SUBSECTIONS · 259 QUESTIONS

- Kubernetes Production Troubleshooting
- EKS Production
- Deployment Process & GitOps
- Kubernetes Security
- TLS & Certificates
- Trick Questions
- Advanced Staff / Principal SRE Scenarios

Prepared for Saroj · Senior SRE / DevOps Engineer · July 2026

Contents

259 QUESTIONS · 7 DOMAINS · 21 SUBSECTIONS

- **Kubernetes Production Troubleshooting** 1
  - Cluster is Down 1
  - Worker Node Problems 3
  - Pod Issues 6
  - Deployment Problems 10
  - Networking 12
  - Storage 16
  - Resource Issues 18
  - Logging & Monitoring 21
  - Security (Cluster-Level) 23
- **EKS Production Questions** 26
  - Control Plane 26
  - Node Groups 27
  - IAM 28
  - Networking (EKS/VPC) 29
  - AWS Load Balancer Controller 31
  - EBS / EFS (EKS) 32
- **Deployment Process Questions** 34
  - Deployment Process 34
- **Kubernetes Security** 40
- **TLS & Certificates** 48
  - TLS & Certificate Concepts 48
  - Certificate Management 57
- **Trick Questions** 61
- **Advanced Staff / Principal SRE Scenarios** 69
  - Advanced Scenarios 69

PART I

Kubernetes Production Troubleshooting

103 questions · cluster, node, pod, deployment, network, storage, resource & ops issues

Cluster is Down

Q1 Entire Kubernetes cluster is inaccessible. Where do you start?

Separate "control plane down" from "network path down" before touching anything. First: `kubectl cluster-info` and `kubectl get nodes` from a machine with known-good connectivity. If that hangs, it's almost always the API server or the path to it, not individual workloads.

```
curl -k -m 5 https://<api-endpoint>:6443/healthz
kubectl get --raw='/readyz?verbose'
```

For EKS, check the console cluster status and the control-plane CloudWatch log group first — AWS owns the control plane, so you're checking for AWS-side incidents before assuming a config error. For self-managed, SSH to a control-plane node: `systemctl status kube-apiserver etcd kube-scheduler kube-controller-manager`, then `journalctl -u kube-apiserver -f`. Check the LB in front of the API servers — one unhealthy target behind a round-robin NLB looks exactly like "cluster is down" for whichever client hits it. Finally check certificate expiry (`openssl x509 -enddate -noout -in apiserver.crt`) — expired API server or etcd certs are the classic silent total-outage cause.

Q2 kubectl command hangs forever. What will you check?

A hang (not an immediate error) points to a network/connectivity problem rather than an application-level rejection. Checklist, in order:

- **DNS/endpoint resolution** — can you resolve and reach the API server host/port at all? `nc -zv <api-host> 6443`
- **kubeconfig context** — `kubectl config current-context` pointing at a stale or wrong endpoint (common after a cluster recreate).
- **VPN / bastion / security group** — private API endpoints need the right network path; a dropped VPN or a security group rule change silently blackholes traffic instead of rejecting it, which is exactly what produces a hang rather than a connection-refused.
- **API server overload** — if you can reach the port but requests never return, the API server itself may be saturated (too many watches, etcd slow). Check `kubectl get --raw='/metrics'` from a node that can reach it, or CloudWatch `apiserver_request_duration_seconds` for EKS.
- **Client-side timeout** — add `--request-timeout=5s -v=6` to see exactly where the client is stuck (DNS, TCP connect, TLS handshake, or waiting on response).

Q3 API Server is not responding. Possible causes?

Group causes by layer so you investigate in the right order instead of guessing:

- **etcd** — API server is stateless-ish but depends entirely on etcd; if etcd quorum is lost or etcd is slow (disk fsync latency), the API server appears hung. Check `etcdctl endpoint health --cluster`.
- **Resource exhaustion** — OOM on the control-plane node, or the apiserver pod/process itself OOMKilled; check `dmesg | grep -i oom` and control-plane node memory.
- **Certificates** — expired serving cert, expired client cert used for etcd communication, or CA mismatch after a rotation.
- **Admission webhooks** — a validating/mutating webhook pointing at a dead service will make every write request hang until timeout; this is a very common "apiserver is slow" root cause that isn't the apiserver at all.
- **Too many watches / client-go storms** — a buggy controller re-listing constantly can saturate the API server.
- **Load balancer / security group** in front of API servers (self-managed) misrouting or blocking traffic.

Interview signal: mention checking `kube-apiserver` logs for `context deadline exceeded` pointing at etcd, versus webhook timeout errors, since they lead down very different remediation paths.

Q4 etcd disk is full. What happens?

etcd stops accepting writes and raises a **NOSPACE alarm**; the cluster becomes effectively read-only, then even reads degrade as compaction and defragmentation can't run. The API server starts returning `etcdserver: mvcc: database space exceeded`, and every write (new pods, deployments, even leases/heartbeats) fails.

```
# Check alarm
etcdctl alarm list
# Compact to a recent revision
etcdctl compact $(etcdctl endpoint status --write-out="json" | jq -r '.[0].Status.header.revision')
# Defragment to actually reclaim disk
etcdctl defrag --cluster
# Clear the alarm once space is reclaimed
etcdctl alarm disarm
```

Root cause is almost always missing compaction/defrag automation or an unbounded event/history retention. Prevention: enable auto-compaction (`--auto-compaction-mode=revision --auto-compaction-retention=1000`), monitor `etcd_mvcc_db_total_size_in_bytes`, and set `--quota-backend-bytes` with alerting well before the ceiling.

Q5 etcd is corrupted. Recovery steps?

First confirm it's actually corruption, not just a lagging member: `etcdctl endpoint status --cluster -w table` and check for a `DB corruption` error in logs. If one member of a 3/5-node cluster is corrupted, the fastest safe path is to remove that member and rejoin it as fresh (etcd's raft will replicate a clean copy from the healthy quorum) — this is far safer than trying to repair the on-disk file.

```
etcdctl member remove <member-id>
# wipe data dir on that node, then
etcdctl member add <name> --peer-urls=https://<ip>:2380
# restart etcd on that node with --initial-cluster-state=existing
```

If *quorum itself* is lost (majority of members corrupted/down), you restore from the most recent snapshot: `etcdctl snapshot restore backup.db --name ... --initial-cluster ... --initial-advertise-peer-urls ...`, then point the apiserver at the restored data dir. This is why scheduled `etcdctl snapshot save` to S3/off-node storage is non-negotiable for production — without a snapshot, full quorum loss means rebuilding cluster state from scratch.

Q6 Scheduler is down. What impact does it have?

Existing running workloads are unaffected — kube-scheduler only assigns new pods to nodes; it isn't in the data path for traffic or for already-scheduled pods. The blast radius is entirely about *new* scheduling decisions: new pods (from deployments, jobs, scale-ups, node replacements) stay stuck in `Pending` with no `nodeName` set, because nothing is assigning them.

This matters a lot in a rolling deployment or an HPA scale-up event during the outage — those pods will queue up Pending until the scheduler comes back. Check `kubectl get pods --all-namespaces --field-selector=status.phase=Pending` to see the backlog, and `kubectl get componentstatuses` / scheduler pod logs (`kube-system`) for the actual cause — usually a leader-election failure, a crash loop, or (in EKS-like setups) a missing/expired credential for talking to the API server.

Q7 Controller Manager crashes continuously.

kube-controller-manager runs the reconciliation loops (ReplicaSet, Endpoints, Node lifecycle, Job, ServiceAccount token, etc.), so a persistent crash means the cluster stops *self-healing*: a dead pod won't be replaced, node NotReady conditions won't trigger pod eviction/rescheduling, new namespaces won't get a default ServiceAccount. Already-running, already-correct state keeps working; drift stops being corrected.

```
kubectl -n kube-system logs -l component=kube-controller-manager --previous
kubectl -n kube-system describe pod <controller-manager-pod>
```

Common causes: leader-election lease contention with a duplicate/misconfigured instance, a bad flag after an upgrade, exhausted node resources causing repeated OOMKill, or a webhook/CRD it depends on being unavailable. Fix the crash loop first (usually a config/flag rollback or resource bump), then verify catch-up by watching endpoint and replica counts stabilize.

Q8 kubelet service stopped on worker nodes.

Existing containers on that node **keep running** (they're managed by containerd/CRI directly), but the node stops reporting heartbeats, so after `node-monitor-grace-period` (default 40s) it flips to `NotReady`, and after `pod-eviction-timeout` (default 5m) the controller manager starts evicting/rescheduling its pods elsewhere. No new pods can be scheduled there, probes stop being executed by that kubelet, and `kubectl exec/logs` to pods on that node fail since those go through the kubelet.

```
systemctl status kubelet
journalctl -u kubelet -n 100 --no-pager
```

Typical causes: disk full on the node (kubelet refuses to start), a corrupted certificate used for kubelet→apiserver auth, a bad config after a node-group AMI/bootstrap change, or cgroup driver mismatch after an OS/containerd upgrade (`SystemdCgroup` mismatch is a classic one post-upgrade).

Q9 kube-proxy crashes.

kube-proxy programs the iptables/IPVS rules that implement Services. If it crashes, **existing rules stay in the kernel** (nothing removes them), so already-established connections and existing Service routing keep working for a while. What breaks is anything that changes: new Services, Endpoints churn (pods rolling, scaling), and any Service whose backend set changes won't get updated rules — so traffic keeps going to old/dead pod IPs, causing intermittent connection resets that look like application flakiness.

```
kubectl -n kube-system get pods -l k8s-app=kube-proxy -o wide
kubectl -n kube-system logs <kube-proxy-pod>
iptables -t nat -L KUBE-SERVICES -n | head
```

Since kube-proxy runs as a DaemonSet, it's usually a per-node issue — a crash loop from a config error, or it being killed by resource pressure on that node. Restarting the DaemonSet pod resyncs rules from current Endpoints/EndpointSlices immediately.

Q10 Cluster DNS stopped working.

Symptoms first: apps can still reach each other by IP and existing connections survive, but anything doing a fresh DNS lookup (Service names, external hosts) starts failing/timing out. This is almost always CoreDNS.

```
kubectl -n kube-system get pods -l k8s-app=kube-dns
kubectl -n kube-system logs -l k8s-app=kube-dns --tail=50
kubectl run dnstest --image=busybox:1.36 --rm -it --restart=Never -- nslookup kubernetes.default
```

Check, in order: (1) are CoreDNS pods actually Running (crash loop from a bad Corefile after an edit, or OOMKilled on large clusters without enough memory headroom); (2) is the `kube-dns` Service's endpoint list empty (label selector mismatch after an upgrade); (3) upstream resolution — CoreDNS forwarding to a VPC resolver that's unreachable (security group/NACL change) will break external names while cluster-internal names still work, a useful signal for narrowing it down; (4) NodeLocal DNSCache, if installed, masking or shifting where the failure actually is.

Worker Node Problems

Q11 One worker node becomes NotReady.

`kubectl describe node <node>` first — the `Conditions` block tells you which sub-signal tripped (Ready/MemoryPressure/DiskPressure/PIDPressure/NetworkUnavailable) and the `Events` section often has the human-readable reason. Then SSH in if you can reach it: `systemctl status kubelet containerd`, `journalctl -u kubelet -n 200`.

Common root causes: kubelet crashed or can't renew its certificate; containerd/Docker hung; the node ran out of disk/memory/PIDs; a security-group or ENI issue cut it off from the API server; or the underlying EC2 instance had a hardware/hypervisor issue. If the node is genuinely unreachable (AWS console shows instance status check failure), the pragmatic move in production is cordon + drain + terminate and let the ASG/node group replace it, rather than debugging a box you're about to throw away — `kubectl cordon <node> && kubectl drain <node> --ignore-daemonsets --delete-emptydir-data`.

Q12 Multiple nodes suddenly become NotReady.

When it's *many* nodes at once, stop thinking "node problem" and start thinking "shared dependency problem." The pattern almost always points to something all nodes rely on simultaneously:

- **Control plane / API server** itself is degraded — nodes can't send heartbeats, so they all flip NotReady together even though the nodes themselves are fine. Check apiserver health before touching any node.
- **A shared network path** — VPC route table change, NAT Gateway failure, a security group rule pushed cluster-wide, or a CNI plugin issue rolled out via a DaemonSet update.
- **DNS resolution** to the API server endpoint failing across the board (e.g., Route53/CoreDNS upstream issue).
- **A bad node-group launch template or AMI** rolled out to all new/replaced nodes.

Check `kubectl get events --all-namespaces --sort-by=.lastTimestamp | tail -50` for a correlated timestamp across nodes, and check the API server's own `--node-monitor-grace-period` logs to rule out the control plane being the actual source.

Q13 Node is Ready but pods aren't getting scheduled.

`kubectl describe pod` on the Pending pod tells you why almost every time — look at the `Events` for a scheduler message. Ready ≠ schedulable; common blockers on an otherwise-healthy node:

- **Taints** without matching tolerations on the pod (`kubectl describe node | grep Taints`).
- **Node selectors / affinity rules** that don't match this node's labels.
- **Allocatable resources** already consumed — requests, not just usage, count against scheduling; `kubectl describe node` shows "Allocated resources" vs capacity.
- **Cordoned node** (`SchedulingDisabled` next to Ready in `kubectl get nodes`) — someone drained it for maintenance and forgot to uncordon.
- **Pod topology spread constraints / anti-affinity** preventing co-location.
- **Max pods per node** limit reached (EKS ties this to instance ENI capacity by default).

Q14 Node DiskPressure.

Kubelet's eviction manager sets the `DiskPressure` condition when disk usage crosses its thresholds (defaults: `nodefs.available<10%`, `imagefs.available<15%`, or inodes equivalents). Once set, the node is marked unschedulable for new pods and the kubelet starts evicting existing pods (highest resource-usage-over-request first) and garbage-collecting unused images/containers to reclaim space.

```
kubectl describe node <node> | grep -A3 DiskPressure
df -h; du -sh /var/lib/containerd/* 2>/dev/null | sort -rh | head
```

Usual causes: excessive container image sprawl (no GC or too many tags pulled), log files growing unbounded inside `/var/log/containers` without a log-rotation/shipping sidecar, or a workload writing to emptyDir/local disk without limits. Fix: clean up manually if urgent (`crictl rmi --prune`), then fix the actual leak — log rotation, image cleanup policy, or moving writes to a properly-sized PVC.

Q15 Node MemoryPressure.

Set when available memory drops below the kubelet's `memory.available` soft/hard eviction threshold (default hard: `<100Mi`). The node stops accepting new pods and the kubelet evicts pods ranked by: is usage over request → then by Pod priority → then by usage-minus-request magnitude — so a low-priority pod using way more memory than it requested is evicted before a well-behaved one, regardless of who "caused" the pressure.

```
kubectl describe node <node> | grep -A3 MemoryPressure
kubectl top pod --all-namespaces --sort-by=memory | head
```

Root causes are usually pods with no memory limits (or limits far above requests) allowing overcommit, a genuine memory leak in one workload, or too many pods packed via low requests on an undersized instance type. Fix at both layers: set sane requests/limits so the scheduler bin-packs correctly, and size node groups against real usage, not just requests.

Q16 Node PIDPressure.

Less common than disk/memory pressure but shows up under high-churn workloads: the node is close to its max PID count (kernel `pid_max`, or the kubelet's configured `podPidsLimit`), usually from a container leaking processes/threads (a fork bomb, zombie processes not reaped because the container's PID 1 isn't reaping children, or a runaway job spawning subprocesses without limits).

```
cat /proc/sys/kernel/pid_max
ps -eLf | wc -l
kubectl describe node <node> | grep -A3 PIDPressure
```

Fix: identify the offending pod (often via `ps` on the node grouped by cgroup, or container runtime stats), set a per-pod `pids` limit via `PodPidsLimit` /pod `securityContext` or a `LimitRange`, and make sure the container image uses a proper init process (tini/dumb-init) so zombies get reaped instead of accumulating.

Q17 Node NetworkUnavailable.

This condition is set by the cloud-controller-manager (or CNI, depending on setup) when the node's network isn't correctly configured yet — most often seen briefly on brand-new nodes before the CNI plugin (e.g. VPC CNI on EKS) finishes initializing routes, and it should clear on its own within a minute or two. If it *persists*, the node will never receive pods (it's treated as not ready for networking purposes even if the base `Ready` condition looks fine in some setups).

```
kubectl describe node <node> | grep -A3 NetworkUnavailable
kubectl -n kube-system get pods -o wide | grep aws-node   # VPC CNI pod on that node
kubectl -n kube-system logs <aws-node-pod> -c aws-node
```

Persistent cases usually trace back to the CNI DaemonSet pod crashing on that node, IP/ENI exhaustion (no addresses left to assign), or a route table/subnet misconfiguration for that node's AZ.

Q18 Node frequently reboots.

Distinguish a kernel-level reboot from Kubernetes just replacing the node (ASG/node-group churn can masquerade as "the node keeps rebooting" when it's actually a new instance each time). Check instance identity/uptime, not just the node name.

```
uptime; last reboot | head
journalctl --list-boots
dmesg -T | grep -iE 'oom|panic|nmi|watchdog'
aws ec2 describe-instances --instance-ids <id> --query 'Reservations[].Instances[].StateTransitionReason'
```

Real repeated reboots usually trace to: kernel OOM killing critical system processes triggering a panic/watchdog reset, an EC2 instance health-check failure triggering auto-recovery, a hung task watchdog (`hung_task_timeout_secs`) firing under heavy I/O wait, or (self-managed) a kubelet/systemd unit configured to reboot the host on certain failures. Cross-check CloudWatch `StatusCheckFailed_System` to rule out underlying hardware before debugging inside the OS.

Q19 Node CPU reaches 100%.

Unlike memory, Kubernetes doesn't OOM-kill for CPU — pods just get throttled by CFS if they have limits, or silently starve each other if they don't. So "CPU at 100%" first needs a "is this a problem?" check: throughput/latency dashboards, not just the number.

```
kubectl top nodes
kubectl top pods --all-namespaces --sort-by=cpu | head
top -H   # on the node, to see which process/thread
```

Diagnose whether it's one runaway pod (missing/too-high CPU limit, or an actual bug like a busy-loop) versus aggregate legitimate load exceeding capacity (undersized node group, missing HPA, or a traffic spike). Check `container_cpu_cfs_throttled_periods_total` in Prometheus to see throttling even before CPU looks saturated at the node level. Fix path differs: set/right-size CPU limits and requests for the noisy neighbor case, or scale out (Cluster Autoscaler / Karpenter) for the legitimate-load case — don't reach for "just add limits" if the real issue is under-provisioning.

Q20 Worker node lost connection with API server.

From the node's side, check whether it's a network problem or an auth problem — they look similar but fix very differently.

```
# On the node
curl -k -m 5 https://<api-endpoint>:6443/healthz
journalctl -u kubelet -n 100 | grep -iE 'unauthorized|timeout|refused|certificate'
```

- **Network path**: security group blocking 6443 outbound/inbound, route table change, NAT/IGW issue for a private node reaching a public API endpoint, or VPN/Direct Connect flapping for hybrid setups.
- **Auth/certs**: kubelet's client certificate expired (common if `rotateCertificates` isn't enabled and manual renewal was missed), or the node's IAM role/aws-auth ConfigMap mapping was changed so it's no longer recognized (EKS-specific — `system:node:<name>` group missing).

Once reconnected, the node self-heals: it re-registers, resumes heartbeats, and the controller manager stops any eviction timers that hadn't yet fired.

Pod Issues

Q21 Pod stuck in Pending.

Pending means the scheduler hasn't (or can't) assign it to a node yet. `kubectl describe pod` → `Events` gives the exact scheduler complaint almost every time:

- `Insufficient cpu/memory` — no node has enough allocatable capacity; either scale out or reduce requests.
- `node(s) had taint ... that the pod didn't tolerate` — missing toleration.
- `0/N nodes are available: N node(s) didn't match node selector/affinity`.
- `persistentvolumeclaim ... not bound` — pod waiting on a PVC that's itself Pending (storage layer issue, see PVC Pending below).
- No events at all, PriorityClass/preemption not helping — check if the scheduler itself is unhealthy.

For Cluster Autoscaler/Karpenter setups, also check whether a scale-up is actually in progress (`kubectl -n kube-system logs -l app=cluster-autoscaler`) — Pending briefly while new capacity comes online is expected, not a bug.

Q22 Pod stuck in ContainerCreating.

The pod *has* been scheduled, so this is a node-side problem preparing the sandbox — different failure domain than Pending. `kubectl describe pod` events again first.

- **Volume mount stuck** — PVC/CSI driver taking too long to attach (EBS attach limits per node, or an EFS mount target unreachable).
- **Secret/ConfigMap not found** — referenced in a volume but doesn't exist or wrong namespace.
- **Image pull in progress** for a very large image, or slow registry — this looks identical to a hang if you don't check pull progress specifically.
- **CNI failing to assign an IP** — `failed to create pod sandbox... failed to setup network`, usually ENI/IP exhaustion.
- **Pause container image unreachable** if using a private/custom pause image and registry auth is misconfigured.

Check node-side: `crictl ps -a` and `journalctl -u kubelet | grep <pod-uid>` for the specific CRI error.

Q23 Pod in CrashLoopBackOff.

The container starts, exits, and Kubernetes restarts it with exponential backoff (10s, 20s, 40s ... capped at 5m). This is a status *about the restart pattern*, not the root cause itself — always look one level deeper.

```
kubectl logs <pod> --previous     # logs from the crashed instance, not the new one
kubectl describe pod <pod>         # exit code + reason
kubectl get events --field-selector involvedObject.name=<pod>
```

Exit code narrows it fast: **0** = process exited cleanly but shouldn't have (bad entrypoint/command, app finishes and quits); **1** = application error, check `--previous` logs; **137** = SIGKILL, almost always OOMKilled (check `describe pod` for `Reason: OOMKilled`) or a liveness probe killing it; **139** = segfault; **143** = SIGTERM, often a graceful-shutdown bug not honoring the signal. Also check dependent services — a container that crashes because it can't reach its database on startup will crash-loop forever until the dependency is fixed, not the container itself.

Q24 Pod ImagePullBackOff.

This is the backoff *wrapper state* after one or more failed pulls — the underlying reason is always in the immediately preceding `ErrImagePull` event, so check that first (see next question). ImagePullBackOff itself just means "I tried, it failed, I'm now waiting before retrying" — there's nothing new to diagnose here beyond the root cause underneath, but it's worth knowing the backoff means you don't need to manually retry; deleting and recreating the pod won't fix anything the backoff wouldn't have retried anyway.

```
kubectl describe pod <pod> | grep -A5 Events
```

Q25 Pod ErrImagePull.

The actual pull attempt failed. Common causes, roughly in likelihood order:

- **Wrong image name/tag** — typo, or tag deleted from the registry (retagged/cleaned up by a retention policy).
- **Missing/invalid** `imagePullSecrets` for a private registry, or an expired token (ECR tokens expire every 12h — if you're not using IRSA/node-role auth and instead a static secret, this bites people constantly).
- **Node can't reach the registry** — private subnet without a NAT/VPC endpoint for ECR, or a security group blocking egress to the registry.
- **Platform/architecture mismatch** — pulling an amd64-only image onto an arm64 (Graviton) node.
- **Registry rate limiting** (Docker Hub's anonymous pull limits are a classic surprise in production).

```
kubectl describe pod <pod> | grep -i "failed to pull"
crictl pull <image>   # reproduce directly on the node
```

Q26 OOMKilled.

The container exceeded its memory **limit** (not request) and the kernel cgroup OOM killer terminated it — this is enforced per-container by the kernel, independent of whether the node itself has free memory. `kubectl describe pod` shows `Last State: Terminated, Reason: OOMKilled, Exit Code: 137`.

```
kubectl describe pod <pod> | grep -A8 "Last State"
kubectl top pod <pod> --containers
dmesg -T | grep -i "killed process"   # on the node, confirms which pid/cgroup
```

Fix path depends on whether the limit is wrong or the app is wrong: if usage genuinely needs more (data grows, cache size, JVM heap sized against a hard limit incorrectly — classic JVM-in-container mistake is not respecting the cgroup limit), raise the limit and verify against real usage trends. If it's an actual leak, raising the limit only delays the crash; profile the app (heap dump, pprof, etc.) instead. Also check whether it's the *sidecar* getting OOMKilled, not the main container — easy to miss in a multi-container pod.

Q27 Evicted Pods.

Eviction is the kubelet (node-pressure eviction) or the API (e.g., during a drain, or manual `Eviction` API call for PodDisruptionBudget-aware removal) proactively removing a pod to protect the node or honor an operation — different from a crash. `kubectl get pod <pod> -o yaml` shows `status.reason: Evicted` and a message naming the pressure type (e.g., `The node was low on resource: memory`).

```
kubectl get pods --all-namespaces --field-selector=status.phase=Failed
kubectl describe node <node> | grep -A3 Pressure
```

Evicted pods aren't automatically deleted (their controller creates a replacement, but the evicted object lingers) — clean up in bulk with a field-selector delete if they accumulate: `kubectl get pods --field-selector=status.phase=Failed -o name | xargs kubectl delete`. Root-cause it the same way as DiskPressure/MemoryPressure above rather than treating symptoms.

Q28 Completed Pod never disappears.

Kubernetes doesn't garbage-collect terminal (Completed/Failed) pods on its own timeline by default in the way people expect — that's governed by the **Pod GC Controller**'s `terminated-pod-gc-threshold` (default 12500 cluster-wide!), so on a small/medium cluster you can accumulate thousands of finished Job pods before GC kicks in at all.

```
kubectl get pods --all-namespaces --field-selector=status.phase=Succeeded | wc -l
```

Practical fix: for Jobs, set `ttlSecondsAfterFinished` in the Job spec so the Job (and its pods) auto-delete a set time after completion — this is the modern, correct fix rather than relying on cluster-wide GC thresholds or cron-job cleanup scripts. For CronJobs, also set `successfulJobsHistoryLimit` / `failedJobsHistoryLimit` to bound history directly.

Q29 Pod Terminating forever.

Delete sends SIGTERM, then Kubernetes waits `terminationGracePeriodSeconds` (default 30s) before SIGKILL. Stuck-forever usually means one of these is blocking the finalizer/grace period logic:

- **Finalizers** on the pod/PV/PVC that never get removed because the controller responsible for them is down or buggy — `kubectl get pod <pod> -o jsonpath='{.metadata.finalizers}'`; if genuinely stuck and you understand the risk, `kubectl patch pod <pod> -p '{"metadata":{"finalizers":null}}'` force-removes it (dangerous — can orphan cloud resources like EBS volumes; use as a last resort).
- **Process not honoring SIGTERM** — app traps and ignores it, or PID 1 isn't forwarding the signal to the real process (missing init), so it always runs the full grace period then gets SIGKILLed — that's actually "working," just slow; fix the app's signal handling or add a shorter grace period.
- **Node itself is gone/unreachable** — kubelet that would report completion never checks in; the object stays Terminating until the node is deleted (`kubectl delete node` for a confirmed-dead node) which force-clears its pods.

Q30 Init Container failed.

Init containers run sequentially, and **none** of the main containers start until every init container exits `0`. The pod status shows `Init:Error` or `Init:CrashLoopBackOff`, and `kubectl describe pod` tells you exactly which init container index/name failed.

```
kubectl logs <pod> -c <init-container-name>
kubectl describe pod <pod> | grep -A5 "Init Containers"
```

Typical causes: a dependency-wait script (waiting for a DB/service) timing out because the dependency really is down; a permissions setup step failing (chown/chmod on a mounted volume, common with non-root securityContexts); or a config-fetch step hitting an unreachable ConfigMap/Secret/external endpoint. Fix the specific init container in isolation — you can often reproduce it with `kubectl debug` or by running that image directly with the same env/mounts.

Q31 Sidecar container crashes.

Depends on your Kubernetes version and how the sidecar is defined. With native sidecars (init container with `restartPolicy: Always`, GA since 1.29), a crashing sidecar restarts independently and won't block the main container from running, but a persistently crash-looping sidecar can still starve the pod's resources or, if it does readiness-affecting work (e.g. an Envoy/Istio proxy), make the *pod* unready even though the main app container is healthy.

```
kubectl logs <pod> -c <sidecar-name> --previous
kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[*].name}{"\n"}
{.status.containerStatuses[*].ready}'
```

Diagnose per-container, not per-pod — check which specific container's status shows the crash. Common sidecar-specific causes: a service-mesh proxy failing to fetch its config from the control plane on startup, a log-shipping sidecar losing its output destination, or a shared-volume contract mismatch between sidecar and main container (wrong path, race on startup order without a proper readiness gate between them).

Q32 Readiness probe fails.

The container keeps running — readiness failure never restarts anything. What it does: removes the pod's IP from the Service's Endpoints/EndpointSlice, so it stops receiving traffic until it passes again. This is the "traffic gate," not the "health gate."

```
kubectl describe pod <pod> | grep -A5 Readiness
kubectl get endpoints <service>   # confirm the pod IP dropped out
```

Common causes: the app takes longer to warm up (connection pools, cache load) than the probe's `initialDelaySeconds` allows; a downstream dependency the readiness check verifies (DB, cache) is degraded; or the probe endpoint itself has a bug/deadlock separate from the app's actual serving path. If pods flap in and out of the Service during normal load, that's usually the readiness threshold being too tight (`failureThreshold` / `periodSeconds`) rather than a real health problem.

Q33 Liveness probe fails.

This is the "kill and restart" gate — after `failureThreshold` consecutive failures, the kubelet kills the container (SIGTERM then SIGKILL after grace period) and restarts it. Unlike readiness, a bad liveness probe can create a self-inflicted outage: if the probe checks something unreliable (a downstream dependency, or a slow endpoint under load), the app gets killed precisely when it's under load and needs to stay up, worsening the incident.

```
kubectl describe pod <pod> | grep -A5 Liveness
kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].restartCount}'
```

Best practice, worth stating explicitly in an interview: liveness probes should only check "is this process alive and not deadlocked," never external dependencies — a DB blip shouldn't cause your app pods to restart. If restarts correlate with load spikes, the probe timeout is likely too aggressive relative to real response time under load.

Q34 Startup probe keeps failing.

The startup probe exists specifically to protect slow-starting apps: while it's running, liveness and readiness probes are **disabled**, so a slow app doesn't get killed by liveness before it finishes booting. If the startup probe itself never succeeds within `failureThreshold × periodSeconds`, the kubelet treats the container as failed and kills it — then it restarts and the cycle repeats, looking like a slow-motion CrashLoopBackOff.

```
kubectl describe pod <pod> | grep -A5 Startup
```

Fix by first confirming real startup time (time from process start to actually-ready in logs), then sizing `failureThreshold × periodSeconds` with real margin above that — e.g. 30 × 10s = 5 minutes of allowed startup. If startup time is inconsistent (dependent on cache warm state, cold JVM JIT, etc.), that variability itself is often the real thing to fix, not just the probe budget.

Q35 Pod cannot resolve DNS.

Narrow it methodically — DNS failures inside one pod (versus cluster-wide) point to different layers than cluster DNS being fully down.

```
kubectl exec -it <pod> -- cat /etc/resolv.conf
kubectl exec -it <pod> -- nslookup kubernetes.default
kubectl exec -it <pod> -- nslookup google.com
```

- Internal names fail, external works (or vice versa) — points at CoreDNS's upstream forwarding config or a stub-domain issue, not total DNS failure.
- `/etc/resolv.conf` missing the cluster search domains or `ndots` setting — usually a `dnsPolicy` misconfiguration on the pod (e.g. `dnsPolicy: Default` instead of `ClusterFirst` inherits the node's resolv.conf instead of the cluster's).
- Just this pod/node affected — check kube-proxy rules for the `kube-dns` Service on that node, or NetworkPolicy blocking UDP/TCP 53 to `kube-system`.
- Intermittent, not total — classic `conntrack` UDP DNS race under load; consider NodeLocal DNSCache to remove the iptables SNAT/conntrack race for DNS traffic.

Q36 Pod can't access internet.

Work outward from the pod: DNS resolves but connection fails, or DNS itself fails first? Then check each network layer in order.

```
kubectl exec -it <pod> -- curl -v -m 5 https://example.com
kubectl exec -it <pod> -- nslookup example.com
```

- **NetworkPolicy** with an egress rule that only allows specific destinations (default-deny egress is common in hardened clusters) — check `kubectl get networkpolicy -A`.
- **Private subnet with no NAT Gateway** (EKS) — nodes/pods in a private subnet need a NAT Gateway or VPC endpoint for any internet-bound or AWS-service-bound traffic.
- **Security group** on the node ENI blocking egress on 443/80.
- **Proxy requirements** — some environments require `HTTP_PROXY` / `NO_PROXY` env vars in the pod spec; missing `NO_PROXY` for internal cluster CIDRs can also break internal traffic through a proxy meant only for external calls.
- **Egress-restrictive CNI/firewall** (e.g. Calico global network policy) applied cluster-wide after a security hardening change.

Q37 Pod can't reach another service.

Isolate whether it's DNS, Service routing, NetworkPolicy, or the target pod itself — each has a distinct signature.

```
kubectl exec -it <pod> -- nslookup <target-service>.<namespace>.svc.cluster.local
kubectl get endpoints <target-service> -n <namespace>
kubectl exec -it <pod> -- curl -v <target-service>.<namespace>:<port>
```

- DNS resolves to a ClusterIP but connection times out — check `kubectl get endpoints`; empty endpoints means the Service's label `selector` doesn't match any pod's labels (very common typo-driven bug).
- Endpoints populated but still failing — check target pod readiness (not-ready pods are excluded from Endpoints, so if *all* target pods are unready you'll see this) and NetworkPolicy on the target namespace restricting ingress by label/namespace selector.
- Works sometimes, not always — look at target pod readiness flapping, or one specific backend pod being unhealthy in a load-balanced set.
- Cross-namespace specifically failing while same-namespace works — almost always a NetworkPolicy scoped to same-namespace traffic only.

Q38 Pod works locally but fails inside Kubernetes.

This is almost always an environment-parity gap, not a Kubernetes bug. Walk the differences systematically:

- **Config/secrets** — local `.env` file vs in-cluster ConfigMap/Secret; a value silently differs or is missing.
- **Networking assumptions** — app hardcodes `localhost` for a dependency that's a separate container/pod in-cluster; needs the Service DNS name instead.
- **Filesystem/permissions** — runs as root locally, but the pod's `securityContext` enforces `runAsNonRoot`/read-only root filesystem, and the app tries to write somewhere it can't.
- **Resource limits** — unconstrained locally, throttled/OOMKilled under the pod's requests/limits.
- **Base image differences** — works on your full dev-machine OS, fails on a slim/distroless production image missing a shared library or CA certificate bundle.
- **Startup ordering** — local setup has the DB already running; in-cluster, dependent services start in parallel and the app doesn't retry/backoff on initial connection failure.

Reproduce as close to production as possible: run the exact image (not source) with `docker run` using the same env/resource limits before assuming it's a "Kubernetes issue."

Deployment Problems

Q39 New deployment never becomes Ready.

Walk the object chain top-down — Deployment → ReplicaSet → Pod — since the failure could be at any level:

```
kubectl rollout status deployment/<name>
kubectl get rs -l app=<name>
kubectl get pods -l app=<name>
kubectl describe pod <pod>
```

If pods exist and are Running but the Deployment still isn't Available, it's a **readiness** problem, not a scheduling/crash problem — check the readiness probe and `minReadySeconds`. If pods never get created at all, check the ReplicaSet's events for admission-webhook rejections or a PodDisruptionBudget/ResourceQuota blocking creation. If it's Pending, fall back to the Pod Pending checklist above.

Q40 Deployment rollout stuck.

```
kubectl rollout status deployment/<name> --timeout=10s
kubectl describe deployment <name>   # look at Conditions: Progressing
```

The `Progressing` condition's reason tells the story: `ProgressDeadlineExceeded` means new pods aren't becoming ready within `progressDeadlineSeconds` (default 600s) — almost always a bad new image/config that's crash-looping or failing readiness, so the rollout is correctly refusing to continue replacing old (working) pods with more broken ones. Check the new ReplicaSet's pods directly rather than the Deployment object; the Deployment is just reporting what its child ReplicaSet can't achieve. Also check for a stuck old ReplicaSet still terminating due to a PodDisruptionBudget preventing enough old pods from going away to make room, if `maxUnavailable` is 0 and `maxSurge` can't create enough headroom (resource quota limiting surge capacity).

Q41 Rolling update hangs.

Same failure family as a stuck rollout, but specifically look at the **surge/unavailable math**: with `maxSurge: 25%, maxUnavailable: 25%` on a 4-replica deployment, Kubernetes can create at most 1 extra pod and take down at most 1 old pod at a time. If the new pod never becomes Ready, the rollout can't progress past that first batch — it isn't "hung," it's correctly waiting forever for a pod that will never pass its readiness probe.

```
kubectl get rs -l app=<name> -o wide   # compare desired/current/ready per RS
kubectl rollout history deployment/<name>
```

Fix: diagnose why the new pod fails readiness (this is where the actual bug lives), then either fix forward or `kubectl rollout undo deployment/<name>` to unblock immediately while you fix the image/config offline.

Q42 ReplicaSet created but Pods aren't.

```
kubectl describe rs <replicaset-name>   # Events section
```

The ReplicaSet controller creates Pod objects, but the API can reject the actual creation — the RS's events show why. Common causes: a **ResourceQuota** in the namespace blocking new pod objects once a limit is hit; a **validating/mutating admission webhook** rejecting the pod spec (policy engine like Kyverno/OPA Gatekeeper denying it, or a broken webhook endpoint causing every create to fail closed); a **PodSecurity admission** label on the namespace rejecting a spec that violates the enforced level (e.g. privileged containers under a `restricted` namespace); or exceeding the **max pods per node** ceiling cluster-wide with no schedulable capacity, though that usually shows Pending pods rather than zero pods at all.

Q43 Rollback failed.

```
kubectl rollout history deployment/<name>
kubectl rollout undo deployment/<name> --to-revision=<N>
kubectl rollout status deployment/<name>
```

A "failed" rollback is really just another rollout, so it fails for the same reasons a forward rollout fails — the revision you're rolling back to isn't actually healthy anymore (its image was deleted from the registry, or a dependency it needs, like an old API version or a ConfigMap key, no longer exists). Also check `revisionHistoryLimit` — if it's set too low, the revision you want to roll back to may have already been garbage-collected and no longer exists in `rollout history` at all, which looks like "rollback failed" but is really "target revision doesn't exist." In that case you need the old manifest/image tag from your GitOps history (ArgoCD/Helm release history) rather than `kubectl rollout undo`.

Q44 Deployment constantly restarting.

Distinguish **container restarts** (rising `RESTARTS` count in `kubectl get pods`, same pod object) from **pod replacement churn** (new pod names/UIDs constantly) — they point to different causes.

- Rising restart count, same pod — liveness probe misconfigured (see Q33), or an actual app crash (CrashLoopBackOff, see Q23).
- New pods constantly created/destroyed — something external is churning them: a controller reconciling against a config that keeps changing (GitOps drift-fighting, or an HPA thrashing between scale up/down because its metric is noisy), a competing tool managing the same Deployment (e.g. both Argo CD and a manual `kubectl edit`, or Argo CD's self-heal fighting a webhook mutation), or a node group with aggressive Spot interruptions constantly evicting and rescheduling.

```
kubectl get events --field-selector involvedObject.kind=Pod --sort-by=.lastTimestamp | tail -30
```

Q45 Wrong image deployed.

```
kubectl get deployment <name> -o jsonpath='{.spec.template.spec.containers[0].image}'
kubectl describe pod <pod> | grep Image:
```

Root causes, in order of how often they actually happen: **using** `:latest` (or any mutable/floating tag) so the manifest didn't change but the tag now points at different content — `imagePullPolicy: IfNotPresent` can then serve a stale cached image on some nodes and the new one on others, giving mixed versions across the same Deployment; a **CI/CD pipeline race** where two builds pushed to the same tag concurrently; a **GitOps sync** that hasn't actually run yet (Argo CD shows OutOfSync, but the person assumed it auto-deployed); or a **values/overlay mismatch** in Helm/Kustomize where the wrong environment overlay got applied. This is the canonical argument for immutable, digest-pinned or semver-tagged images in production, never `:latest`.

Q46 Secret missing after deployment.

```
kubectl get secret <name> -n <namespace>
kubectl describe pod <pod> | grep -A3 -i secret
```

Check namespace first — Secrets are namespace-scoped, and a very common mistake is a Secret created in the wrong namespace (or in the old namespace during a migration) while the Deployment moved. If it's genuinely gone: an **External Secrets Operator / Sealed Secrets** sync failure (check the operator's own logs, since the Secret is only a projection of an external source of truth like AWS Secrets Manager or Vault); a **Helm/Kustomize apply** that pruned it because it was removed from the chart/overlay (pruning can delete resources not currently declared); or a **TTL/rotation job** that deleted-and-is-still-recreating it. If the pod is already running and referencing the Secret as an env var (not a mounted volume), note that env-var Secrets are only read at container start — recreating the Secret won't update a running pod; it needs a restart.

Q47 ConfigMap updated but application still uses old values.

This is expected behavior, not a bug, for the common case: **env vars sourced from a ConfigMap are only read once, at container start** — updating the ConfigMap doesn't push new values into an already-running process. Mounted ConfigMap *volumes* do update on the filesystem (via kubelet sync, typically within ~1 minute, subject to `sync-frequency` and cache propagation delay), but the application itself still has to notice the file changed and reload — most apps don't watch their config files for changes, so even a volume mount often needs an app-level restart or hot-reload logic.

```
kubectl rollout restart deployment/<name>
```

Production pattern worth mentioning: add a checksum/hash of the ConfigMap content as a pod template annotation (common in Helm charts via `checksum/config`) so a ConfigMap change automatically triggers a rolling restart through the normal Deployment mechanism, rather than relying on in-place file updates being noticed.

Q48 Application serves old version after deployment.

Work backward through every caching layer between the client and the new pod, since the Deployment itself may be perfectly correct:

- **CDN/edge cache** (CloudFront, etc.) serving a cached response — check cache headers and invalidation, not the cluster.
- **Old pods still in the Service's Endpoints** — a slow/stuck rollout (see above) means both old and new versions are live and load-balanced simultaneously; client just happened to hit an old one.
- **Client-side connection keep-alive / DNS caching** — a long-lived client connection to a specific pod IP via a LoadBalancer won't automatically move to a new pod.
- **Image actually didn't change** — see Q45 (mutable tag, stale cache on the node with `imagePullPolicy: IfNotPresent`).
- **Wrong environment** — deployed to staging/wrong cluster context by mistake; verify `kubectl config current-context` and the actual running image digest, not just "I ran the deploy command."

Networking

Q49 Service not routing traffic.

```
kubectl get svc <name> -o wide
kubectl get endpoints <name>
kubectl describe svc <name>
```

Start at Endpoints, not the Service — an empty Endpoints/EndpointSlice list is the single most common root cause and it isolates the problem instantly: it means the Service's `selector` doesn't match any pod's labels (typo, or labels changed on a redeploy without updating the Service), or every matching pod is failing readiness. If Endpoints are populated correctly but traffic still doesn't land, move to kube-proxy: confirm its pods are healthy on the relevant nodes and that iptables/IPVS rules exist for the Service's ClusterIP (`iptables -t nat -L | grep <service-name>`). Also check port mismatches — `targetPort` not matching the container's actual listening port is a frequent, easy-to-miss cause that produces exactly this symptom.

Q50 ClusterIP unreachable.

ClusterIP is a virtual IP that only means anything through kube-proxy's rules — it isn't assigned to any real interface, so "unreachable" almost always means kube-proxy didn't program it, or you're trying to reach it from somewhere kube-proxy doesn't cover (e.g. from outside the cluster, where ClusterIP was never meant to be reachable at all — that's the single most common "bug report" here that isn't a bug).

```
kubectl exec -it <any-pod> -- curl -v <clusterip>:<port>
iptables -t nat -L KUBE-SERVICES -n | grep <clusterip>
```

If unreachable even from inside the cluster: check kube-proxy is running on the source node, confirm Endpoints aren't empty, and check for a NetworkPolicy default-deny without an explicit allow for that traffic.

Q51 NodePort inaccessible.

NodePort opens the same port (default range 30000–32767) on **every** node, regardless of whether that node runs a matching pod — kube-proxy forwards internally to wherever the pod actually lives. So "inaccessible" usually isn't about which node you hit; check these instead:

- **Security group** on the EC2 instances not allowing inbound on the NodePort range from wherever you're connecting.
- **Testing from outside** the VPC without a route/public IP to the node at all.
- **Endpoints empty** for the backing Service (same root cause as ClusterIP issues).
- **Firewall/OS-level** rule on the node itself (rare, but possible with custom AMIs).

```
curl -v <node-external-ip>:<nodeport>
```

Q52 LoadBalancer pending forever.

`EXTERNAL-IP` stuck on `<pending>` means whatever is supposed to provision the cloud load balancer never did. Check who's responsible first — in EKS that's the **AWS Load Balancer Controller** or the legacy in-tree/cloud-provider integration, and if that controller isn't installed/running at all, Services of type LoadBalancer simply never get an IP, silently.

```
kubectl describe svc <name>   # Events section
kubectl -n kube-system logs -l app.kubernetes.io/name=aws-load-balancer-controller --tail=50
```

Common causes: the controller isn't deployed or is crashing; missing IAM permissions (IRSA role lacking ELB/EC2 permissions) so the controller's API calls fail silently in its own logs; missing required **subnet tags** (`kubernetes.io/role/elb` for public, `kubernetes.io/role/internal-elb` for internal — a very common EKS gotcha); or an **ELB service quota** exhausted in the account/region.

Q53 Ingress returns 404.

A 404 from the ingress controller itself (check response headers/body for `nginx` vs your app's own 404 page — they look different and mean very different things) means the request reached the controller but didn't match any configured routing rule — the request never even got to your app.

```
kubectl describe ingress <name>   # confirm host/path rules and backend
kubectl get ingressclass
curl -v -H "Host: your.domain.com" http://<ingress-controller-ip>/path
```

Common causes: `host` in the Ingress rule doesn't match the request's `Host` header exactly (case-sensitive, no trailing dot mismatches); `path` / `pathType` mismatch (`Exact` vs `Prefix` behaving differently than expected); multiple Ingress objects with conflicting rules and no clear precedence; or the Ingress referencing a Service/port that doesn't exist, which some controllers surface as a generic 404 rather than a 503.

Q54 Ingress returns 502.

502 means the ingress controller matched a rule and tried to proxy to a backend, but the backend connection itself failed — this is a downstream problem, not a routing problem, which is the key distinction from a 404.

```
kubectl -n ingress-nginx logs -l app.kubernetes.io/name=ingress-nginx --tail=100
kubectl get endpoints <backend-service>
```

Common causes: backend pods are unready/crashing (empty or all-unhealthy Endpoints); the app's actual listening port doesn't match the Service's `targetPort`; the backend pod is overwhelmed and refusing/resetting connections under load; a mismatched protocol (Ingress configured for HTTP but backend expects gRPC/HTTP2 without the right annotation, e.g. `nginx.ingress.kubernetes.io/backend-protocol: "GRPC"`); or a keep-alive/timeout mismatch between the ingress controller and backend causing connection resets mid-request under load.

Q55 TLS handshake failed.

```
openssl s_client -connect <host>:443 -servername <host>
kubectl get secret <tls-secret> -o jsonpath='{.data.tls\.crt}' | base64 -d | openssl x509 -noout -dates -subject
```

Narrow down the layer: **expired certificate** (check `notAfter` above — most common cause by far, especially when cert-manager renewal silently failed); **SNI mismatch** — the server presented a cert for a different hostname than requested, often because the Ingress's `tls.hosts` doesn't match the actual request Host, or SNI routing sent the connection to the wrong backend; **protocol/cipher mismatch** — an old client or a strict TLS policy (minimum TLS 1.2/1.3 enforced) rejecting each other; **incomplete certificate chain** — missing intermediate cert in the `tls.crt` Secret, which works in some clients (that cache intermediates) and fails in strict ones; or **SSL passthrough vs termination mismatch** — the ingress terminating TLS when the backend expected passthrough, or vice versa.

Q56 NGINX ingress crashes.

```
kubectl -n ingress-nginx logs <pod> --previous
kubectl -n ingress-nginx describe pod <pod>
```

Common causes: an invalid or conflicting annotation on some Ingress object causing the controller to generate a broken nginx.conf and fail to reload (check logs for `nginx: [emerg]` config errors around the time of the crash, often traceable to whichever Ingress was just created/edited); OOMKilled under a very large number of Ingress objects/hosts (nginx.conf size and reload cost both scale with Ingress count, so this shows up on high-tenant clusters); a custom `ConfigMap` for the controller with an invalid directive; or the admission webhook validating new Ingress objects itself becoming unreachable, which can cascade into apply failures across the cluster, not just the controller pod. Isolate by checking whether the crash correlates with a specific Ingress change — `kubectl get ingress -A --sort-by=.metadata.creationTimestamp` around the crash timestamp.

Q57 AWS Load Balancer unhealthy.

```
aws elbv2 describe-target-health --target-group-arn <arn>
```

The reported reason code narrows it immediately: `Target.FailedHealthChecks` — the health check path/port doesn't match what the pod actually serves (check the Ingress/Service annotations for `healthcheck-path` / `healthcheck-port` against the app's real health endpoint); `Target.Timeout` — app too slow to respond within the configured timeout, or a security group blocking the health-check port specifically; `Target.NotRegistered` — with IP-mode target groups (used for direct pod-IP routing via the AWS Load Balancer Controller), pods aren't being registered at all, often an IAM/controller issue; and for **instance-mode** target groups, an unhealthy target can simply mean no matching pod is currently scheduled on that particular node, which is normal churn, not an incident, provided other targets are healthy.

Q58 Cross-node communication fails.

Same-node pod-to-pod usually still works (local bridge/veth), so "only cross-node fails" is a strong, specific signal — it isolates the problem to whatever carries traffic *between* nodes.

```
kubectl exec -it <pod-on-node-a> -- ping <pod-ip-on-node-b>
kubectl -n kube-system logs <cni-pod-on-node-b>
```

- **Security group** rules blocking the CNI's overlay/encapsulation traffic between node ENIs (for VPC CNI, pod IPs are real VPC IPs so it's usually the node-to-node SG rule itself, not an overlay port).
- **Route table** missing routes for the pod CIDR of the other node (relevant for CNIs that use per-node pod subnets, like some VPC CNI custom-networking setups).
- **MTU mismatch** between nodes/AZs causing large-packet drops while small packets (like ICMP ping) succeed — a classic false-negative trap; test with a real payload size, not just ping.
- **Different AZ/subnet** with a missing peering or transit gateway route in more complex network topologies.

Q59 DNS lookup fails.

Covered in depth under Pod Issues (Q35), but at the platform/cluster level specifically, also check: CoreDNS `Corefile` syntax after any edit (a bad edit crash-loops every CoreDNS replica at once, unlike a single-pod DNS issue); CoreDNS replica count vs cluster size (default 2 replicas can bottleneck on very large/high-QPS clusters, showing up as *slow* or intermittently failing lookups rather than total failure); and the CoreDNS Service's `ClusterIP` being hardcoded into node kubelet config (`--cluster-dns`) — if that Service is ever deleted and recreated with a different IP, every node's DNS silently breaks until kubelet config is corrected.

Q60 NetworkPolicy blocks traffic.

```
kubectl get networkpolicy -n <namespace> -o yaml
```

Remember the default-deny mental model: the moment *any* NetworkPolicy selects a pod for a given direction (ingress or egress), that direction becomes deny-all-except-explicitly-allowed for that pod — policies are additive/whitelist, never subtractive, and there's no way to write a policy that "blocks" traffic that isn't otherwise already restricted by another policy selecting the same pod.

Debug by checking, per direction: does any policy's `podSelector` match this pod at all? If yes, does at least one policy's `ingress` / `egress` rule explicitly allow the specific traffic (by `podSelector`, `namespaceSelector`, or `ipBlock`, and the right port)? A very common gotcha: a namespace-scoped policy allowing traffic "from the same namespace" doesn't automatically allow traffic from `kube-system` (e.g. an ingress controller or DNS) unless explicitly permitted — teams enabling default-deny often break DNS or health checks by accident this way.

Q61 Pods communicate intermittently.

Intermittent is the hardest category because it usually implicates a *subset* of backends or a *race* rather than a config that's simply wrong everywhere. Suspects, roughly by frequency:

- **One bad backend pod** among several behind a Service — load-balanced traffic only fails when it lands on that one; check per-pod error rates, not aggregate.
- **Readiness flapping** under load, pods bouncing in/out of Endpoints.
- **Conntrack table exhaustion** on busy nodes (`nf_conntrack: table full` in `dmesg`) causing random packet drops under high connection churn — common with lots of short-lived connections (e.g. DNS-over-UDP at scale).
- **kube-proxy iptables rule staleness** during rapid pod churn (a brief window where rules reference a just-deleted pod IP).
- **MTU/fragmentation** issues that only manifest for larger payloads, not small test pings.

Correlate failures against pod restarts, deploys, and node CPU/conntrack metrics on a timeline rather than trying to reproduce it on demand.

Q62 CoreDNS restart loop.

```
kubectl -n kube-system logs -l k8s-app=kube-dns --previous
kubectl -n kube-system get cm coredns -o yaml
```

Most common cause by far: a **syntax error or a loop in the** `Corefile` after a manual edit — CoreDNS's own `loop` plugin actually detects forwarding loops (e.g. it forwards to itself via a misconfigured upstream) and deliberately crashes to avoid an infinite request storm, which is a feature, not a bug, but confuses people expecting it to just "handle" the loop. Other causes: OOMKilled from undersized memory limits on a cluster with heavy DNS QPS; a missing/unreachable upstream resolver referenced in `forward` that causes cascading timeouts and liveness-probe failures; or a botched CoreDNS version upgrade with a breaking Corefile syntax change between versions. Fix by reverting the ConfigMap to the last-known-good version first (fastest path to recovery), then diagnose the actual config change offline.

Q63 CNI plugin failure.

```
kubectl -n kube-system get pods -o wide | grep -E 'aws-node|calico|cilium'
kubectl describe pod <failing-pod> | grep -i "failed to setup network"
```

A CNI failure shows up as pods stuck in `ContainerCreating` with a sandbox-network error, since the CNI plugin is what actually assigns the pod its IP and wires its interface. On EKS with VPC CNI specifically: check **IP/ENI exhaustion** first (`kubectl -n kube-system logs <aws-node-pod>` for `InsufficientCidrBlocks` or similar), then whether the `aws-node` DaemonSet pod on that specific node is even Running, then IAM permissions for the CNI's node role. On Calico/Cilium: check the agent pod's own health/readiness and whether BGP peering (Calico) or the eBPF datapath (Cilium) is actually up on that node — a CNI agent that's Running but not fully initialized still blocks pod networking on that node.

Storage

Q64 PVC Pending.

```
kubectl describe pvc <name>   # Events section has the exact reason
kubectl get storageclass
```

With dynamic provisioning (the normal case), Pending means the **CSI provisioner** hasn't created a backing volume yet. Common causes: the referenced `StorageClass` doesn't exist (typo, or it was deleted); the CSI controller pod itself is down/crashing (check `kubectl -n kube-system get pods -l app=ebs-csi-controller` for EBS); an EBS/EFS **service quota** hit in the account; requested `accessModes` not supported by the StorageClass's provisioner (e.g. asking for `ReadWriteMany` from an EBS-backed class, which only supports `ReadWriteOnce` — a very common mismatch); or, for topology-aware provisioning, no node in the PVC's required zone yet (waits for a consumer pod to be scheduled first if `volumeBindingMode: WaitForFirstConsumer`, which is correct behavior, not a bug).

Q65 PV not binding.

For statically-provisioned PVs (pre-created by an admin rather than dynamically), binding to a PVC requires an exact match on several fields simultaneously: `storageClassName` (empty string and unset are treated differently — a classic gotcha), `accessModes`, and `capacity` (the PVC's requested size must be ≤ the PV's capacity, and the PV must not already be `Bound` to a different claim).

```
kubectl get pv <name> -o yaml | grep -A5 spec
kubectl get pvc <name> -o yaml | grep -A5 spec
```

Also check `claimRef` on the PV — if it was previously bound and released, it may still carry a stale `claimRef` pointing at the old claim, which blocks it from binding to a new one until that's cleared (relevant with `persistentVolumeReclaimPolicy: Retain`, where the PV isn't deleted or auto-rebound after its PVC is deleted).

Q66 StorageClass missing.

```
kubectl get storageclass
kubectl get storageclass -o jsonpath='{.items[?(@.metadata.annotations.storageclass\.kubernetes\.io/is-default-class=="true")].metadata.name}'
```

If a PVC doesn't specify `storageClassName` at all, it relies on whichever StorageClass is annotated as the cluster default — if none is marked default (or two are, which is also invalid), dynamic provisioning silently never happens and the PVC sits Pending with no obvious error pointing at "no default StorageClass." This is common right after standing up a fresh EKS cluster, since EKS doesn't ship a default gp2/gp3 StorageClass automatically in newer setups the way some other distributions do — you need to install the EBS CSI driver as an add-on and create/mark a default StorageClass explicitly.

Q67 VolumeMount failed.

```
kubectl describe pod <pod> | grep -A5 "MountVolume"
journalctl -u kubelet | grep <pod-uid>
```

Distinguish attach failures (cloud-level, e.g. EBS can't attach to the instance) from mount failures (the volume is attached to the node but the filesystem mount inside the pod's namespace fails). Common mount-level causes: the volume's filesystem is corrupted or was never formatted (first-use edge case with some CSI drivers); a path in `subPath` doesn't exist inside the volume; SELinux/AppArmor context mismatch on hardened nodes; or the volume is still mounted read-write on a different node from a previous pod that wasn't cleanly terminated (see Multi-Attach error under EBS/EFS below) — single-attach volumes can't be mounted by two kubelets at once, and the second one just fails and retries.

Q68 EBS volume not attaching.

```
aws ec2 describe-volumes --volume-ids <vol-id> --query 'Volumes[].Attachments'
kubectl -n kube-system logs -l app=ebs-csi-controller -c csi-provisioner
```

Root causes, most to least common: **volume and node in different Availability Zones** (EBS volumes are AZ-locked; this is the single most common EBS attach failure and usually traces to a topology/zone-affinity mismatch between where the pod got scheduled and where the volume was provisioned); **per-instance volume attach limit reached** (varies by instance type, roughly 25–28 EBS volumes on many instance families, easy to hit on dense bin-packed nodes); **IAM permissions** missing on the CSI controller's IRSA role for `ec2:AttachVolume`; or the volume already attached elsewhere and not cleanly detached after a node failure (force-detach may be needed: `aws ec2 detach-volume --force`, understanding the data-integrity risk if the old node wasn't actually dead).

Q69 EFS mount failed.

```
kubectl -n kube-system logs -l app=efs-csi-node
telnet <efs-mount-target-ip> 2049
```

Unlike EBS, EFS is network-attached (NFS over port 2049) and not AZ-locked, so the failure modes are different: **mount target's security group** not allowing inbound 2049 from the node's security group (the single most common cause); no **mount target in the AZ** the node lives in (EFS needs a mount target per AZ you use); DNS resolution failure for the EFS mount helper's hostname if VPC DNS resolution is disabled; or the **EFS access point** (if used) having a POSIX user/permission configuration the pod's `securityContext` can't satisfy. EFS mounts tend to fail slow (long hangs) rather than fast, since it's a network mount with retries — that's a useful tell versus EBS's fast attach-level failures.

Q70 StatefulSet lost storage.

StatefulSet pods keep a stable identity-to-PVC binding (`<volumeClaimTemplate>-<statefulset>-<ordinal>`), so "lost storage" usually means one of these broke that binding rather than data actually vanishing:

- The **PVC itself was deleted** — StatefulSets don't delete their PVCs automatically by default when a pod/StatefulSet is deleted (this is intentional, to protect data), but someone or some automation (e.g. an overly aggressive Helm uninstall, or explicit `persistentVolumeClaimRetentionPolicy: Delete` in newer Kubernetes) removed it.
- The pod was **rescheduled to a different AZ** than its AZ-locked EBS volume, so it can never mount it again on the new node — check zone-aware scheduling/topology constraints for the StatefulSet.
- The **underlying PV was deleted** directly (with `Delete` reclaim policy, deleting the PVC also deletes the cloud volume itself — unrecoverable without a snapshot).

Recovery, if the PV/EBS volume itself still exists, is to recreate a PVC that binds to the exact same PV (static binding via matching `claimRef` / `volumeName`) rather than letting it dynamically provision a new empty one.

Q71 Disk corruption.

```
dmesg -T | grep -iE 'ext4-fs error|xfs|i/o error'
fsck -n /dev/<device>   # dry-run first, never fsck a mounted filesystem live
```

At the node level: check for underlying EBS volume issues (degraded volume, or an unclean shutdown that skipped a proper filesystem unmount — e.g. a Spot interruption or hard instance termination mid-write). At the application level: a StatefulSet workload (databases especially) writing through a filesystem that doesn't guarantee the durability semantics the app assumes (e.g. missing `fsync` discipline, or a volume type without the IOPS/consistency guarantees the workload needs). Recovery path depends entirely on whether you have snapshots — this is the strongest argument for scheduled EBS snapshots or application-level backups (e.g. `pg_basebackup`, Velero) rather than treating the PV as durable by default; cordon/drain the node, don't attempt live repair on a mounted production filesystem, and restore from the last known-good backup/snapshot if corruption is confirmed.

Q72 Read-only filesystem.

Two very different causes produce the identical symptom — check which one first, since the fix is opposite in each case:

- **Intentional** — `securityContext.readOnlyRootFilesystem: true` is set (a hardening best practice); the fix isn't to remove it, it's to mount an explicit writable volume (`emptyDir`, typically) at exactly the paths the app needs to write to (temp files, caches, sockets) alongside the read-only root.
- **Unintentional** — the kernel remounted the filesystem read-only itself after detecting an I/O error (check `dmesg` for `Remounting filesystem read-only`), which is the kernel protecting against further corruption after a disk/EBS problem — this is really the "Disk corruption" scenario above wearing a different symptom.

```
kubectl exec -it <pod> -- mount | grep " / "
kubectl get pod <pod> -o jsonpath='{.spec.containers[0].securityContext}'
```

Q73 CSI Driver failure.

A CSI driver has two halves that fail independently — know which one you're debugging: the **controller** (a Deployment, handles provisioning/attaching, cluster-wide) versus the **node plugin** (a DaemonSet, handles mounting on each node).

```
kubectl -n kube-system get pods -l app=ebs-csi-controller
kubectl -n kube-system get pods -l app=ebs-csi-node
kubectl -n kube-system logs <csi-pod> -c ebs-plugin
```

Controller failures block new provisioning/attaching cluster-wide (new PVCs stuck Pending) but don't affect already-mounted volumes. Node-plugin failures on a specific node block mounting/unmounting *on that node only* — existing mounts elsewhere are unaffected. Common causes for either: IRSA/IAM permission gaps after a policy change, a version skew between the CSI driver and the cluster's Kubernetes version after an upgrade, or the driver's own webhook/CRDs (`VolumeAttachment`, `CSINode`) in a bad state. Check `kubectl get volumeattachment` to see the CSI's own view of what's attached where.

Resource Issues

Q74 Pods consuming too much CPU.

```
kubectl top pods --all-namespaces --sort-by=cpu | head -20
kubectl exec -it <pod> -- top -H
```

Decide first whether it's a **missing/wrong limit** (letting one pod monopolize a node) or **genuine load** that needs scaling, since the fixes are opposite. If limits exist and it's still high, profile inside the container (language-specific: `pprof` for Go, async-profiler/JFR for JVM, `py-spy` for Python) rather than guessing — a hot loop, an inefficient query pattern retried in a tight loop, or GC pressure from oversized allocations are the usual suspects. Also check for CPU throttling as a *cause* of apparent high usage — a throttled pod can look like it's "using a lot" while actually just spinning inefficiently around blocked work; correlate with `container_cpu_cfs_throttled_periods_total`.

Q75 Memory leak.

The Kubernetes-level signature is distinctive: memory usage climbs steadily and monotonically over hours/days rather than following request load, eventually hitting the limit and getting OOMKilled, restarting, then climbing again — a very recognizable sawtooth in a memory-over-time graph.

```
kubectl top pod <pod> --containers
# Chart memory over time via Prometheus:
container_memory_working_set_bytes{pod="<pod>"}
```

Confirm it's a real leak (not just a growing cache that's actually fine, or a language runtime that legitimately holds more heap under sustained load before GC reclaims it) by checking whether memory *ever* comes back down under low load. Once confirmed, this is an application-debugging problem, not a Kubernetes one — heap dump / profiler is the next step, not adjusting limits. Setting a lower memory limit only makes it OOMKill (and restart, clearing the leak) more frequently; it's a mitigation, never a fix.

Q76 High iowait.

```
top   # look at %wa
iostat -x 2 5
kubectl exec -it <pod> -- iostat -x 1 3
```

High `%wa` means CPUs are idle specifically because they're waiting on disk I/O to complete — the fix is almost never "add more CPU." Check what's actually generating the I/O: a noisy-neighbor pod doing heavy disk writes on shared node-local storage (no `ephemeral-storage` limits to contain it); an EBS volume under-provisioned for its IOPS/throughput needs relative to actual load (gp3 lets you provision IOPS/throughput independently of size — a common fix); swapping (Kubernetes disables swap by default, but if it's enabled on the node, memory pressure can manifest as iowait); or a database/log-heavy workload that needs a faster volume type (io2) rather than general-purpose gp3.

Q77 High load average.

Load average counts processes waiting for *any* resource (CPU, disk I/O, uninterruptible sleep), not just CPU demand — so a high number alone doesn't tell you what's actually saturated. Always read it alongside `%wa`, CPU utilization, and run-queue length rather than in isolation.

```
uptime
vmstat 1 5   # 'r' column = runnable processes, 'b' = blocked
kubectl top node
```

High load with low CPU usage and high `%wa` → I/O-bound (see previous question). High load with high CPU usage → genuinely CPU-bound, likely overcommitted pods or undersized nodes. High load with normal CPU/IO → check for processes stuck in uninterruptible sleep (`D` state in `ps aux`), often a hung NFS/EFS mount or a kernel-level driver issue.

Q78 CPU throttling.

The CFS (Completely Fair Scheduler) quota system enforces CPU **limits** in fixed time windows (default 100ms period); if a container uses its whole allotted quota before the period ends, it's frozen until the next period — even if the node has plenty of idle CPU sitting right there unused. This is the single most under-diagnosed latency cause in production Kubernetes: p99 latency spikes that don't correlate with visible CPU-percentage usage.

```
# The metric that actually reveals it (CPU % alone won't):
rate(container_cpu_cfs_throttled_periods_total[5m])
  / rate(container_cpu_cfs_periods_total[5m])
```

Fix: raise the CPU limit (or remove it and rely on requests + node-level capacity for isolation, a legitimate and increasingly common pattern for latency-sensitive services), or for a quick multi-threaded-app mitigation, ensure the language runtime's thread pool is sized against the *limit*, not the node's full core count (a very common Java/Node.js gotcha where the runtime auto-detects host CPU count instead of the cgroup quota).

Q79 HugePage issues.

HugePages are a distinct resource type from regular memory in Kubernetes (`hugepages-2Mi` / `hugepages-1Gi`) requested explicitly in the pod spec — they must be reserved at the OS level on the node *before* any pod can consume them, and unlike regular memory, the kernel won't reclaim/swap them under pressure.

```
cat /proc/meminfo | grep -i huge
kubectl describe node <node> | grep hugepages
```

Common failure: a pod requests `hugepages-2Mi` but the node has zero pre-allocated (needs `vm.nr_hugepages` set via kernel boot params or sysctl at node-bootstrap time, which requires either a custom AMI/launch template userdata or a privileged DaemonSet) — the pod stays Pending exactly like an insufficient-CPU/memory scheduling failure, but the "Insufficient hugepages-2Mi" message is easy to miss if you're not specifically looking for it. This mostly comes up with high-performance workloads (DPDK networking, certain databases, ML workloads) rather than typical microservices.

Q80 ResourceQuota exceeded.

```
kubectl describe resourcequota -n <namespace>
kubectl describe quota <name> -n <namespace>   # Used vs Hard
```

A namespace-level ResourceQuota rejects new object creation outright once the limit is hit — the API call itself fails with a clear `exceeded quota` error, so this rarely needs deep diagnosis, but the impact is easy to misdiagnose as something else (a Deployment scale-up silently produces zero new pods, and without checking the ReplicaSet events you might chase a scheduling problem that doesn't exist). Once a compute-resource quota is set on a namespace, **every pod in it must specify requests/limits** for the resources the quota governs, or its creation is rejected entirely — a common surprise the first time a team adds a ResourceQuota to a namespace that previously had unconstrained pods. Fix is either raising the quota (capacity planning conversation) or freeing it up by cleaning up unused objects (old ReplicaSets, completed Jobs still holding their share).

Q81 LimitRange issues.

```
kubectl describe limitrange -n <namespace>
```

LimitRange sets per-object defaults and min/max bounds within a namespace — it's easy to confuse with ResourceQuota (namespace-aggregate) since they're often used together but operate at different scopes (LimitRange = per-pod/container, ResourceQuota = namespace-wide sum). Two common gotchas worth stating explicitly: (1) if you set requests but not limits on a container, and a LimitRange defines a default `limit` , that default gets silently injected — your pod ends up with a limit you never wrote, sometimes causing unexpected throttling or OOMKills; (2) if a container's explicit request/limit falls outside the LimitRange's `min` / `max` bounds, pod **creation is rejected** with a clear admission error — if a whole Deployment suddenly can't create any pods after a namespace policy change, check LimitRange before assuming it's a scheduling problem.

Q82 HPA not scaling.

```
kubectl describe hpa <name>   # Events + current/target metrics
kubectl get apiservices | grep metrics
```

Checklist, in the order the HPA controller itself actually needs things to work: (1) **metrics-server** (or a custom/external metrics adapter for non-CPU/memory metrics) must be installed and healthy — `kubectl top pods` failing means the HPA has no data at all and silently does nothing; (2) target pods must have **resource requests set** — HPA computes utilization as a percentage of the request, so a pod with no CPU request has no denominator and can't be scaled on CPU; (3) check `describe hpa` 's `Conditions` for `ScalingActive: False` and its reason; (4) **stabilization windows** (`behavior.scaleDown.stabilizationWindowSeconds` , default 300s for scale-down) intentionally delaying action, which looks like "not scaling" but is working as designed; (5) already at `maxReplicas` , or blocked by a ResourceQuota/cluster capacity ceiling on the scale-up side.

Q83 VPA recommendation wrong.

```
kubectl describe vpa <name>   # Recommendation section: Target/Lower/Upper bound
```

VPA's recommendations are built from a historical usage model, so "wrong" recommendations are usually a data/timing problem rather than a bug: too little history yet (VPA needs meaningful time — ideally days, spanning normal peak/trough cycles — to build a reliable percentile model; a fresh install or a recently-changed workload will recommend based on incomplete data); a workload with genuinely **bursty, non-representative usage** (a batch job that spikes briefly skews the percentile-based recommendation upward for what's normally a light workload); or running **VPA and HPA on the same metric** (CPU) simultaneously, which is explicitly unsupported and produces fighting/unstable recommendations — if both are needed, scope HPA to CPU/custom metrics and VPA to memory only, or don't combine them on the same resource dimension. Also confirm the VPA's `updateMode` (`Off` just recommends, `Auto`/`Recreate` actually evicts and resizes pods) matches what you expect it to be doing.

Logging & Monitoring

Q84 Prometheus targets missing.

Check the Prometheus UI's **Targets** page (or `/api/v1/targets`) first — it tells you whether a target is known-but-down (service discovery found it, scrape fails) versus never-discovered (service discovery itself didn't find it), which are very different problems.

```
kubectl get servicemonitor -A   # if using Prometheus Operator
kubectl get endpoints <target-service>
```

Never-discovered: a missing/mismatched `ServiceMonitor`/`PodMonitor` label selector, or the Prometheus Operator's own `serviceMonitorSelector` not including the namespace/labels the ServiceMonitor uses. Known-but-down: the target's `/metrics` endpoint isn't reachable (wrong port, NetworkPolicy blocking Prometheus's namespace, or the app doesn't actually expose metrics on the annotated port) — check the specific scrape error shown in the Targets UI rather than guessing.

Q85 Metrics missing.

```
curl localhost:9090/api/v1/query?query=up{job="<job>"}
promtool query instant http://localhost:9090 'up'
```

If the target shows `up` but a specific metric is missing, it's usually one of: the metric name changed between app/library versions (check the app's `/metrics` endpoint directly with `curl` to confirm it's actually being exposed); a relabeling rule in the scrape config dropping it (`metric_relabel_configs` with an unintended `drop` action); the metric only appears after a certain code path executes (e.g. an error-counter that's genuinely zero-cardinality until the first error, which some client libraries don't pre-initialize); or cardinality limiting/sample limits on the scrape config silently truncating a high-cardinality metric set. Always confirm at the source (`curl the /metrics endpoint`) before assuming Prometheus itself is the problem.

Q86 Alertmanager not sending alerts.

Trace the pipeline in order — each stage has its own failure mode and its own place to check: **Prometheus rule evaluation** (is the alert even firing? `/alerts` page in Prometheus, state should be `firing` not stuck `pending`) → **Prometheus → Alertmanager delivery** (check Prometheus's own `alertmanager` targets and its logs for delivery errors) → **Alertmanager routing** (`amtool config routes test` to verify your route tree actually matches the alert's labels — a very common bug is a route with matchers that don't match the alert's actual label set, silently falling through to a catch-all or nowhere) → **inhibition/silence rules** (an active silence or inhibition rule suppressing it, intentionally or by accident) → **receiver delivery** itself (webhook/PagerDuty/Slack integration credentials expired or endpoint unreachable — check Alertmanager's own logs for the delivery attempt and its HTTP response code).

Q87 Grafana dashboard blank.

Isolate whether it's a **datasource** problem or a **query/dashboard** problem — test the datasource connection in Grafana's datasource settings page first (Grafana has a built-in "Test" button that confirms basic connectivity independent of any specific dashboard).

```
-- Run the panel's exact PromQL directly against Prometheus, bypassing Grafana entirely
up{job="my-app"}
```

If the raw query returns data in Prometheus/the datasource directly but the panel is still blank: check the dashboard's time range (classic — someone set an absolute past range that no longer has data, or the panel is querying a much shorter range than data retention supports); template/dashboard variables resolving to a value that matches nothing (a `$namespace` variable pointing at a namespace that no longer exists after a migration); or a datasource UID mismatch after a dashboard was imported into a different Grafana instance than it was exported from.

Q88 Loki not receiving logs.

```
kubectl -n logging logs -l app.kubernetes.io/name=promtail --tail=50
curl -s http://loki:3100/ready
curl -s http://loki:3100/metrics | grep loki_ingester
```

Check both ends of the pipe: on the **shipper side** (Promtail/Fluent Bit/Alloy), confirm it's actually tailing the expected log paths and its own logs don't show delivery errors to Loki (connection refused, 429s from Loki rate-limiting); on the **Loki side**, check ingester health and, very commonly, **per-tenant rate limits** (`ingestion_rate_mb` / `ingestion_burst_size_mb`) silently dropping logs past a threshold with a 429 the shipper may or may not retry correctly. Also check label cardinality — Loki indexes only labels, not log content, so a shipper configuration that generates high-cardinality labels (e.g. including a request ID as a label) can make Loki reject or badly perform on ingestion.

Q89 FluentBit stopped forwarding logs.

```
kubectl -n logging logs -l app=fluent-bit --tail=100
kubectl -n logging exec -it <fluent-bit-pod> -- cat /var/log/flb-storage/*  # if using filesystem buffering
```

Common causes: the **output backend rejecting or throttling** (Elasticsearch/OpenSearch bulk rejections from mapping conflicts or cluster yellow/red health, or a downstream endpoint's auth token expiring); **buffer/backpressure** — Fluent Bit's `storage.type filesystem` buffer filling up when the output is slower than input, eventually pausing ingestion entirely (check `mem_buf_limit` and disk space for the buffer path); a **parser failure** silently dropping malformed lines if `Parser` config doesn't match actual log format after an app changed its log format; or simply the DaemonSet pod itself being OOMKilled on a node with heavy log volume and insufficient memory limits. Fluent Bit's own internal metrics endpoint (`/api/v1/metrics`) is worth scraping into Prometheus specifically so this class of silent-drop failure is visible before someone notices logs are missing.

Q90 OpenTelemetry collector overloaded.

```
kubectl top pod -l app=otel-collector
curl localhost:8888/metrics | grep otelcol_processor_dropped
```

Diagnose whether the bottleneck is **ingestion** (too much telemetry volume arriving) or **export** (backend can't keep up, causing internal queues to back up), since scaling the collector itself only helps the first case. Check the collector's own self-telemetry: `otelcol_processor_dropped_spans` / `_metric_points` confirms actual data loss versus just latency; `otelcol_exporter_queue_size` approaching `queue_size` capacity points at a slow/unavailable exporter destination, not the collector's ingestion capacity. Common fixes: add a **batch processor** with sane batch sizes/timeouts if missing, tune `sending_queue` / `retry_on_failure` on exporters, use the **memory_limiter** processor to fail fast and shed load rather than OOMKilling under a burst, or move from a single Deployment collector to per-node DaemonSet collection plus a smaller central gateway tier to spread the ingestion load.

Q91 Prometheus high memory usage.

Almost always a **cardinality** problem, not a "Prometheus needs more resources" problem — every unique label-value combination is a separate time series held in memory, so it scales with the product of labels, not the number of metric names.

```
curl -s localhost:9090/api/v1/status/tsdb | jq '.data.seriesCountByMetricName'
topk(10, count by (__name__)({__name__=~".+"}))
```

Find the worst offenders with `topk` queries against `__name__` and label combinations, then look for the classic culprits: a label containing a request ID, user ID, pod IP, or raw URL path (instead of a templated route) — any of these turn one logical metric into millions of series. Fix at the source (remove the high-cardinality label from the app/exporter) rather than just scaling Prometheus's memory, since the growth is usually unbounded. `metric_relabel_configs` with a `drop` action is a stop-gap while the real fix ships.

Q92 Large TSDB.

```
du -sh /prometheus/*
curl -s localhost:9090/api/v1/status/tsdb
```

Root causes are the same cardinality issue as high memory (Q91) plus retention configuration: `--storage.tsdb.retention.time` set too long for the actual disk provisioned, or a high scrape frequency (`scrape_interval`) multiplying total sample count unnecessarily for metrics that don't need 15s resolution. Practical mitigations, roughly in order of effort: reduce retention to match real operational need (most incident investigation happens within days, not months — use long-term storage like Thanos/Mimir/Cortex for anything needed beyond that instead of growing local TSDB); fix cardinality at the source; and consider recording rules to pre-aggregate expensive high-cardinality queries into smaller derived series rather than re-querying raw data repeatedly. Compaction itself (TSDB automatically compacts 2h blocks into larger ones) can also stall and bloat disk usage if the Prometheus process was OOMKilled mid-compaction repeatedly — check for orphaned/tmp block directories.

Q93 Duplicate metrics.

```
kubectl get servicemonitor -A -o wide
curl -s localhost:9090/api/v1/targets | jq '.data.activeTargets[].labels'
```

Usually one of: **two ServiceMonitors/scrape configs matching the same target** (overlapping label selectors, or the same app scraped once via a ServiceMonitor and again via a manual static scrape config left over from before the operator was adopted); **multiple Prometheus replicas without proper federation/dedup**, each independently scraping and both being queried through the same Grafana datasource without a dedup layer (Thanos Query's dedup feature exists specifically for this in HA Prometheus setups — missing it is the classic cause of doubled values in dashboards); or a pod exposing the same metric on two different ports/paths that both got scraped. Fix by tightening selectors so exactly one scrape config matches each target, and enabling proper dedup at the query layer for genuinely-HA Prometheus pairs rather than trying to prevent the intentional duplication at the source.

Security (Cluster-Level)

Q94 ServiceAccount compromised.

Immediate containment, then investigation — don't reverse that order under pressure. First rotate/invalidate: delete the associated token Secret (legacy long-lived tokens) or, for projected/bound tokens (the modern default), the token is short-lived and audience-bound already, but you should still revoke by removing the ServiceAccount's RoleBindings immediately to cut off access regardless of token validity window.

```
kubectl get rolebinding,clusterrolebinding -A -o json | jq '.items[] | select(.subjects[]?.name=="<sa-name>")'
kubectl delete rolebinding <binding> -n <namespace>
```

Then investigate blast radius via the audit log: `kubectl get events` won't show this — you need API server audit logs (CloudWatch for EKS control-plane logging) filtered by `user.username: system:serviceaccount:<ns>:<name>` to see every action it actually took. Rotate the SA's token, review what permissions it had (was it over-privileged relative to what it needed — the root cause worth fixing regardless of how the compromise happened), and check whether it was used to create new resources (backdoor pods, new RoleBindings) that need separate cleanup.

Q95 RBAC misconfiguration.

```
kubectl auth can-i <verb> <resource> --as=<user-or-sa> -n <namespace>
kubectl auth can-i --list --as=<user-or-sa>
```

`kubectl auth can-i --as` is the fastest way to reproduce exactly what a user/SA can and can't do without trial-and-error trying real operations. Two directions of misconfiguration to watch for: **too restrictive** (a Role missing a verb/resource a legitimate workflow needs — e.g. forgetting `watch` alongside `list` for a controller that needs to react to changes, not just poll) and **too permissive** (a ClusterRoleBinding used where a namespaced RoleBinding would have sufficed, or `*` wildcards in `verbs`/`resources` granted broadly "to make it work" and never tightened afterward). Also check for RoleBindings referencing a Role/ClusterRole that no longer exists (silently grants nothing, looks like it should work but doesn't) and remember RBAC is purely additive — there's no explicit "deny" rule, so the fix for over-permission is always removing bindings, never adding a deny.

Q96 Privileged container running.

```
kubectl get pods -A -o json | jq '.items[] | select(.spec.containers[]?.securityContext?.privileged==true) |
.metadata.name'
```

A `privileged: true` container has essentially full access to the host (all capabilities, device access, ability to affect other containers) — it defeats almost every other container isolation boundary, so finding one unexpectedly is a real finding, not a nitpick. Legitimate uses are narrow: certain CNI/CSI node-plugin DaemonSets genuinely need it for the host-level operations they perform (mounting filesystems, configuring network interfaces) — check if it's one of those (kube-system namespace, known infra component) versus an application workload that has no legitimate reason for it.

Prevention going forward: enforce via **Pod Security Admission** at the `restricted` or `baseline` level (which disallows `privileged`) on application namespaces, or a policy engine (Kyverno/OPA Gatekeeper) with an explicit allow-list for the few namespaces that legitimately need exceptions.

Q97 Secret leaked.

Rotate the actual credential at its source **first**, before anything else — a leaked database password or API key is dangerous regardless of what's in the Kubernetes Secret object, and the Kubernetes-side cleanup doesn't matter until the underlying credential is invalidated.

```
# Then update the k8s side
kubectl create secret generic <name> --from-literal=key=<new-value> --dry-run=client -o yaml | kubectl apply -f
-
kubectl rollout restart deployment/<name>   # so pods pick up the new value
```

Then find how it leaked: was it committed to git (requires history rewriting/BFG plus assuming it's compromised forever since git history is hard to fully scrub once pushed), exposed in a pod's environment via a debugging tool or logged accidentally by the app, or was the Secret object itself readable by a principal that shouldn't have had RBAC access to `get secrets` in that namespace? Longer-term fix: move to short-lived, dynamically-injected credentials (External Secrets Operator pulling from Vault/Secrets Manager with rotation) instead of long-lived static Secrets, so a single leak has a small blast-radius window.

Q98 Container running as root.

```
kubectl get pod <pod> -o jsonpath='{.spec.securityContext.runAsUser}'
kubectl exec <pod> -- id
```

Running as root inside the container isn't automatically catastrophic (container root ≠ host root, given proper namespace isolation), but it removes a defense-in-depth layer — if there's ever a container-escape vulnerability, root-in-container makes it root-on-host, whereas a non-root process has a real additional barrier to cross. Fix: set `securityContext.runAsNonRoot: true` and an explicit `runAsUser` at the pod or container level, which many base images require you to also prepare for (some images hardcode root-owned file permissions that break under a non-root UID — needs a Dockerfile fix, not just a Kubernetes spec change). Enforce cluster-wide via Pod Security Admission's `restricted` profile or an admission policy, rather than relying on every team remembering to set it per-workload.

Q99 Image vulnerability detected.

```
trivy image <image>:<tag>
grype <image>:<tag>
```

Triage by actual exploitability, not just CVSS score in isolation: is the vulnerable package/function path actually reachable/used by the application (many scanners flag transitive dependencies that are present but never invoked), is it exposed at the network boundary or purely internal, and is a patched version available yet (if not, a compensating control — NetworkPolicy restricting exposure, WAF rule — buys time). Production response: patch and redeploy the base image (rebuild from an updated base, don't just patch the running container), and if this is recurring, this is the argument for **image scanning gates in CI** (fail the build/block the push on critical CVEs) plus an **admission-time policy** (Kyverno/OPA validating against a scan result or a signed-image requirement) so vulnerable images can't reach the cluster in the first place, rather than finding out after the fact.

Q100 Pod Security Admission blocking deployment.

```
kubectl get ns <namespace> --show-labels | grep pod-security
kubectl describe pod <pod>   # or the create error itself, PSA rejects synchronously
```

Pod Security Admission enforces at one of three levels via a namespace label (`pod-security.kubernetes.io/enforce: privileged|baseline|restricted`), and a rejection is immediate and synchronous on the create/update call — the error message actually names the specific violated rule (e.g. "must not set `allowPrivilegeEscalation=true` ", " `hostNetwork` is not allowed"), so read the error rather than guessing which of the ~15 restricted-profile checks tripped.

Resolution paths: fix the pod spec to comply (the right answer for application workloads under `restricted`); move the specific workload to a namespace with a looser enforcement level if it has a genuine, reviewed reason to need something like `hostNetwork` ; or use `audit` / `warn` modes alongside `enforce` during rollout so teams see violations ahead of a hard cutover rather than being surprised by a blocked deploy in production.

Q101 NetworkPolicy bypass.

NetworkPolicy is enforced by the CNI plugin, not the API server — if traffic is flowing that a policy should block, first confirm the **CNI actually supports NetworkPolicy enforcement** at all (this genuinely varies; some CNI configurations are policy-enforcement-capable and some aren't, and a policy object can exist and look "applied" while doing nothing if the CNI silently ignores it).

```
kubectl get pods -n kube-system -o wide | grep -E 'calico|cilium'
kubectl describe networkpolicy <name> -n <namespace>
```

If the CNI does enforce policies, other bypass causes: the policy's `podSelector` doesn't actually match the pods you think it does (label drift after a redeploy); traffic going through a path the policy doesn't cover (e.g. a policy scoped to Pod IPs while traffic actually routes through a hostNetwork proxy or a Service's external IP in a way that changes the effective source); or the policy was applied to the wrong namespace entirely. Validate with a real connectivity test from a test pod matching the "should be blocked" profile, not just by reading the YAML.

Q102 Unauthorized kubectl access.

Containment first: identify and revoke the specific credential used (rotate/delete the compromised user cert, OIDC token, or ServiceAccount token; for EKS, update `aws-auth` /access entries to remove the offending IAM principal's cluster mapping). Then investigate scope via the **audit log** — this is the only reliable source of "what did they actually do," since `kubectl get events` is far too limited and short-lived for this.

```
# EKS control-plane audit logging must be enabled ahead of time to have this available
aws eks update-cluster-config --name <cluster> --logging '{"clusterLogging":[{"types":
["audit"],"enabled":true}]}'
```

Filter CloudWatch Logs (or your audit sink) by the compromised identity and review every mutating verb (`create` / `update` / `delete` / `patch`) it issued in the window of compromise. Longer-term: this is exactly the argument for short-lived credentials (IAM roles + OIDC federation rather than long-lived static kubeconfig files handed out broadly), least-privilege RBAC so any single compromised identity has a small blast radius, and audit logging enabled by default rather than turned on only after an incident.

Q103 API server exposed publicly.

Public exposure alone isn't automatically a breach — EKS's public endpoint is authenticated (client certs, IAM auth via `aws-iam-authenticator` , or OIDC), so the real question is whether authentication/authorization are correctly restrictive, not just "is it reachable." That said, minimizing the exposed surface is still the right default posture.

```
aws eks describe-cluster --name <cluster> --query 'cluster.resourcesVpcConfig'
```

For EKS, the fix is usually the `endpointPublicAccess` / `endpointPrivateAccess` configuration: switch to private-only, or public with a restricted `publicAccessCidrs` allow-list (office/VPN CIDRs only) rather than `0.0.0.0/0` . Confirm anonymous access is disabled (`system:anonymous` shouldn't have any bindings — `kubectl auth can-i --list --as=system:anonymous`), TLS is enforced with no fallback to plaintext, and that whatever is currently exposed doesn't include the etcd port (2379/2380) or kubelet port (10250) directly to the internet, which are far more dangerous to expose than the API server itself.

PART II

EKS Production Questions

33 questions · control plane, node groups, IAM, networking, ALB controller & storage

Control Plane

Q104 Worker nodes cannot join cluster.

```
kubectl get nodes
kubectl -n kube-system logs -l k8s-app=aws-node   # if it even shows up
# On the node itself:
journalctl -u kubelet -n 100
```

The two most common causes, both EKS-specific: (1) the node's **IAM role isn't mapped** in `aws-auth` ConfigMap (or the newer EKS access entries) — without `system:bootstrappers` / `system:nodes` group mapping for that role, the node can authenticate to AWS but the API server rejects it as an unrecognized identity; (2) **security group** rules between the node and the control plane ENIs blocking required ports (443 for API, plus kubelet's 10250 for the control plane to reach back for exec/logs). Also check the node's user-data/bootstrap script actually ran (`/var/log/cloud-init-output.log`) — a custom AMI or launch template that broke the standard `bootstrap.sh` invocation is a common self-inflicted cause after a "let's customize the AMI" change.

Q105 EKS API unavailable.

```
aws eks describe-cluster --name <cluster> --query 'cluster.status'
```

Since AWS owns and auto-scales the EKS control plane, first rule out an AWS-side event via the Personal Health Dashboard/Service Health Dashboard before assuming a config problem on your end — genuine EKS control-plane outages are rare but do happen and there's nothing to fix on your side in that case except wait and communicate status. If the cluster status itself is healthy but you specifically can't reach it: check `endpointPublicAccess` / `publicAccessCidrs` (did someone tighten the CIDR allow-list and lock out the current network?), VPN/Direct Connect path for private-endpoint clusters, or a client-side kubeconfig pointing at a stale/deleted cluster endpoint after a cluster recreate.

Q106 Managed Node Group update failed.

```
aws eks describe-nodegroup --cluster-name <c> --nodegroup-name <ng> --query 'nodegroup.health'
```

EKS-managed node group updates use a rolling replace under the hood, respecting PodDisruptionBudgets during drain — a stuck/failed update very often traces to a **PDB that can never be satisfied** (e.g. `minAvailable` equal to the total replica count, which mathematically blocks any voluntary eviction). Check the node group's `health.issues` field for the specific AWS-reported reason; other common causes: launch template changes referencing an AMI that no longer exists or fails to boot, insufficient capacity for the instance type/AZ combination during the rolling replace, or IAM permission gaps on the node role preventing new nodes from joining (same as Q104, but surfacing mid-update instead of at initial creation).

Q107 Cluster upgrade failed.

EKS upgrades the control plane first, then you separately upgrade node groups and add-ons — a "failed upgrade" report needs you to identify which of those three layers actually failed, since the remediation is completely different for each.

```
aws eks describe-update --name <cluster> --update-id <id>
```

Most common real causes: a workload using a **deprecated/removed API version** that no longer exists in the target Kubernetes version (the control plane upgrade itself can succeed while existing manifests using removed APIs start failing to reconcile afterward — always run `pluto` / `kubent` against removed APIs before upgrading, not after); an **add-on version incompatibility** (VPC CNI, CoreDNS, kube-proxy versions pinned too far behind the new control-plane minor version); or webhook/CRD conflicts where a third-party operator's admission webhook wasn't updated for new API behavior and starts rejecting valid requests post-upgrade.

Q108 API version deprecated.

```
kubectl api-resources
pluto detect-all-in-cluster   # or kubent, purpose-built for this
kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis
```

Kubernetes removes deprecated APIs entirely after a defined support window (not just warns forever) — the classic incidents here are `extensions/v1beta1` and `apps/v1beta1` Ingress/Deployment manifests that were fine for years and then simply stopped being accepted on a minor-version upgrade, breaking GitOps applies with no runtime warning until that exact moment. The apiserver itself exposes a metric (`apiserver_requested_deprecated_apis`) specifically to catch this ahead of time — alert on it well before any planned upgrade. Fix is migrating manifests to the current stable API group/version (e.g. `networking.k8s.io/v1` for Ingress) and re-applying, which is safe to do proactively regardless of your current cluster version.

Node Groups

Q109 Managed Node Group unhealthy.

```
aws eks describe-nodegroup --cluster-name <c> --nodegroup-name <ng> --query 'nodegroup.health.issues'
```

The `health.issues` field gives a structured, actionable reason code directly — always check this before digging into individual instances. Common codes: `AsgInstanceLaunchFailures` (capacity or launch-template problem, check the ASG's own activity history); `NodeCreationFailure` (nodes launched but never joined — see Q104's IAM/security-group checklist); `Ec2SecurityGroupNotFound` or `Ec2SubnetInvalidConfiguration` (infrastructure drift, often from manual changes outside IaC); or `IamInstanceProfileNotFound` after someone modified/deleted the node role outside of Terraform/CloudFormation. Cross-reference with `aws autoscaling describe-scaling-activities` for the underlying ASG to see the actual EC2-level launch failure reason.

Q110 Self-managed node not joining.

Self-managed nodes skip all the guardrails EKS-managed node groups give you automatically, so check everything manually that would otherwise be handled for you:

- **User-data / bootstrap script** actually invoked `/etc/eks/bootstrap.sh` with the correct cluster name, endpoint, and CA data — `/var/log/cloud-init-output.log` shows exactly what ran and any errors.
- `aws-auth` **ConfigMap** mapping for the node's instance role — self-managed nodes are especially easy to forget here since there's no automatic wiring the way managed node groups get.
- **Security groups** allowing control-plane ↔ node communication in both directions on the required ports.
- **AMI/kubelet version compatibility** with the cluster's Kubernetes version (must be within the supported skew, typically the same minor version or one behind).

Q111 Mixed Spot and On-demand strategy.

The production pattern is separate node groups per capacity type, labeled and tainted so scheduling is explicit rather than accidental: a Spot node group tainted `spot=true:NoSchedule`, and workloads that can tolerate interruption opt in via a matching toleration plus a node-affinity preference. Nothing should land on Spot by default — that's the safety property you want.

```
kubectl get nodes -l eks.amazonaws.com/capacityType=SPOT
kubectl get nodes -l eks.amazonaws.com/capacityType=ON_DEMAND
```

Design guidance worth stating in an interview: keep stateless, horizontally-scaled, interruption-tolerant workloads (most web/API tiers behind a Service, with enough replicas that losing one is a non-event) on Spot; keep anything stateful, singleton, or latency-critical-during-interruption (databases, leader-elected controllers, the ingress controller itself in small clusters) on On-Demand. Combine with the **Node Termination Handler** (or Karpenter's built-in interruption handling) so a Spot reclaim triggers a graceful drain via the 2-minute EC2 interruption notice, rather than an ungraceful kill.

Q112 Spot nodes terminated.

```
kubectl get events --field-selector reason=NodeNotReady
# AWS Node Termination Handler / Karpenter interruption logs:
kubectl -n kube-system logs -l app=aws-node-termination-handler
```

AWS gives a ~2-minute interruption notice before actually reclaiming a Spot instance — the entire point of the Node Termination Handler (or Karpenter's native handling) is to catch that notice and proactively cordon+drain in that window, so pods get a graceful eviction (respecting PDBs, terminationGracePeriod) instead of a hard kill with zero warning. If terminations are causing real disruption, check: is NTH/Karpenter interruption handling actually installed and running (surprisingly often skipped when a team adds a Spot node group ad hoc); are PodDisruptionBudgets set sensibly for workloads on Spot (too strict a PDB can make the drain fail to complete within the 2-minute window, so the pod gets killed anyway when the hard deadline hits); and is Spot diversification (multiple instance types/AZs) configured to reduce simultaneous interruption correlation across the whole node group.

Q113 Auto Scaling Group issues.

```
aws autoscaling describe-scaling-activities --auto-scaling-group-name <asg> --max-records 10
```

`describe-scaling-activities` is the single best diagnostic call here — it shows the actual reason for every recent scale event/failure directly from AWS, rather than inferring from Kubernetes-side symptoms. Common causes: **capacity not available** for the requested instance type/AZ combination (especially with a narrow instance-type selection and no diversification); **service quota** limits (vCPU-based EC2 quotas are the usual ceiling, not "number of instances"); a **launch template** referencing a deregistered AMI or an invalid IAM instance profile; or the **Cluster Autoscaler/Karpenter** requesting scale-up but the ASG's own `MaxSize` being lower than what's needed, which silently caps growth without any error visible from the Kubernetes side at all — always check the ASG's configured min/max/desired against what the autoscaler thinks it can request.

IAM

Q114 Pod cannot access S3.

```
kubectl exec -it <pod> -- aws sts get-caller-identity
kubectl describe sa <service-account> -n <namespace>
```

`get-caller-identity` run from inside the pod tells you immediately whether the pod is even assuming the IAM role you expect — if it returns the *node's* IAM role instead of the intended pod-specific role, that's IRSA not being applied (see next question). If it does show the correct assumed role, then it's a pure IAM problem: the role's policy is missing the specific S3 action/resource (check for a bucket-policy-side restriction too — both the IAM role's policy *and* the S3 bucket policy must allow the access; a bucket policy with an explicit deny or a missing principal will block access even with a perfect IAM role policy), or an S3 **VPC endpoint policy** (if using a Gateway endpoint for S3) additionally restricting which buckets are reachable from the VPC.

Q115 IRSA not working.

IRSA (IAM Roles for Service Accounts) depends on a specific chain all being correct simultaneously: the cluster's **OIDC provider** registered in IAM, the ServiceAccount's `eks.amazonaws.com/role-arn` annotation, and the role's **trust policy** scoped to that exact OIDC provider + namespace + ServiceAccount name.

```
kubectl describe sa <sa> -n <ns> | grep role-arn
aws iam get-role --role-name <role> --query 'Role.AssumeRolePolicyDocument'
kubectl exec -it <pod> -- env | grep AWS_
kubectl exec -it <pod> -- aws sts get-caller-identity
```

Checklist: is the OIDC provider actually associated with the cluster (`aws eks describe-cluster --query 'cluster.identity.oidc.issuer'` and compare against IAM's registered provider); does the trust policy's `Condition` match the exact `system:serviceaccount:<namespace>:<name>` string (a common typo source); is the **pod identity webhook** actually injecting the `AWS_ROLE_ARN` / `AWS_WEB_IDENTITY_TOKEN_FILE` env vars and the projected token volume (if missing, the pod was likely created before the ServiceAccount annotation was added — the webhook only mutates pods at creation time, so existing pods need to be recreated, not just the SA edited).

Q116 AWS Load Balancer Controller permissions missing.

```
kubectl -n kube-system logs -l app.kubernetes.io/name=aws-load-balancer-controller | grep -i "not
authorized\|AccessDenied"
```

The controller's logs show the exact denied API action, which maps directly to a missing statement in its IRSA policy — usually because the IAM policy JSON that ships with the controller's install docs wasn't kept in sync after a controller version upgrade added new required permissions (this genuinely happens across versions; always check the release notes' IAM policy diff when upgrading the controller). Verify the policy is attached to the exact role the controller's ServiceAccount references, and that the trust policy's OIDC condition matches `kube-system:aws-load-balancer-controller` (or whatever namespace/SA name you actually used) precisely.

Q117 External DNS permissions issue.

```
kubectl -n kube-system logs -l app.kubernetes.io/name=external-dns
```

ExternalDNS needs Route53 permissions scoped correctly: `route53:ListHostedZones` / `ListResourceRecordSets` (read, to know current state) plus `route53:ChangeResourceRecordSets` scoped to the specific hosted zone ARN(s) it manages (write). A common misconfiguration is scoping the policy to the wrong zone ID after a domain migration, or forgetting that if you use the `--txt-owner-id` ownership-tracking pattern, ExternalDNS also needs permission to manage the TXT registry records it uses to avoid conflicting with manually-managed records — a permissions gap here causes it to correctly refuse to touch records it can't verify ownership of, which looks like "it's ignoring my Ingress" rather than an obvious auth error.

Q118 EBS CSI Driver IAM issue.

```
kubectl -n kube-system logs -l app=ebs-csi-controller -c csi-provisioner | grep -i "unauthorized\|AccessDenied"
```

Same IRSA-chain check as Q115, applied to the `ebs-csi-controller-sa` ServiceAccount specifically. The managed policy `AmazonEBSCSIDriverPolicy` covers the standard actions (`CreateVolume` , `AttachVolume` , `DeleteVolume` , `CreateSnapshot` , etc.) — if using KMS-encrypted volumes, also verify the role has `kms:CreateGrant` / `Decrypt` on the specific CMK, which is a separate, easy-to-forget grant beyond the base EBS policy. If the EKS cluster was created without the EBS CSI driver as a managed add-on (self-installed via Helm instead), double-check the IRSA annotation and trust policy were wired up manually and correctly, since the managed add-on path handles more of this automatically via `eksctl` /Terraform EKS modules.

Networking (EKS/VPC)

Q119 ENI exhausted.

Every EC2 instance type has a hard limit on attachable ENIs (Elastic Network Interfaces), and by default VPC CNI's max-pods-per-node calculation is derived directly from that limit — a `t3.medium` supports far fewer pods (~17) than an `m5.xlarge` (~58), which surprises people who think "pods per node" is a Kubernetes-level setting they control freely.

```
aws ec2 describe-instance-types --instance-types <type> --query
'InstanceTypes[].NetworkInfo.MaximumNetworkInterfaces'
kubectl describe node <node> | grep -A2 "Allocatable:"
```

Fixes: use larger instance types (more ENIs/IPs per instance = better pod density) if right-sizing allows it, or enable **VPC CNI prefix delegation** (assigns /28 IP prefixes instead of one IP per ENI slot, dramatically raising pod density per instance without changing instance type), which is the standard modern fix for this specific constraint.

Q120 IP addresses exhausted.

```
aws ec2 describe-subnets --subnet-ids <id> --query 'Subnets[].AvailableIpAddressCount'
kubectl -n kube-system logs <aws-node-pod> | grep -i "InsufficientCidrBlocks\|no available IP"
```

With the default VPC CNI model, every pod consumes a real routable IP from the VPC subnet — on a busy cluster with small subnets (a very common mistake: provisioning a /24 "because it seemed like plenty" without accounting for pod density × node count), you exhaust subnet IPs well before any compute limit. Fixes, roughly by effort: add additional CIDR blocks / larger subnets if the VPC design allows it; enable **prefix delegation** (same fix as ENI exhaustion — it also reduces the rate of individual IP allocation calls, which matters at scale); or use **custom networking** (a secondary CIDR just for pod IPs, separate from node IPs) to get a much larger effective pool without re-architecting the whole VPC.

Q121 VPC CNI issue.

```
kubectl -n kube-system get ds aws-node
kubectl -n kube-system logs <aws-node-pod> -c aws-node --tail=100
```

The `aws-node` DaemonSet has to be Running and healthy on every node for pod networking to work at all on that node — check per-node status, not just "is the DaemonSet healthy overall" (a partial rollout can leave some nodes fine and others broken). Beyond IP/ENI exhaustion (covered above), other VPC CNI-specific issues: a version mismatch after a manual CNI upgrade that didn't match the EKS-recommended compatible version for that cluster version; `WARM_ENI_TARGET` / `WARM_IP_TARGET` environment variables tuned too conservatively, causing pods to wait on IP allocation under bursty scheduling even when the subnet has capacity; or the CNI plugin conf file on the node (`/etc/cni/net.d/`) being corrupted/missing after a node bootstrap issue.

Q122 Pods cannot reach RDS.

```
kubectl exec -it <pod> -- nc -zv <rds-endpoint> 5432
kubectl exec -it <pod> -- nslookup <rds-endpoint>
```

Because pods get real VPC IPs with VPC CNI, this is a standard AWS networking/security-group problem, not a Kubernetes one — check in order: (1) **RDS security group** allowing inbound on the DB port from the *node/pod security group*, not just "the VPC CIDR" loosely (best practice is a specific SG-to-SG rule, not a broad CIDR); (2) **route tables** if RDS and the cluster are in different subnets/VPCs (peering or Transit Gateway route needed, plus the RDS-side route back); (3) **NetworkPolicy** egress rules if default-deny egress is enforced and the RDS `ipBlock` isn't explicitly allowed; (4) DNS resolution of the RDS endpoint specifically, if using a custom DNS setup that doesn't forward RDS's Route53-managed hostname correctly.

Q123 Private subnet no internet.

```
aws ec2 describe-route-tables --filters "Name=association.subnet-id,Values=<subnet>"
```

By design, a private subnet has no route to an Internet Gateway — outbound internet access requires an explicit route to a **NAT Gateway** (or NAT instance) in the route table associated with that subnet. Missing entirely, this manifests as pods failing to pull images from public registries, reach external APIs, or resolve dependencies during builds. Checklist: does the private subnet's route table have a `0.0.0.0/0 → nat-gateway-id` route; is the NAT Gateway itself healthy and in the correct AZ (a NAT Gateway is AZ-specific — a private subnet routed to a NAT Gateway in a *different* AZ works but adds cross-AZ data transfer cost, while one with no reachable NAT at all in any AZ fails outright); and for AWS-service-only traffic (S3, ECR, etc.), whether VPC endpoints are configured as the lower-cost, lower-latency alternative to routing everything through NAT.

Q124 NAT Gateway failure.

```
aws ec2 describe-nat-gateways --filter "Name=vpc-id,Values=<vpc>"
```

Check `State` first (a NAT Gateway can be `failed` at creation, most often from an **Elastic IP** allocation problem or a subnet quota issue). If it's `available` but traffic still isn't flowing: NAT Gateways have a per-Gateway **bandwidth ceiling** (up to 100 Gbps burst, but sustained heavy traffic from many nodes routed through a single NAT Gateway can bottleneck, especially with only one deployed instead of one-per-AZ) and a connection-tracking limit that can be exhausted under very high connection-churn workloads, manifesting as intermittent timeouts rather than total failure. Production pattern: one NAT Gateway per AZ (not a single shared one) both for availability (an AZ failure doesn't take down egress for the whole VPC) and to spread the bandwidth/connection load.

Q125 Route table issue.

```
aws ec2 describe-route-tables --route-table-ids <rtb>
```

Most route table incidents trace back to a **manual change outside of Terraform/CloudFormation** that IaC then either fought with (drift causing a subsequent apply to unexpectedly change routing) or never saw at all. Checklist: correct subnet associations (a subnet accidentally associated with the wrong route table after a refactor is a classic silent breakage — traffic routes as if it were still in the old subnet's network posture); route priority/specificity (more specific CIDR routes win, so an overly broad manually-added route can shadow an intended more specific one, or vice versa); and for multi-VPC setups, whether Transit Gateway or peering routes were correctly propagated to every route table that needs them, not just the "obvious" one.

Q126 Security Group problem.

```
aws ec2 describe-security-groups --group-ids <sg>
```

Security groups are stateful and **default-deny** on inbound (nothing implicit is allowed in), while outbound is open by default unless explicitly restricted — know which direction you're actually debugging. With VPC CNI, pods share the **node's** security group by default (unless using `SecurityGroupPolicy` /pod-level security groups on Nitro instance types), so a rule change intended to affect "the application" actually affects every pod on that node group, which is a common source of confusing side effects when one team edits an SG for their own workload. Checklist: SG-to-SG rules (referencing another security group as the source, not a CIDR) correctly reference the current SG IDs after any infra recreation (SG IDs change on recreate, and a rule referencing an old, now-orphaned SG ID silently allows nothing); and NACLs at the subnet level as a second, stateless layer that security-group-only debugging sometimes misses entirely.

AWS Load Balancer Controller

Q127 ALB not created.

```
kubectl describe ingress <name>   # Events section
kubectl -n kube-system logs -l app.kubernetes.io/name=aws-load-balancer-controller --tail=50
```

Checklist, most to least common: missing `kubernetes.io/ingress.class: alb` annotation (or `ingressClassName: alb` on newer versions) — without it, the controller doesn't consider the Ingress its responsibility at all, and it just sits there with no events explaining why nothing happened; the controller pod itself not running/crashing; missing **subnet tags** (`kubernetes.io/role/elb=1` for public-facing, `kubernetes.io/role/internal-elb=1` for internal) — without correct tags on at least two subnets across different AZs, the controller can't determine where to place the ALB; and IAM permission gaps (see Q116). Always read the Ingress's own `Events` first — the controller annotates the Ingress object directly with its reconciliation errors.

Q128 Target Group unhealthy.

```
aws elbv2 describe-target-health --target-group-arn <arn>
```

First determine **target type** — `instance` mode registers node instance IDs (health check hits `NodePort`, routed via kube-proxy to wherever the pod actually is), while `ip` mode (set via `alb.ingress.kubernetes.io/target-type: ip`) registers pod IPs directly, bypassing kube-proxy entirely and requiring the pod's security group to allow the health-check traffic directly. Most common cause either way: the **health-check path/port** annotation (`alb.ingress.kubernetes.io/healthcheck-path`) doesn't match a real endpoint the app actually serves (defaults to `/`, which 404s for many APIs that don't serve anything at root). For `ip` mode specifically, also check the pod's readiness (unready pods aren't registered at all, which can look identical to "unhealthy" in the target group if nothing is registered) and that the pod security group allows inbound from the ALB's security group on the container port.

Q129 ALB listener missing.

```
aws elbv2 describe-listeners --load-balancer-arn <arn>
```

A missing HTTPS (443) listener specifically almost always means the controller couldn't resolve which **ACM certificate** to attach — check `alb.ingress.kubernetes.io/certificate-arn` is set explicitly, or (if relying on auto-discovery by host) that the Ingress's `host` field matches a certificate's domain/SAN in ACM exactly, since the controller's cert-discovery-by-hostname feature fails silently if there's no match rather than erroring loudly. A missing HTTP (80) listener usually just means `alb.ingress.kubernetes.io/listen-ports` was set to only include 443, which is a deliberate choice (HTTPS-only) rather than a bug — confirm whether an HTTP-to-HTTPS redirect was actually intended and, if so, whether the `ssl-redirect` action annotation is present.

Q130 SSL certificate not attached.

```
aws acm list-certificates --query 'CertificateSummaryList[].DomainName'
kubectl describe ingress <name> | grep -i certificate
```

Beyond the listener-level cause above, check: the certificate is in `ISSUED` **status** in ACM, not still `PENDING_VALIDATION` (a cert stuck pending DNS validation forever because the validation CNAME was never actually created in the DNS zone is extremely common right after requesting a new cert); the certificate's **region matches the ALB's region** (ACM certificates are region-scoped, and referencing a certificate ARN from the wrong region fails silently for ALB, unlike CloudFront which needs us-east-1 specifically — a frequent source of confusion between the two services); and if using multiple hostnames on one ALB via SNI, that each hostname's certificate is included in the `certificate-arn` annotation's comma-separated list, since the controller doesn't auto-attach every ACM cert in the account.

Q131 Ingress ignored.

```
kubectl get ingressclass
kubectl get ingress <name> -o yaml | grep -A2 ingressClassName
```

"Ignored" (zero events, zero reconciliation attempt at all) almost always means the controller doesn't think this Ingress belongs to it — check for an `IngressClass` mismatch first: if multiple ingress controllers are installed in the cluster (e.g. both NGINX and AWS ALB Controller, common during a migration between them), an Ingress without an explicit `ingressClassName` may default to whichever controller is marked as the cluster default, not the one you intended. Also check the controller's own `--ingress-class` startup flag if customized away from the default `alb`, and confirm the controller's `watch-namespace` configuration (if restricted) actually includes the namespace this Ingress lives in — a controller scoped to watch only specific namespaces will silently skip everything else.

EBS / EFS (EKS)

Q132 EBS volume Pending.

Layer this on top of the general PVC-Pending checklist (see Storage section) with EKS-specific causes: the **EBS CSI driver add-on isn't installed** at all (surprisingly common on freshly-created clusters, since it's an optional add-on, not bundled by default); IRSA permission gaps on the CSI controller (Q118); or a `StorageClass` requesting a `volumeBindingMode: WaitForFirstConsumer` topology that can't be satisfied because the pod's node-affinity/zone constraints conflict with where EBS can actually provision.

```
kubectl get pods -n kube-system | grep ebs-csi
eksctl get addon --cluster <cluster>
```

Q133 EFS mount timeout.

```
kubectl -n kube-system logs -l app=efs-csi-node
nc -zv <mount-target-ip> 2049
```

Slow-fail (timeout, not immediate rejection) is the EFS-specific signature since it's a network filesystem with retry behavior baked in, unlike EBS's fast attach-level errors. Checklist beyond the general EFS causes covered in Storage: mount target's security group specifically allowing inbound 2049 from the **node/pod security group** (not just "the VPC" broadly — a tightened SG after a security review is a common regression trigger here); a mount target existing in the node's AZ at all (EFS needs one per AZ in use, and adding a new AZ to a node group without provisioning a matching mount target is an easy miss); and, for `efs-utils`-based mounting, whether the **stunnel/TLS** encryption-in-transit option is enabled and correctly configured, since a TLS handshake mismatch here manifests as a hang rather than a clean rejection.

Q134 CSI driver crash.

```
kubectl -n kube-system describe pod <csi-pod>
kubectl -n kube-system logs <csi-pod> -c ebs-plugin --previous
```

Distinguish controller-pod crashes (cluster-wide impact on new provisioning/attach operations) from node-plugin crashes (single-node impact on mount/unmount there only) — same distinction as the general CSI question in the Storage section, but specifically for EBS/EFS. Version skew after a Kubernetes cluster upgrade is a very common trigger (the CSI driver version must stay within its supported compatibility matrix for the cluster's Kubernetes minor version), as is resource starvation (default resource requests on the CSI pods sometimes undersized relative to actual load on very large clusters with high PVC churn) causing repeated OOMKills.

Q135 Multi-Attach error.

Standard EBS volumes (`gp2/gp3/io1/io2` without Multi-Attach explicitly enabled) support **ReadWriteOnce** — attached to exactly one node at a time. The classic trigger: a StatefulSet pod on Node A goes down uncleanly (node crash, not a graceful drain), the pod's controller reschedules it to Node B, but Node A never got the chance to cleanly detach the volume first — Node B's attach attempt then fails with `Multi-Attach error ... Volume is already exclusively attached`.

```
aws ec2 describe-volumes --volume-ids <vol> --query 'Volumes[].Attachments'
kubectl get volumeattachment | grep <pv-name>
```

Fix: confirm the old node is genuinely dead (not just slow), then force-detach if needed (`aws ec2 detach-volume --force`) or delete the stale `VolumeAttachment` object so the CSI driver's view matches reality, allowing the new node to attach cleanly. This is exactly why `terminationGracePeriodSeconds` and proper node draining matter for StatefulSets on EBS — a graceful drain avoids this failure mode entirely by detaching before rescheduling.

Q136 Wrong Availability Zone.

EBS volumes are permanently bound to the AZ they were created in — a pod that gets rescheduled (by the Cluster Autoscaler, a node replacement, or just normal churn) to a node in a *different* AZ than its EBS volume simply can never mount it; this isn't a transient error to retry through, it's a hard topology constraint.

```
kubectl get pv <pv> -o jsonpath='{.spec.nodeAffinity}'
kubectl get node <node> -o jsonpath='{.metadata.labels.topology\.kubernetes\.io/zone}'
```

Prevention is the right frame here, not reaction: modern EBS-backed StorageClasses set `volumeBindingMode: WaitForFirstConsumer` specifically so the volume is provisioned *after* the pod is scheduled, in the same AZ the scheduler already chose — if this is set to `Immediate` instead, the volume can be created in a "random" AZ before scheduling even happens, setting up exactly this failure the first time the pod moves. If already stuck, either move the pod back via node-affinity/taints, or (for genuinely cross-AZ resilience) migrate to a snapshot-and-restore in the correct AZ, or to EFS if the workload's access pattern actually needs zone-independence.

PART III

Deployment Process Questions

20 questions · strategies, GitOps/ArgoCD, CI/CD pipelines, rollback & release safety

Deployment Process

Q137 Explain your production deployment process.

Structure the answer as a pipeline, naming the gate at each stage rather than just the tools: **commit** (conventional commits, branch protection) → **CI** (build, unit tests, SAST/image scan, artifact pushed to a registry with an immutable tag/digest) → **GitOps sync** (a manifest/Helm-values change in a separate config repo is what actually triggers deployment — Argo CD or Flux reconciles the cluster to match Git, not a direct `kubectl apply` from the pipeline) → **staged rollout** (dev → staging with automated smoke tests → production via canary or rolling update) → **post-deploy validation** (health checks, SLO/error-rate dashboards watched for a defined bake period) → **automated rollback** on breach of a defined threshold.

The signal interviewers are listening for: separation of "build" from "deploy" (GitOps), declarative desired-state over imperative scripts, and a rollback path that's as automated and rehearsed as the forward path — not something improvised during an incident.

Q138 Blue-Green deployment.

Two complete, independent environments ("blue" = current production, "green" = new version) run simultaneously; traffic cuts over atomically from blue to green (a Service selector change, or a load-balancer/Ingress weight flip) once green is validated. In Kubernetes, this is typically two Deployments with distinct labels and a single Service whose `selector` is repointed — or, more robustly, two Ingress/ALB target groups with a weighted or all-or-nothing traffic shift.

Trade-off to state explicitly: instant rollback (just flip the selector back) at the cost of running **2× the compute** during the transition and needing careful handling of any stateful/DB schema changes that both versions must tolerate simultaneously until cutover completes. Best suited for releases where you want a hard, instant, all-or-nothing switch rather than gradual exposure.

Q139 Canary deployment.

Route a small, controlled percentage of live traffic (e.g. 5%) to the new version alongside the stable version, watch real production metrics (error rate, latency, business KPIs) against the stable baseline, then progressively increase the split (5% → 25% → 50% → 100%) if metrics stay healthy, or automatically roll back to 0% if they don't.

```
# Argo Rollouts canary step example
strategy:
  canary:
    steps:
      - setWeight: 5
      - pause: {duration: 10m}
      - setWeight: 25
      - pause: {duration: 10m}
      - setWeight: 100
```

Key distinction from Blue-Green: canary exposes *real* traffic gradually and can auto-analyze metrics mid-rollout (via Argo Rollouts' `AnalysisTemplate` against Prometheus, for instance) to make the promote/abort decision data-driven rather than a single manual gate. Requires a service mesh or an ingress controller capable of weighted traffic splitting (Istio, Argo Rollouts + NGINX/ALB traffic-shaping annotations) — plain Kubernetes Deployments alone don't give you fine-grained percentage control.

Q140 Rolling Update.

Kubernetes' built-in default strategy for Deployments: replace old pods with new ones incrementally, governed by `maxSurge` (how many extra pods beyond desired count can exist during rollout) and `maxUnavailable` (how many can be down at once), with each new pod required to pass its readiness probe before the next old pod is removed.

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 25%
    maxUnavailable: 0
```

Setting `maxUnavailable: 0` guarantees full capacity throughout (requires `maxSurge` > 0 to have room to create new pods first). Trade-off versus Blue-Green/Canary: no traffic-percentage control (it's binary per-pod, not weighted), and both old and new versions serve real traffic simultaneously for the rollout's duration — which makes it unsuitable alone for changes requiring strict version isolation (e.g. an incompatible API contract change) without additional safeguards like API versioning or feature flags.

Q141 Recreate deployment.

```
strategy:
  type: Recreate
```

All old pods are terminated **before** any new ones are created — guaranteed downtime during the gap, in exchange for guaranteeing **zero version overlap** at any point in time. This is the right (sometimes only correct) choice when running old and new versions simultaneously would cause active harm rather than just inconvenience: an incompatible database schema migration where the old code would corrupt data under the new schema, a singleton workload where two versions running at once would conflict (a leader-elected controller, certain licensing constraints), or local state/cache that can't be shared or safely duplicated across versions. Interview signal: know when *not* to default to RollingUpdate — "always use rolling update" is the wrong universal answer.

Q142 GitOps deployment.

The core principle: **Git is the single source of truth for desired cluster state**, and a controller running inside the cluster (Argo CD, Flux) continuously reconciles actual state to match it — rather than a CI pipeline reaching out and pushing changes via `kubectl apply` with cluster credentials. This flips the traditional push model to pull: the cluster pulls its own desired state from Git on an interval/webhook, rather than an external system pushing into it.

Benefits worth naming explicitly: the entire deployment history is Git history (trivial audit trail, and rollback is just `git revert`); no long-lived cluster credentials need to live in CI (the GitOps controller's in-cluster credentials never leave the cluster); and drift detection is automatic and continuous (Argo CD flags `OutOfSync` the moment someone manually `kubectl edit`s something, rather than that drift silently persisting until the next deploy). Trade-off: an extra layer of indirection (config repo separate from app repo) that teams new to it sometimes find adds friction for simple changes.

Q143 ArgoCD sync process.

Argo CD's controller continuously compares **live state** (what's actually in the cluster) against **target state** (what's declared in the Git repo at the tracked revision), reporting the app as `Synced` or `OutOfSync` based on a live diff — independent of and in addition to `Healthy` / `Degraded`, which reflects whether the resources are actually working (pods Running, Ingress has an address, etc.).

```
argocd app get <app>
argocd app sync <app> --prune
argocd app diff <app>
```

Sync can be **manual** (a human/pipeline triggers it explicitly) or **automated** (auto-sync on every Git change, optionally with `selfHeal: true` to also auto-revert manual cluster drift back to Git state, and `prune: true` to delete resources removed from Git). Sync hooks (`PreSync` / `Sync` / `PostSync`, e.g. a database migration Job that must run before the new app version rolls out) let you sequence non-declarative steps into an otherwise declarative reconciliation loop.

Q144 Helm deployment flow.

```
helm upgrade --install <release> ./chart -f values-prod.yaml --namespace <ns> --atomic --wait
helm rollback <release> <revision>
helm history <release>
```

Helm templates raw manifests using chart templates plus environment-specific `values.yaml` overlays, then applies the rendered result as a tracked **release** with its own revision history, independent of Kubernetes' own object history. `--atomic` is worth highlighting specifically: it automatically rolls back to the previous release if the upgrade fails (pods never become healthy within the wait timeout), which is the safety property that turns a bare `helm upgrade` into something production-safe. In a GitOps setup, Argo CD/Flux typically renders the Helm chart itself as part of reconciliation rather than a human running `helm upgrade` directly — worth mentioning that distinction, since "Helm" and "GitOps" are complementary layers, not competing choices.

Q145 GitHub Actions → ArgoCD.

The key architectural point: GitHub Actions never touches the cluster directly — its job ends the moment it commits a manifest/values change to the config repo, and Argo CD (running inside the cluster) picks that change up independently. This is what makes it GitOps rather than "CI/CD with extra steps."

```
# In the GitHub Actions workflow, after build+push:
- name: Update image tag in config repo
  run: |
    git clone https://github.com/org/config-repo
    cd config-repo
    yq -i '.image.tag = "${{ github.sha }}"' apps/myapp/values-prod.yaml
    git commit -am "Update myapp image to ${{ github.sha }}"
    git push
```

The full flow: PR merged to app repo → Actions builds & pushes an image tagged with the commit SHA (immutable, never `:latest`) → Actions updates the corresponding tag/digest in the separate GitOps config repo → Argo CD detects the config repo change (webhook or poll interval) → syncs the cluster to match → reports back to Slack/GitHub via a notification webhook once `Healthy`. Splitting app-repo CI from config-repo CD this way is itself the design decision worth calling out.

Q146 Jenkins → Docker → Registry → ArgoCD.

Same architectural shape as the GitHub Actions flow, described with a Jenkins pipeline (`Jenkinsfile`) instead of Actions — useful to be fluent in both since many regulated/legacy enterprises (banks, insurance) still run Jenkins alongside newer GitHub Actions adoption.

```
pipeline {
  stages {
    stage('Build')  { steps { sh 'docker build -t $REGISTRY/app:$GIT_COMMIT .' } }
    stage('Scan')   { steps { sh 'trivy image $REGISTRY/app:$GIT_COMMIT' } }
    stage('Push')   { steps { sh 'docker push $REGISTRY/app:$GIT_COMMIT' } }
    stage('Update GitOps repo') {
      steps { sh 'yq -i ".image.tag=\\"$GIT_COMMIT\\"" config-repo/values.yaml && git commit -am "deploy
$GIT_COMMIT" && git push' }
    }
  }
}
```

Jenkins' role stops at "push a new image and update the config repo" — it has no cluster credentials and makes no direct cluster calls, same separation-of-concerns principle as above. Worth mentioning: Jenkins agents doing Docker builds need either privileged Docker-in-Docker or a rootless builder (Kaniko/Buildah) if running inside Kubernetes itself as a Jenkins agent, which is its own security consideration.

Q147 Rollback strategy.

Good rollback strategy is designed *before* the incident, not improvised during one — the strongest interview answer names the mechanism at each layer: application code (`kubectl rollout undo` / `helm rollback` / `git revert` on the GitOps config repo, whichever is your actual source of truth — for GitOps, reverting Git is the correct rollback, not `kubectl rollout undo`, since the latter fights the reconciler); database (this is the hard part — see next question, since most schema changes aren't trivially reversible); and traffic (for canary/blue-green, rollback is just re-weighting traffic back to the known-good version, which is far faster than a rolling-update rollback that has to replace pods).

Define rollback triggers as objective thresholds ahead of time (error rate > X%, p99 latency > Y ms, sustained for Z minutes) rather than a judgment call made under pressure, and automate the trigger where possible (Argo Rollouts' automated analysis is the concrete example to cite).

Q148 Database migration handling.

The core technique to name: **expand/contract** (a.k.a. parallel-change) — split every schema change into backward-compatible steps so old and new application code can both run against the database simultaneously during rollout, which every zero-downtime deployment strategy above secretly depends on.

- **Expand**: add the new column/table nullable or with a default, without removing the old one; deploy application code that writes to both old and new; backfill existing data.
- **Migrate**: deploy application code that reads from the new structure, still writing to both for safety.
- **Contract**: once fully rolled out and stable, a separate later deployment removes the old column/table.

Never run destructive/irreversible migrations (dropping a column, renaming in place) in the same deploy as the application change that depends on it — that's what removes your ability to roll back cleanly. In Kubernetes specifically, migrations typically run as a `Job` (or an Argo CD `PreSync` hook) that must complete successfully before the new Deployment rolls out, never as an init container racing against multiple pod replicas starting in parallel.

Q149 Zero downtime deployment.

Zero downtime is the sum of several individually-necessary pieces, and a strong answer lists all of them rather than just "use rolling updates": a **readiness probe** so traffic never reaches a pod before it's actually ready; `preStop` hook with a short sleep (e.g. 5–10s) so the pod is removed from Service Endpoints *before* SIGTERM is sent, closing the race where a connection routes to a pod that's already begun terminating; graceful shutdown in the application itself (catch SIGTERM, stop accepting new connections, finish in-flight requests, then exit — within `terminationGracePeriodSeconds`); `maxUnavailable: 0` on the rollout so full capacity is maintained throughout; and, at the database layer, backward-compatible migrations (expand/contract, previous question) so old and new pods running simultaneously during the rollout don't break against each other's expectations of the schema.

Miss any one of these and you get brief-but-real error spikes during every deploy, even though "the deployment succeeded."

Q150 Feature flags.

The key distinction to draw explicitly: feature flags **decouple deployment from release**. Deploying new code (shipping it to production, behind a flag defaulted off) becomes a low-risk, frequent, boring event; releasing a feature (flipping the flag on for some or all users) becomes a separate, instant, and instantly-reversible decision that doesn't require a new deployment or rollback at all — you just flip the flag back off.

This matters for the deployment-strategy conversation specifically because it changes the risk calculus: a risky change can go out via a boring rolling update (since it's dark-launched, doing nothing yet), then get exposed gradually to real users via flag percentage rollout, independent of and often faster-iterating than the Kubernetes-level rollout mechanics. Worth naming the trade-off too: flag debt (stale flags left in code long after a feature is fully rolled out) is a real, common maintenance cost if flag lifecycle isn't managed deliberately (expiry dates, cleanup as part of Definition of Done).

Q151 Pre-deployment validation.

Everything that should fail the pipeline **before** anything touches production, roughly in the order a pipeline should run them (fail fast, cheapest checks first): manifest linting/schema validation (`kubeconform` / `kubeval` against the target cluster's actual API schema, catching removed/deprecated API versions before apply); policy validation (Kyverno/OPA Conftest run against the rendered manifests in CI, the same policies that will be enforced at admission time, so violations are caught in the pipeline instead of at deploy time); image vulnerability scanning with a defined severity gate; unit and integration tests; and a `helm template --dry-run` or `kubectl apply --dry-run=server` against a real API server to catch anything a purely static check would miss (admission webhook rejections, CRD schema mismatches).

The unifying principle worth stating: every one of these should also be enforceable at admission time in the cluster as a second, independent layer — pre-deployment validation is a fast-feedback convenience, not a substitute for cluster-side enforcement, since not every change to the cluster goes through the pipeline (manual `kubectl` access, other tooling).

Q152 Post-deployment validation.

The shift in mindset from pre-deployment: you're no longer checking "is this safe to ship," you're checking "did shipping it actually work," against real production signal. Layers, in order of how quickly they'd catch a problem: **rollout status** itself (`kubectl rollout status` /Argo CD `Healthy`, confirming pods actually became Ready, not just that the apply succeeded); **smoke tests** against real production endpoints (next question); **SLO/error-budget dashboards** watched for a defined bake period (the specific window is a real design decision — long enough to catch a slow leak or a delayed cache-related regression, short enough to not block the next deploy indefinitely); and **synthetic monitoring** / real-user-monitoring correlated against the deployment timestamp specifically (so a dashboard annotation marks exactly when the rollout happened, making before/after comparison unambiguous rather than relying on someone remembering the exact time).

Q153 Smoke testing.

A deliberately small, fast set of checks against the *real deployed* environment confirming the absolute basics work — not a full regression suite, and that distinction is the point: smoke tests should run in seconds to a couple of minutes, gating the rollout from proceeding to the next stage (next canary step, or 100% traffic) rather than running the full test suite against production, which would be both slow and risky to run repeatedly against live traffic.

```
# Example smoke-test gate as a post-deploy pipeline step
curl -f https://api.internal/healthz || exit 1
curl -f https://api.internal/v1/status | jq -e '.status == "ok"'
# A real end-to-end critical-path check, not just a health endpoint:
curl -f -X POST https://api.internal/v1/orders/test-checkout
```

Good smoke tests cover the critical business path end-to-end (can a user actually log in, can a checkout actually complete) rather than only pinging a liveness endpoint, since a pod can report healthy while a downstream dependency it needs for real functionality is broken.

Q154 Health checks.

In a deployment-process context (versus the earlier Pod Issues section's probe-mechanics focus), the interview angle is usually: how do health checks fit into the release pipeline as gates? A rollout's success/failure determination should be driven by the **same** readiness signal that gates production traffic (the pod's readiness probe), not a separate, weaker "did the pod start" check — if `progressDeadlineSeconds` /Argo Rollouts' analysis is watching readiness, a bad deploy is caught and halted automatically without a human needing to notice first.

Also worth distinguishing application-level health (is this specific service okay) from dependency health (are the things it depends on okay) — a good production health-check design surfaces both separately (e.g. `/healthz` for "am I alive" used by liveness, and a richer `/readyz` checking DB/cache connectivity used by readiness), rather than one endpoint conflating them, which is exactly the design mistake that causes a DB blip to trigger unnecessary liveness-driven restarts (see Q33).

Q155 Progressive delivery.

The umbrella term for canary + automated analysis + automated rollback, as an evolution beyond "just do a canary and have someone watch a dashboard." The tooling to name: **Argo Rollouts** or **Flagger**, both of which replace a plain Deployment object with a custom resource that natively understands weighted traffic shifting, integrates with a metrics provider (Prometheus, Datadog) to automatically query defined success criteria at each step, and can promote or abort *without human intervention* based on those results.

```
kind: AnalysisTemplate
spec:
  metrics:
    - name: error-rate
      successCondition: result < 0.01
      provider:
        prometheus:
          query: |
            sum(rate(http_requests_total{status=~"5..",app="myapp"}[5m]))
            / sum(rate(http_requests_total{app="myapp"}[5m]))
```

The strategic point worth making: progressive delivery moves release-safety decisions from "a human watches a dashboard and decides" to "a defined, version-controlled policy decides," which is both faster (no waiting on a human to notice) and more consistent (the same bar is applied every time, not whatever the on-call engineer happens to notice that day).

Q156 Deployment freeze.

A defined window (year-end retail peak, a regulated change-control blackout period, a major event) where non-emergency deployments are explicitly paused — the operational maturity signal in this answer is having it be a **policy enforced by tooling**, not a verbal agreement people are supposed to remember: Argo CD supports pausing auto-sync for specific applications/projects, and a policy engine (Kyverno/OPA) can outright block deployments during a defined window at the admission layer, independent of whether someone remembers to hold off in CI.

Always pair a freeze policy with an explicit **emergency-change exception path** (who can approve breaking the freeze, and how that approval itself is logged/audited) — a freeze with no override mechanism just means a genuine production-down incident during the freeze window has no sanctioned way to get fixed quickly, which defeats the purpose of having change control in the first place. This maps directly onto ITIL/CAB-style change management in regulated environments (banking, healthcare), where freeze windows are often a compliance requirement, not just an operational nicety.

PART IV

Kubernetes Security

25 questions · cluster hardening, admission control, runtime security & supply chain

Q157 How do you secure a Kubernetes cluster?

Structure the answer in layers (defense in depth), which is itself the signal interviewers want — naming one control isn't enough:

- **Cluster/infra**: private API endpoint or restricted CIDR allow-list, control-plane audit logging enabled, nodes on hardened AMIs, latest supported Kubernetes version (patched CVEs).
- **Authentication/authorization**: OIDC/IAM-federated auth (no long-lived static kubeconfigs handed out broadly), least-privilege RBAC, anonymous auth disabled.
- **Admission control**: Pod Security Admission at `restricted` for application namespaces, plus a policy engine (Kyverno/OPA Gatekeeper) for anything PSA doesn't cover (image provenance, required labels, resource limits).
- **Workload**: non-root, read-only root filesystem, dropped Linux capabilities, no privilege escalation, resource limits set.
- **Network**: default-deny NetworkPolicies with explicit allow-lists.
- **Secrets**: encryption at rest (envelope encryption via KMS), short-lived dynamic secrets over static ones.
- **Supply chain**: image scanning gates in CI, signed images verified at admission.
- **Runtime**: Falco or equivalent for anomalous behavior detection.

Q158 API Server security.

The API server is the single most security-critical component — every other control in the cluster is enforced *through* it. Key hardening points: `--anonymous-auth=false` (disable unauthenticated access entirely); `--authorization-mode=Node,RBAC` (never `AlwaysAllow` in production); restrict or disable the insecure port entirely (modern Kubernetes removed it by default, but verify on older/self-managed clusters); TLS enforced for all client and etcd communication with short-lived, rotated certificates; admission controllers enabled for `PodSecurity`, `NodeRestriction` (prevents a compromised kubelet from modifying resources outside its own node's scope — an important, often-missed control) and any custom validating/mutating webhooks; and audit logging enabled and shipped somewhere durable and access-controlled, not just written to local disk on the control-plane node.

Q159 RBAC.

Role-Based Access Control: `Role` / `RoleBinding` (namespace-scoped) and `ClusterRole` / `ClusterRoleBinding` (cluster-scoped, though a ClusterRole can also be bound namespace-locally via a RoleBinding — a subtlety worth stating, since it's how you reuse one ClusterRole's rule-set across many namespaces without duplicating YAML). Rules specify `apiGroups`, `resources`, and `verbs`; access is purely additive across all bindings that apply to a subject — there is no explicit deny.

```
kubectl auth can-i list pods --as=system:serviceaccount:ns:sa-name -n ns
kubectl create role pod-reader --verb=get,list,watch --resource=pods -n ns
kubectl create rolebinding read-pods --role=pod-reader --serviceaccount=ns:sa-name -n ns
```

Least-privilege in practice: avoid wildcard `resources: ["*"]` / `verbs: ["*"]`, prefer namespaced Roles over cluster-wide ClusterRoleBindings whenever the workload only needs access within its own namespace, and periodically audit for unused/over-broad bindings (`kubectl-who-can` or similar tooling helps here).

Q160 Pod Security Admission.

The built-in, in-tree replacement for the deprecated-and-removed PodSecurityPolicy, enforced via namespace labels rather than a cluster-wide policy object — a meaningfully simpler model. Three standard levels: `privileged` (unrestricted), `baseline` (blocks known privilege-escalation vectors while staying broadly compatible with common workloads), and `restricted` (the hardened, heavily locked-down profile — non-root, no privilege escalation, dropped capabilities, seccomp required).

```
kubectl label ns my-app \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/audit=restricted \
  pod-security.kubernetes.io/warn=restricted
```

The three modes (`enforce` / `audit` / `warn`) can be set independently, which is the key rollout pattern to mention: start with `audit` + `warn` at `restricted` cluster-wide to see what would break, fix workloads, then flip to `enforce` once clean — rather than a hard cutover that breaks everything at once.

Q161 Admission Controllers.

Plugins that intercept API requests after authentication/authorization but before persistence to etcd, in two flavors: **mutating** (can modify the object — e.g. injecting a sidecar, defaulting a field) run first, then **validating** (can only accept/reject, not modify) run second, on the mutated result. Built-in examples worth naming: `NamespaceLifecycle` , `LimitRanger` , `ResourceQuota` , `PodSecurity` , `NodeRestriction` ; custom ones are implemented as webhooks (`MutatingWebhookConfiguration` / `ValidatingWebhookConfiguration`) pointing at your own admission service (Kyverno, OPA Gatekeeper, or a bespoke one).

Operational gotcha worth mentioning proactively: a webhook configured with `failurePolicy: Fail` that becomes unreachable can block *all* matching API operations cluster-wide, not just the one it was meant to gate — always set a sensible `timeoutSeconds` and think carefully about `failurePolicy` for anything broad-scoped, since a bug in your policy engine shouldn't be able to take down the whole cluster's ability to create pods.

Q162 OPA Gatekeeper.

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
  name: require-team-label
spec:
  match:
    kinds: [{apiGroups: [""], kinds: ["Namespace"]}]
  parameters:
    labels: ["team"]
```

Open Policy Agent's Kubernetes integration: policies are written in **Rego** (a purpose-built declarative policy language) and packaged as reusable `ConstraintTemplates` , then instantiated cluster-specifically as `Constraint` objects (like the example above). Gatekeeper runs as a validating (and optionally mutating) admission webhook. The main trade-off versus Kyverno, worth stating explicitly since it's a common interview follow-up: Rego has real expressive power for complex cross-resource policies but a steeper learning curve, whereas Kyverno's policies are plain Kubernetes-native YAML, which most teams find faster to author and review, at some cost to expressiveness for the most complex policy logic.

Q163 Kyverno.

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-privileged
spec:
  validationFailureAction: Enforce
  rules:
    - name: no-privileged-containers
      match: {resources: {kinds: ["Pod"]}}
      validate:
        message: "Privileged containers are not allowed."
        pattern:
          spec:
            containers:
              - =(securityContext):
                  =(privileged): "false"
```

A Kubernetes-native policy engine where policies are themselves Kubernetes resources written in plain YAML (no separate DSL to learn, unlike Gatekeeper's Rego) — supports `validate` (allow/deny), `mutate` (auto-fix/inject defaults, e.g. auto-adding resource limits if missing rather than just rejecting), and `generate` (auto-create related resources, e.g. a default NetworkPolicy whenever a new namespace is created) rule types. `validationFailureAction: Audit` vs `Enforce` gives the same safe-rollout pattern as PSA's audit/warn/enforce split — ship new policies in Audit mode first, review violations, then flip to Enforce.

Q164 Falco.

Runtime security — the layer that catches what admission control and image scanning can't, because it observes actual behavior at runtime rather than static configuration. Falco taps kernel-level syscalls (via eBPF or a kernel module) and evaluates them against a rules engine in real time, flagging things like: a shell spawned inside a container that shouldn't have interactive access, an unexpected outbound connection from a pod that normally has none, a write to a sensitive path like `/etc/shadow`, or a process attempting to load a kernel module from inside a container.

```
- rule: Terminal shell in container
  desc: A shell was spawned by a container
  condition: >
    spawned_process and container
    and shell_procs and proc.tty != 0
  output: "Shell spawned in container (user=%user.name container=%container.name)"
  priority: WARNING
```

The distinction worth stating clearly: admission control and image scanning are *preventive* (stop bad things before they run); Falco is *detective* (notice bad things happening right now) — both are needed, since preventive controls can't catch a zero-day or a legitimate-looking process behaving maliciously post-deployment.

Q165 Network Policies.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: default-deny, namespace: my-app}
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
```

The security-posture default worth advocating for explicitly: start every namespace with a **default-deny-all** policy (empty `podSelector` matches every pod in the namespace, with no `ingress` / `egress` rules meaning nothing is allowed), then add narrowly-scoped allow rules per legitimate traffic flow. This is a meaningful mindset shift from "block the bad stuff" to "explicitly allow only what's needed" — the same whitelist-over-blacklist principle as good firewall design generally. Remember it requires a NetworkPolicy-enforcing CNI (Calico, Cilium; not all CNIs enforce it) and that default-deny commonly breaks DNS/health-check traffic if `kube-system` isn't explicitly allowed — always test thoroughly in a lower environment before enforcing cluster-wide.

Q166 Secrets Encryption.

By default, Kubernetes Secrets are only **base64-encoded** in etcd — not encrypted — which means anyone with etcd access (or an etcd backup file) can trivially read every Secret in the cluster. Encryption at rest requires explicit configuration: an `EncryptionConfiguration` on the API server specifying a provider (KMS-backed is the production-grade choice, e.g. AWS KMS via EKS's native "envelope encryption" feature, which lets you enable it with one setting rather than self-managing the encryption config file).

```
aws eks associate-encryption-config --cluster-name <cluster> \
  --encryption-config '[{"resources":["secrets"],"provider":{"keyArn":"<kms-key-arn>"}}]'
```

Worth stating as a nuance: enabling encryption going forward doesn't retroactively encrypt *existing* Secrets already in etcd — you need to re-write them (e.g. `kubectl get secrets -A -o json | kubectl replace -f -`) after enabling encryption for existing data to actually be protected. Beyond etcd-at-rest encryption, the stronger modern pattern is avoiding static Secrets objects in the cluster altogether where possible, via External Secrets Operator pulling short-lived credentials from Vault/AWS Secrets Manager at pod-start time.

Q167 Image Scanning.

```
trivy image --severity CRITICAL,HIGH --exit-code 1 myregistry/app:1.2.3
```

Effective image scanning is a **gate**, not a report — the meaningful design decision is where in the pipeline it blocks: in CI (fails the build on defined severity thresholds, `--exit-code 1` above), in the registry (ECR image scanning, Trivy/Grype/Clair run automatically on push, with results queryable before deploy), and again at **admission time** in the cluster (Kyverno's `verifyImages` can require a passing scan attestation before allowing a pod to run an image, catching anything that slipped past CI, or an image deployed via a path that bypassed the pipeline entirely). Scanning at only one of these three points leaves a gap; the strongest answer names the gate at each stage rather than treating "we scan images" as a single yes/no capability. Also worth mentioning: scan base images independently and keep them current, since a perfectly clean scan today can go stale the moment a new CVE is published against a dependency already baked into a running image.

Q168 Runtime Security.

The umbrella category covering everything that protects a container *while it's running*, as distinct from build-time (image scanning) and deploy-time (admission control) controls: Falco-style syscall/behavior monitoring (Q164); kernel-level confinement mechanisms — seccomp (restricting which syscalls a process may make at all), AppArmor/SELinux (restricting file/network/capability access by profile); dropped Linux capabilities (`drop: ["ALL"]` then add back only what's genuinely needed, e.g. `NET_BIND_SERVICE` if binding to a privileged port); and read-only root filesystems limiting what a compromised process can persist or modify.

The unifying principle to state explicitly: runtime security assumes prevention will eventually fail for something (a zero-day, a supply-chain compromise in a dependency) and focuses on **limiting blast radius** and **detecting anomalous behavior quickly** when it does, rather than only trying to prevent every possible bad thing from happening in the first place.

Q169 CIS Benchmark.

```
kube-bench run --targets master,node,etcd,policies
```

The Center for Internet Security publishes a detailed, versioned Kubernetes Benchmark — a checklist of specific, numbered configuration recommendations across the control plane, worker nodes, etcd, and policies (RBAC minimization, PSA enforcement, etc.), each with a rationale and a remediation command. `kube-bench` (Aqua Security's open-source tool) automates running the benchmark's checks against a live cluster and reports PASS/FAIL/WARN per item with the specific remediation.

Practical note worth including: on managed control planes (EKS), many control-plane-level checks aren't your responsibility or aren't checkable at all (AWS manages that layer) — run `kube-bench` with the `eks` profile, which scopes checks to what's actually under the customer's control in a shared-responsibility model, rather than flagging findings for infrastructure you don't manage. CIS Benchmark compliance is also frequently an explicit requirement in regulated environments (PCI-DSS, SOC 2) worth connecting to the compliance side of the conversation.

Q170 Audit Logging.

Records every request made to the API server — who (user/SA), what (verb + resource), when, and the outcome — independent of and far more complete than `kubectl get events`, which is short-lived and limited to a subset of object lifecycle events. This is the primary forensic source after any security incident (Q102's "unauthorized kubectl access" leans entirely on this being enabled beforehand).

```
aws eks update-cluster-config --name <cluster> \
  --logging '{"clusterLogging":[{"types":["audit","authenticator"],"enabled":true}]}'
```

Audit policy defines **levels** per rule (`None`, `Metadata` only, `Request` body included, `RequestResponse` full body both ways) — `RequestResponse` on every request generates enormous log volume and cost, so a well-designed policy applies richer levels only to sensitive resources (Secrets, RBAC objects) and `Metadata`-only broadly elsewhere. For EKS specifically, enable it via cluster logging (shipped to CloudWatch) rather than assuming it's on by default — it isn't, and this is a very common gap discovered only after an incident already needs it.

Q171 Disable anonymous authentication.

```
kubectl auth can-i --list --as=system:anonymous
```

By default, unauthenticated requests to the API server are mapped to the `system:anonymous` user in the `system:unauthenticated` group — if any RBAC binding (even accidentally, via a broad ClusterRoleBinding) grants that group permissions, anyone who can reach the API server network endpoint at all gets access without ever presenting credentials. Run the command above as a standing check: it should return nothing, or close to it.

On self-managed clusters, `--anonymous-auth=false` is the API server flag; on EKS, anonymous auth is effectively controlled by never binding RBAC permissions to `system:unauthenticated` / `system:anonymous` (rather than a single toggle, since EKS manages the control-plane flags itself) — the actionable takeaway is auditing bindings for that group/user specifically, since it's an easy thing to introduce accidentally via an overly broad "allow everyone to read X" RoleBinding meant for authenticated users.

Q172 Disable privileged containers.

```
pod-security.kubernetes.io/enforce: baseline   # or restricted; both disallow privileged
```

Both `baseline` and `restricted` Pod Security Admission levels disallow `privileged: true` outright — the simplest, cluster-wide way to enforce this is labeling application namespaces with one of those levels rather than relying on every team remembering to omit the field. For the small number of legitimate exceptions (CNI/CSI node DaemonSets in `kube-system` that genuinely need host-level access), scope those namespaces separately at `privileged` level rather than weakening the policy for everyone. A policy-engine rule (Kyverno's `disallow-privileged-containers` from the sample policy library) is the equivalent enforcement mechanism if you need more granular control (allow-list specific namespaces/ServiceAccounts) than PSA's namespace-level-only granularity provides.

Q173 Root filesystem read-only.

```
securityContext:
  readOnlyRootFilesystem: true
volumes:
  - name: tmp
    emptyDir: {}
volumeMounts:
  - name: tmp
    mountPath: /tmp
```

Limits what a compromised process can actually do even after gaining code execution — it can't drop a malicious binary, modify application files in place, or persist a backdoor to the container's own filesystem, since the root filesystem is immutable at runtime. The practical rollout challenge (worth naming, since it's the real-world friction point): most applications need *some* writable path (temp files, a Unix socket, a cache directory), so this requires explicit `emptyDir` volumes mounted at exactly those specific paths rather than a blanket read-only root breaking the app outright — identifying every path an app writes to (via strace or just careful testing) is the actual work involved in adopting this control, not the one-line securityContext setting itself.

Q174 SecurityContext.

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]
seccompProfile:
  type: RuntimeDefault
```

The single object that bundles together most of the workload-level hardening controls discussed across this section, settable at the pod level (applies to all containers) or overridden per-container. A strong "hardened baseline" to recite from memory in an interview: `runAsNonRoot: true` + explicit `runAsUser`, `readOnlyRootFilesystem: true`, `allowPrivilegeEscalation: false` (blocks a process from gaining more privileges than its parent, e.g. via a setuid binary), `capabilities.drop: ["ALL"]` then add back only what's genuinely required, and `seccompProfile.type: RuntimeDefault`. This combination is essentially what the `restricted` Pod Security Admission level enforces — worth connecting those two together explicitly to show you understand PSA is really just automating this checklist.

Q175 Seccomp.

```
securityContext:
  seccompProfile:
    type: RuntimeDefault   # or Localhost with a custom profile file
```

Secure Computing Mode restricts which **syscalls** a container's processes may invoke at all — a much lower-level control than RBAC or NetworkPolicy, operating at the kernel boundary. `RuntimeDefault` uses the container runtime's built-in profile (blocks dozens of rarely-needed, historically-exploited syscalls like certain `ptrace`/kernel-module-loading calls) and is a safe, broadly-compatible default worth enabling everywhere as a baseline. `Localhost` with a custom JSON profile lets you go further — allow-listing only the exact syscalls a specific application actually uses (generated by tracing a real run with `strace` or a tool like `oci-seccomp-bpf-hook`), which meaningfully shrinks the kernel attack surface for that specific workload but requires real per-app profiling effort to get right without breaking functionality.

Q176 AppArmor.

```
metadata:
  annotations:
    container.apparmor.security.beta.kubernetes.io/my-container: localhost/my-profile
```

A Linux Security Module (path-based, unlike SELinux's label-based model) that confines a process by explicit profile rules — which files it can read/write/execute, which network operations it can perform, which capabilities it may use — loaded and named on each node, then referenced by a container via annotation (older mechanism) or the newer `securityContext.appArmorProfile` field. Complements seccomp rather than replacing it: seccomp restricts *which syscalls* can be made at all; AppArmor restricts *what those syscalls can act on* (which specific files/paths/resources) — using both together is the standard hardened-workload pattern, each closing a different half of the gap.

Practical note: profiles must be loaded on every node that might run the workload (via a node-bootstrap step or a DaemonSet), and it's Ubuntu/Debian-family default (Amazon Linux 2/2023 uses SELinux instead) — relevant when choosing/confirming node AMI family for a hardening initiative.

Q177 SELinux.

```
securityContext:
  seLinuxOptions:
    level: "s0:c123,c456"
```

The label-based counterpart to AppArmor's path-based model, and the default mandatory access control mechanism on RHEL/Amazon Linux node families (relevant for EKS specifically, since Amazon Linux 2/2023 nodes use SELinux, while AppArmor is the Ubuntu-family default) — every process and file gets a security label, and policy rules define which labels may interact with which, enforced independent of standard Unix file permissions (so even a process running as root is still confined by its SELinux context). Kubernetes can auto-assign a unique MCS (Multi-Category Security) label per pod, which provides an additional isolation layer between pods sharing a node even without full container-runtime-level isolation — a defense specifically against one compromised container reading/tampering with another's files on the same node via a shared-kernel escape.

Q178 Non-root containers.

```
FROM node:20-slim
RUN useradd -u 10001 -m appuser
USER appuser
# ... vs enforcing it at the k8s level regardless of the image:
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
```

Two layers work together here, worth distinguishing explicitly: the **image** itself should be built to run as non-root (a `USER` directive in the Dockerfile, and file permissions set for that user during the build) *and* the **pod spec** should enforce `runAsNonRoot: true` regardless — the Kubernetes-level setting is what actually guarantees it even if someone deploys an image that wasn't built correctly (the pod simply fails to start rather than silently running as root). Common friction migrating existing images: many base images and their default entrypoints assume root (binding to a privileged port like 80/443 requires root by default unless `NET_BIND_SERVICE` capability is explicitly added back, or you simply use a port >1024 and let a Service/Ingress remap it, which is usually the simpler fix).

Q179 Service Accounts.

Kubernetes' identity mechanism for **pods** talking to the API server (distinct from User accounts, which are for humans and aren't even a first-class API object — they're managed entirely externally via whatever auth method the cluster uses). Every namespace gets a `default` ServiceAccount automatically, and every pod uses it unless one is explicitly specified — a real, common security gap is workloads unintentionally running under `default` with whatever permissions happen to be bound to it, rather than a purpose-specific SA with exactly the access that workload needs.

```
kubectl get sa default -n my-app -o yaml
automountServiceAccountToken: false   # disable auto-mounting when a pod doesn't need API access at all
```

Hardening checklist: create a dedicated SA per workload (never share one broadly across unrelated apps), set `automountServiceAccountToken: false` on the `default` SA and on any workload that genuinely doesn't need to talk to the API server at all (most application pods don't), and use the modern **bound, time-limited, audience-scoped tokens** (the default since 1.24, replacing long-lived Secret-based tokens) rather than legacy non-expiring ones.

Q180 IAM Roles for Service Accounts (IRSA).

EKS's mechanism for giving individual pods fine-grained, temporary AWS credentials scoped to exactly what that specific workload needs — without it, every pod on a node inherits whatever broad IAM permissions the *node's* instance role has, which is a major over-privilege problem at scale (one compromised pod effectively has every permission every workload on that node needs, combined).

```
eksctl create iamserviceaccount \
  --cluster my-cluster --namespace my-app --name my-sa \
  --attach-policy-arn arn:aws:iam::123456789012:policy/MyAppPolicy \
  --approve
```

Mechanically: the cluster's OIDC provider is registered with IAM, the ServiceAccount carries a `role-arn` annotation, and a mutating webhook injects a projected, audience-scoped token plus `AWS_ROLE_ARN` / `AWS_WEB_IDENTITY_TOKEN_FILE` env vars into any pod using that SA — the AWS SDK then automatically exchanges that token for temporary STS credentials scoped to exactly the IAM role's permissions. This is the mechanism underlying nearly every EKS add-on's own permissions (EBS CSI, ALB Controller, ExternalDNS, all covered earlier) and is the standard answer to "how do you avoid giving every pod the node's full IAM permissions."

Q181 Supply chain security.

Securing everything *upstream* of the running workload — the code, dependencies, build process, and artifact provenance — since a perfectly hardened cluster still runs whatever image you feed it. Layers to name: **dependency scanning** (SCA tools flagging known-vulnerable libraries in application code, not just OS packages in the base image); **SBOM generation** (Software Bill of Materials — a complete manifest of every component in an image, generated at build time via Syft or similar, so you can instantly answer "are we affected by CVE-X" across your whole fleet without re-scanning everything from scratch); **image signing and verification** (Cosign/Sigstore signing images at build time, with admission-time verification via Kyverno's `verifyImages` rejecting any unsigned or improperly-signed image — preventing an attacker who compromises the registry from substituting a malicious image with the same tag); and **build provenance/SLSA** attestations proving an image was built by the expected CI pipeline from the expected source commit, not tampered with in between.

The framing that ties it together for an interview answer: supply-chain security assumes an attacker's easiest path isn't attacking the running cluster directly, but compromising something upstream that the cluster will trust and run automatically — a compromised dependency, a poisoned base image, or a tampered build pipeline.

PART V

TLS & Certificates

40 questions · PKI internals, cert-manager, ACME, rotation & certificate management

TLS & Certificate Concepts

Q182 How TLS works inside Kubernetes.

Kubernetes is built on TLS almost everywhere internally, not just at the ingress edge — a good answer maps out the full set of trust relationships rather than just "the API is HTTPS":

- **API server serving cert**: presented to every client, signed by the cluster's CA.
- **Client certs**: kubelets, controller-manager, scheduler, and admin users each authenticate to the API server via their own client cert, signed by the same CA, validated against the CA's public cert.
- **kubelet serving cert**: the API server acts as a *client* back to each kubelet (for `exec` / `logs` / `port-forward`), which is its own separate TLS relationship in the opposite direction.
- **etcd**: peer-to-peer TLS between etcd members, plus client TLS between the API server and etcd — both usually on a separate, dedicated CA from the main cluster CA.
- **ServiceAccount tokens**: not TLS certs themselves, but JWTs whose signature is verified using a dedicated service-account signing key — a related but distinct trust chain.

Every one of these certs expires and needs rotation — the "what could go wrong" half of this question is almost always about one of these expiring silently (covered in the specific expired-cert questions below).

Q183 API Server certificate.

```
openssl x509 -in apiserver.crt -noout -text | grep -A2 "Subject Alternative Name"
kubeadm certs check-expiration
```

The serving certificate the API server presents to every client — must include every hostname/IP clients might use to reach it as a SAN: the cluster's internal Service IP (typically `10.96.0.1` or similar), `kubernetes.default.svc` and its variants, the actual load-balanced endpoint hostname, and (for kubeadm clusters) each control-plane node's IP if reached directly. A common real incident: adding a new control-plane node or changing the LB endpoint without regenerating this cert to include the new SAN, causing clients hitting the new path to fail TLS verification even though the cert is otherwise valid and unexpired. On EKS this is entirely managed by AWS; on kubeadm/self-managed clusters, it's generated by `kubeadm init` / `kubeadm certs renew` from the cluster CA.

Q184 kubelet certificate.

Kubelet actually holds **two** distinct certs, easy to conflate but serving opposite directions of trust: a **client cert** it presents when authenticating *to* the API server (identity `system:node:<name>`, authorized via the `Node` authorization mode plus `NodeRestriction` admission), and a **serving cert** it presents when the API server connects *to it* for `exec` / `logs` / `attach` / `port-forward` operations.

```
kubectl get csr   # kubelet serving-cert rotation requests appear here if not auto-approved
cat /var/lib/kubelet/config.yaml | grep -A3 serverTLSBootstrap
```

`rotateCertificates: true` (client cert) and `serverTLSBootstrap: true` (serving cert) in kubelet config enable automatic rotation via the certificates API before expiry — without these enabled, expired kubelet certs are a classic manual-renewal miss that silently breaks `kubectl logs` / `exec` to a node while the node itself still reports Ready (since heartbeats use a different path than the exec/logs cert).

Q185 Client certificate.

```
openssl x509 -in client.crt -noout -subject
# CN=jane.doe,O=platform-admins
```

Client certs authenticate a user or component to the API server — the certificate's `CN` **(Common Name)** becomes the Kubernetes username, and any `O` **(Organization)** fields become group memberships, both fed directly into RBAC evaluation. This is the classic (if increasingly legacy) way humans authenticate to a self-managed cluster before OIDC federation became the norm; it has a real operational weakness worth naming explicitly — there's no built-in revocation mechanism for individual client certs in vanilla Kubernetes (no CRL/OCSP checked by default), so a leaked or off-boarded user's cert remains valid until it naturally expires unless you rotate the entire CA. This is exactly why most production clusters today prefer OIDC-based auth (short-lived tokens, centrally revocable through the identity provider) over long-lived client certs for human users, reserving client certs mainly for cluster components.

Q186 Service Account token.

```
kubectl create token my-sa --duration=1h
kubectl exec <pod> -- cat /var/run/secrets/kubernetes.io/serviceaccount/token | cut -d. -f2 | base64 -d
```

Not a certificate but worth distinguishing precisely from the certs above since they're often confused: a signed **JWT**, verified via the API server checking its signature against a dedicated service-account signing key (not the cluster's TLS CA). Since Kubernetes 1.24+, the default mechanism is **bound, time-limited, audience-scoped projected tokens** (via the `TokenRequest` API, auto-refreshed by the kubelet before expiry, typically ~1 hour validity) — a major security improvement over the old mechanism (pre-1.24 default) of a static, non-expiring token stored directly in a Secret object, which if leaked was valid forever with no built-in expiry. `audience` scoping means a token minted for talking to the API server can't be replayed against a different audience (e.g. a cloud provider's STS endpoint) even if intercepted.

Q187 CA certificate.

The root of trust that signs every other cluster-internal certificate — every client and server validates the certs they receive against this CA's public certificate, which is why its own compromise or unexpected expiry is catastrophic (every cert it ever signed becomes untrustworthy at once, requiring a full cluster-wide re-issuance, not just one component's renewal).

```
kubeadm certs check-expiration | grep "ca "
openssl x509 -in /etc/kubernetes/pki/ca.crt -noout -enddate
```

Production clusters typically run **two separate CAs**: one for general cluster components (API server, kubelet, client certs) and a distinct one for etcd's peer/client TLS — isolating etcd's trust chain limits blast radius if one CA is ever compromised. CA certs are deliberately given a very long validity (kubeadm defaults to 10 years) precisely because rotating the CA itself is a disruptive, all-hands operation requiring every downstream cert to be reissued and every client's trust store updated — unlike routine leaf-certificate rotation, which should be frequent and automated.

Q188 Certificate rotation.

The operational goal: certs renew automatically, well before expiry, with zero manual intervention and zero downtime — manual rotation is a process that should exist as a documented fallback, not the default path. Per component: **kubelet client/serving certs** auto-rotate via `rotateCertificates` / `serverTLSBootstrap` (Q184); **API server/controller-manager/scheduler certs** on kubeadm clusters rotate via `kubeadm certs renew`, ideally automated as a scheduled job well ahead of the 1-year default expiry rather than discovered when something breaks; **Ingress/application TLS certs** rotate via cert-manager, which handles the entire renew-before-expiry lifecycle automatically once configured (the modern standard approach, covered in depth below).

```
kubeadm certs check-expiration   # run this proactively, not reactively
kubectl get certificate -A       # cert-manager's view of everything it manages
```

The single most valuable habit to mention: alert on certificate expiry *well* ahead of time (30 days is a common threshold) rather than finding out from an outage — nearly every "expired certificate" incident in this handbook is a monitoring gap, not a genuinely unavoidable failure.

Q189 kubeadm certificate renew.

```
kubeadm certs check-expiration
kubeadm certs renew all
systemctl restart kubelet   # and restart static-pod control-plane components to pick up new certs
kubeadm certs certificate-key   # if re-uploading for join operations
```

On self-managed/kubeadm clusters, control-plane certs (API server, controller-manager, scheduler, front-proxy, etcd) default to a **1-year** validity — genuinely easy to forget between clusters that are stood up once and rarely touched again. `kubeadm certs renew all` regenerates every cert signed by the local CA using the existing CA key (so it doesn't require the disruptive full CA rotation), but the static-pod components (API server, etc., run as kubelet-managed static pods rather than Deployments) need to actually restart to pick up the newly-written certificate files — simply renewing the files on disk without a restart leaves the running process using its old, in-memory certificate until restarted. Automating a periodic check (cron running `check-expiration`, alerting well before the 1-year mark) turns this from a recurring fire-drill into a non-event.

Q190 Expired API Server certificate.

The signature outage: every `kubectl` command and every internal component talking to the API server fails TLS verification simultaneously — this looks exactly like "the entire cluster is down" (Q1/Q3) but the root cause is purely a certificate date, not any actual service failure. `openssl s_client -connect <endpoint>:6443 2>&1 | grep -A2 "Verify return code"` confirms it fast — a return code referencing certificate expiry rather than a connection failure narrows it immediately.

```
openssl s_client -connect <api-endpoint>:6443 -servername <api-endpoint> </dev/null 2>&1 | grep -E "Verify
return|notAfter"
kubeadm certs renew apiserver
# then restart the static pod, e.g. by touching its manifest or restarting kubelet on that node
```

On kubeadm clusters: `kubeadm certs renew apiserver`, then force the static pod to restart (kubelet watches the manifest directory, so a manifest touch or kubelet restart on that control-plane node picks up the new cert). On EKS, the control plane's certs are entirely AWS-managed and this scenario doesn't apply to the control plane itself — if you see this symptom on EKS, it's almost certainly a client-side or component-level cert instead (e.g. a self-managed component's client cert to the API server), not the API server's own serving cert.

Q191 Expired kubelet certificate.

Narrower blast radius than an expired API server cert: the affected node itself stops being able to authenticate to the API server (client cert expired) — it goes NotReady since it can no longer send heartbeats — or, if it's the *serving* cert that expired, the node may still show Ready (heartbeats use the client cert) while `kubectl exec` / `logs` / `port-forward` to pods on that specific node fail, which is a confusing, easy-to-misdiagnose split symptom worth knowing about explicitly.

```
journalctl -u kubelet | grep -i "certificate has expired\|x509"
kubectl get csr
```

If `rotateCertificates` / `serverTLSBootstrap` were enabled, this shouldn't happen at all under normal operation — a genuine expiry usually means auto-rotation was never enabled, or the `CertificateSigningRequest` for the rotation was created but never approved (some clusters require manual or controller-based CSR approval rather than full auto-approval). Fix: manually approve any pending CSRs (`kubectl certificate approve <csr>`), or for a fully expired cert with no pending rotation, may require manually re-bootstrapping the node's kubelet credentials from scratch.

Q192 etcd certificate expired.

Arguably the most severe of all the expiry scenarios here, since etcd is the source of truth for all cluster state: if the API server's client cert for talking to etcd expires, the API server can no longer read or write *anything* — every operation fails, which again looks identical to "cluster is completely down" from the outside, but the fix is narrowly scoped to one cert relationship.

```
ETCDCTL_API=3 etcdctl --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  endpoint health
kubeadm certs renew etcd-server etcd-peer etcd-healthcheck-client apiserver-etcd-client
```

etcd has its own dedicated set of certs (server, peer, and the API server's client cert to it) often on a separate CA from the main cluster CA — renew all of them together via `kubeadm certs renew` with the specific etcd-related cert names, then restart etcd and the API server (both are typically static pods, so a manifest touch or kubelet restart on the affected node applies it). This scenario is the strongest possible argument for automated expiry alerting well ahead of the 1-year kubeadm default, given how severe and confusing the blast radius is if discovered reactively.

Q193 Mutual TLS.

Both sides of a connection present and verify a certificate, rather than the usual one-way TLS where only the server proves its identity to the client. In Kubernetes contexts, mTLS shows up in two very different places worth distinguishing: (1) **cluster-internal component auth** — kubelet↔API-server communication is already mTLS by design (each side presents a client cert the other verifies against the shared CA); (2) **service mesh mTLS** (Istio, Linkerd) — automatically encrypting and mutually authenticating *every* pod-to-pod connection within the mesh, transparently to the application, using short-lived certs the mesh's own CA issues and rotates continuously (often hourly), without the application code needing any TLS awareness at all.

```
# Istio: require strict mTLS for a namespace
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata: {name: default, namespace: my-app}
spec:
  mtls: {mode: STRICT}
```

The value proposition worth stating: mesh-level mTLS gives you zero-trust pod-to-pod security (every hop authenticated and encrypted) without touching application code, at the cost of the mesh's own operational complexity and a small latency/resource overhead per hop.

Q194 TLS handshake process.

Worth walking through TLS 1.3's streamlined handshake specifically (the current standard, and meaningfully different from 1.2, which interviewers sometimes probe as a follow-up): **ClientHello** (client proposes supported cipher suites, TLS version, and — new in 1.3 — a key-share guess sent optimistically in the same message) → **ServerHello** (server picks a cipher suite, sends its own key share, and immediately sends its certificate and a `Finished` message, all in the same round trip) → client verifies the certificate chain against its trusted CA store, computes the shared secret via the exchanged key shares (Diffie-Hellman based), sends its own `Finished` → encrypted application data can now flow.

The concrete improvement worth naming: TLS 1.3 does this in **one round trip** (versus two in TLS 1.2), directly reducing connection-setup latency, and removes support for several older, weaker cipher suites entirely rather than just deprioritizing them — a meaningful security hardening, not just a performance one.

Q195 TLS versions.

```
nginx.ingress.kubernetes.io/ssl-protocols: "TLSv1.3 TLSv1.2"
```

Production baseline to state confidently: **TLS 1.2 minimum, 1.3 preferred** — TLS 1.0 and 1.1 are formally deprecated (removed from major browsers and disallowed by PCI-DSS since mid-2018) and should never be enabled anywhere in a production posture. In Kubernetes, this is configured at whichever layer terminates TLS: the Ingress controller's annotations/ConfigMap (NGINX ingress example above), the ALB's SSL policy (`alb.ingress.kubernetes.io/ssl-policy`, referencing an AWS-predefined policy like `ELBSecurityPolicy-TLS13-1-2-2021-06`), or the service mesh's mTLS configuration if terminating there instead. Worth noting for compliance-heavy interviews: PCI-DSS and similar frameworks mandate a documented minimum TLS version and periodic verification that it's actually enforced (not just configured once and assumed to remain that way after infrastructure changes) — connect this to CIS Benchmark/audit-style verification rather than a one-time setting.

Q196 Cipher suites.

```
openssl s_client -connect example.com:443 -cipher 'ECDHE-RSA-AES128-GCM-SHA256' </dev/null 2>&1 | grep -i cipher
```

The negotiated combination of algorithms for key exchange, authentication, bulk encryption, and integrity — e.g. `ECDHE-RSA-AES128-GCM-SHA256` reads as: ECDHE for forward-secret key exchange, RSA for server authentication, AES-128-GCM for authenticated bulk encryption. The property worth naming explicitly in an interview: **forward secrecy** (via ECDHE/DHE key exchange) means even if a server's private key is later compromised, previously-recorded encrypted traffic still can't be decrypted retroactively, since each session's actual encryption key was never derived solely from the long-term private key. Modern guidance: prefer AEAD ciphers (GCM/ChaCha20-Poly1305) over older CBC-mode suites (vulnerable to padding-oracle-style attacks historically), and let TLS 1.3's built-in cipher restriction do most of this work for you rather than hand-curating a suite list, since 1.3 already dropped the legacy weak options from its negotiable set entirely.

Q197 Self-signed certificate.

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes \
  -subj "/CN=internal.svc.local" -addext "subjectAltName=DNS:internal.svc.local"
```

A certificate signed by its own private key rather than a recognized CA — perfectly fine and common for internal-only cluster traffic (a client explicitly configured to trust that specific cert or an internal CA), but it will always fail verification for any client relying on the standard public trust store (a browser, a default HTTP client), which is the classic "why does my curl fail but the internal service works fine" confusion. Legitimate uses: local development, purely internal service-to-service traffic where you control both ends and distribute trust explicitly, or as the CA cert itself for an internal PKI (see next question) — the CA's own root cert is inherently self-signed, by definition, since there's nothing above it to sign it. Never appropriate for anything a browser or external client will hit directly in production; use cert-manager with a real (internal or public) CA instead.

Q198 Internal CA.

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: {name: internal-ca}
spec:
  ca: {secretName: internal-ca-key-pair}
```

A private Certificate Authority you operate yourself (versus relying on a public CA like Let's Encrypt) — the standard choice for service-to-service TLS/mTLS entirely internal to your infrastructure, where a publicly-trusted certificate isn't necessary (no external client ever verifies it) and self-signed-per-service certs would be unmanageable at scale. cert-manager's `ca` issuer type (shown above) automates issuing/renewing leaf certs from a root you control, which is exactly the pattern a service mesh's own internal CA also follows for mTLS. Trade-off worth naming: you're now responsible for the CA's own security and lifecycle (protecting the root key, planning its eventual rotation, distributing trust to every internal client) — a real operational responsibility a public CA otherwise handles for you, in exchange for not needing internet-reachable domain validation for purely internal traffic.

Q199 External CA.

A publicly-trusted CA (Let's Encrypt, DigiCert, Amazon's ACM as a managed public CA) whose root is already present in every standard browser/OS trust store — required for anything an external user or client will connect to directly without any special trust configuration on their end (any public-facing website/API). Trade-offs versus an internal CA: no self-hosted CA infrastructure/key custody to manage, but less control over issuance policy, subject to the external CA's own validation requirements (domain control validation, at minimum) and rate limits (Let's Encrypt specifically has documented per-domain issuance rate limits worth knowing about if automating certs at scale).

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: {name: letsencrypt-prod}
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: platform-team@example.com
    privateKeySecretRef: {name: letsencrypt-prod-key}
    solvers:
      - http01: {ingress: {class: nginx}}
```

Common real pattern: external CA (Let's Encrypt via cert-manager) for public-facing Ingress endpoints, internal CA for everything service-to-service behind the edge — using each where it actually fits, rather than one CA type for everything.

Q200 cert-manager.

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata: {name: my-app-tls, namespace: my-app}
spec:
  secretName: my-app-tls-secret
  duration: 2160h    # 90d
  renewBefore: 360h  # renew 15d before expiry
  dnsNames: [app.example.com]
  issuerRef: {name: letsencrypt-prod, kind: ClusterIssuer}
```

The de-facto standard Kubernetes-native certificate lifecycle manager: it watches `Certificate` resources (or annotations directly on an `Ingress`, which it auto-converts into a Certificate behind the scenes), requests/validates/issues certs against a configured `Issuer` / `ClusterIssuer` (ACME, an internal CA, Vault, or a handful of others), stores the result as a standard `kubernetes.io/tls` Secret, and — the actual point of using it — automatically renews well before `renewBefore` triggers, with zero manual intervention as long as it stays healthy. This turns certificate expiry from a recurring operational fire-drill (the theme of nearly every question in this section) into a fully automated non-event, which is exactly why it's worth naming as the default answer whenever "how do you manage TLS certs in Kubernetes" comes up.

Q201 Let's Encrypt.

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: {name: letsencrypt-prod}
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: platform-team@example.com
    privateKeySecretRef: {name: letsencrypt-prod-key}
    solvers:
      - http01: {ingress: {class: nginx}}
```

A free, automated, publicly-trusted CA operated by the nonprofit ISRG — issues via the **ACME protocol** (Automated Certificate Management Environment), which is precisely what cert-manager automates end-to-end in Kubernetes. Certs are short-lived by design (90 days), which is a deliberate security choice forcing automation rather than a limitation to work around — if your renewal process can't handle a 90-day cycle reliably and unattended, that's the actual gap to fix, not a reason to seek a longer-lived cert elsewhere. Two practical specifics worth knowing: a staging endpoint (`acme-staging-v02...`) exists specifically for testing your issuer config without burning against production rate limits, and per-domain issuance rate limits are real and documented — worth checking before scripting bulk certificate requests across many subdomains at once.

Q202 ACM integration with EKS.

```
metadata:
  annotations:
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:us-east-1:123456789012:certificate/abc-123
```

AWS Certificate Manager as an alternative to cert-manager+Let's Encrypt for public-facing certs specifically on AWS load balancers — ACM issues and auto-renews free public certificates, but critically, **the private key never leaves AWS**; it's usable only when attached to an ALB/NLB/CloudFront listener, not exportable for use inside pods directly. This is the key trade-off to state versus cert-manager: ACM is zero-maintenance for anything terminating at an AWS-managed load balancer (no in-cluster renewal automation needed at all, AWS handles it), but doesn't help for TLS terminated *inside* the cluster (at an Ingress controller pod, or for mTLS between services) — cert-manager remains the tool for that use case. Most production EKS setups actually use both: ACM for the ALB's public-facing listener, cert-manager/internal-CA for anything terminated further inside the mesh.

Q203 TLS termination.

The point where encrypted traffic is decrypted back to plaintext — naming *where* this happens in your stack is one of the most common follow-up questions, since it directly determines what's encrypted versus plaintext at each hop. Common patterns, from simplest to most secure: (1) terminate at the **load balancer/ALB** only — traffic from ALB to pods is plain HTTP inside the VPC (simplest to operate, weakest posture, acceptable in many threat models since VPC traffic is already isolated); (2) terminate at the **Ingress controller** pod, re-encrypt or not to backend pods depending on requirements; (3) **end-to-end TLS** all the way to the pod, terminating only at the application itself (strongest, most operationally heavy, often achieved via a service mesh handling it transparently rather than the application implementing TLS itself).

```
alb.ingress.kubernetes.io/backend-protocol: HTTPS   # re-encrypt ALB-to-pod instead of terminating at ALB only
```

The right answer depends entirely on the compliance/threat model — PCI-DSS cardholder data environments, for instance, often mandate encryption for the entire path, not just to the edge.

Q204 SSL passthrough.

```
nginx.ingress.kubernetes.io/ssl-passthrough: "true"
```

The load balancer/Ingress forwards the raw encrypted TLS stream straight to the backend **without terminating it at all** — the backend pod itself holds the private key and does the actual TLS handshake and decryption. Distinguish clearly from termination-then-re-encryption (Q203's "end-to-end" option): with passthrough, the edge never touches the private key or plaintext at any point, whereas re-encryption terminates twice (decrypting and re-encrypting), meaning the edge briefly holds plaintext in memory between the two legs.

Real cost worth naming: passthrough means the edge (ALB/Ingress controller) can no longer route based on HTTP-layer content — no path-based routing, no header inspection, no SNI-based multiplexing of multiple backends behind one listener the way a terminating proxy can, since it's operating purely at the TCP/SNI layer. Used when the backend itself must control its own TLS entirely (a legacy app with hard-coded cert requirements, or a security policy mandating the edge never decrypt traffic it's merely routing).

Q205 Re-encryption.

```
alb.ingress.kubernetes.io/backend-protocol: HTTPS
```

TLS is terminated at the edge (ALB/Ingress, using the public-facing cert — ACM or Let's Encrypt), then a *new, separate* TLS session is established from the edge to the backend pod (typically using an internal CA's cert, since this leg never needs public trust). This is the middle ground between full termination-only (plaintext inside the VPC, Q203's simplest option) and full passthrough (edge never sees plaintext at all, Q204) — you get the edge's routing/inspection capabilities (path-based rules, WAF integration, header manipulation) *and* encryption all the way to the pod, at the cost of a brief window where the edge itself holds decrypted plaintext in memory between the two legs, plus the operational overhead of managing two separate certificate chains (public-facing and internal).

Worth naming as the generally-recommended default for anything handling sensitive data in a regulated environment, since it balances real security (nothing traverses the VPC in plaintext) against operational practicality (you keep edge-level routing features).

Q206 Certificate chain.

```
openssl s_client -connect example.com:443 -showcerts </dev/null 2>&1 | grep -c "BEGIN CERTIFICATE"
openssl verify -CAfile ca-bundle.crt leaf-cert.pem
```

The ordered sequence from the **leaf certificate** (the specific domain's cert) up through zero or more **intermediate CAs** to a **root CA** that's already trusted by the client — verification walks this chain, checking each cert's signature against the next one up, until it reaches a root already in the trust store. A very common real-world break: a server sends only the leaf cert, omitting the intermediate — browsers often work anyway because they cache/fetch intermediates from prior visits or AIA (Authority Information Access) extensions, but strict clients, mobile apps, and server-to-server calls without that caching fail outright with a chain-verification error, which is the classic "works in my browser but fails from curl/our backend service" symptom.

Fix: always serve the **full chain** (leaf + intermediates, concatenated, root omitted since the client already has it) — this is exactly what the `tls.crt` in a Kubernetes `kubernetes.io/tls` Secret should contain, not just the bare leaf.

Q207 SAN (Subject Alternative Name).

```
openssl x509 -in cert.pem -noout -ext subjectAltName
# X509v3 Subject Alternative Name:
#     DNS:app.example.com, DNS:www.example.com, IP Address:10.0.1.5
```

The extension listing every hostname/IP a certificate is actually valid for — modern TLS clients (all current browsers, and Go's `crypto/tls` since 1.15) validate **only** against the SAN list, ignoring the legacy Common Name field entirely for hostname matching, even though CN is often still populated for backward compatibility. A cert with a perfectly matching CN but a SAN list that doesn't include the hostname being connected to will fail verification — this exact mismatch is one of the most common "why is my valid-looking cert being rejected" incidents, especially with older or hand-rolled OpenSSL cert-generation scripts that only set CN without also setting SAN (a mistake OpenSSL's config made surprisingly easy for years, requiring an explicit `-addext` or config-file `[alt_names]` section that's easy to forget). Always verify SAN explicitly when debugging any hostname-mismatch TLS error, not just CN.

Q208 Common Name (CN).

```
openssl x509 -in cert.pem -noout -subject
# subject=CN = app.example.com, O = My Company
```

Historically the primary field hostname verification relied on, now effectively **deprecated for that purpose** — as of Chrome 58+ (2017) and Go 1.15+ (2020), hostname validation looks exclusively at the SAN extension (previous question) and ignores CN entirely for that check, even though CN is still commonly populated in generated certs for backward compatibility with very old clients. Interview signal worth stating precisely: a cert with CN set correctly but no matching SAN entry **will fail** in any modern client — this is a real, still-common misconfiguration from certificate-generation tooling or scripts that predate this shift and never got updated to also populate SAN. CN in client certificates serves a different purpose entirely (not hostname validation) — it typically maps directly to the Kubernetes username in client-cert-based authentication (Q185), which is a distinct use of the same field.

Q209 Wildcard certificate.

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata: {name: wildcard-example-com}
spec:
  secretName: wildcard-example-com-tls
  dnsNames: ["*.example.com"]
  issuerRef: {name: letsencrypt-prod, kind: ClusterIssuer}
```

A single certificate valid for **any single-level subdomain** (`*.example.com` matches `app.example.com` and `api.example.com`, but **not** `a.b.example.com` — wildcards don't span multiple levels, a detail worth stating precisely) — useful when many subdomains need TLS and issuing/rotating one cert per subdomain individually would be unwieldy at scale. Important operational constraint worth naming: wildcard certs from public CAs (including Let's Encrypt) require **DNS-01 challenge validation** specifically — HTTP-01 can't prove control over an entire domain's subdomain space, only one specific hostname, so this forces a DNS-integrated ACME solver (Route53, Cloudflare, etc.) rather than the simpler HTTP-01 flow usable for single-hostname certs. Security trade-off worth mentioning: a compromised wildcard private key exposes every subdomain simultaneously, a larger blast radius than per-subdomain certs.

Q210 Ingress TLS.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  tls:
    - hosts: [app.example.com]
      secretName: app-example-com-tls
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend: {service: {name: my-app, port: {number: 80}}}
```

The standard, native way to attach TLS to an Ingress: the `spec.tls` block references a Secret (of type `kubernetes.io/tls`) containing the cert+key, which the Ingress controller (NGINX, ALB, etc.) picks up for terminating TLS on that host. The `cert-manager.io/cluster-issuer` annotation is the piece that closes the loop end-to-end: cert-manager watches for this annotation, auto-creates the underlying `Certificate` resource, requests/validates/issues the cert from the referenced Issuer, and populates the referenced Secret — meaning you never manually create or rotate the Secret yourself once this is wired up. This annotation-driven pattern (versus manually authoring a separate `Certificate` object) is the more common way teams actually use cert-manager day-to-day, since it keeps the TLS configuration co-located with the Ingress itself.

Q211 Secret type `kubernetes.io/tls`.

```
kubectl create secret tls app-example-com-tls \
  --cert=fullchain.pem --key=privkey.pem -n my-app
kubectl get secret app-example-com-tls -o jsonpath='{.data.tls\.crt}' | base64 -d | openssl x509 -noout -dates
```

A dedicated Secret type (distinct from generic `Opaque`) that Kubernetes and tooling understand specifically as a certificate/key pair — it enforces exactly two required data keys, `tls.crt` (the certificate, ideally the **full chain** per Q206, not just the bare leaf) and `tls.key` (the private key), both base64-encoded like any Secret data field. Ingress controllers, cert-manager, and most TLS-consuming tooling specifically expect this type and these exact key names — using a generic `Opaque` Secret with differently-named keys for the same data will silently fail to work with anything expecting the standard type, since tooling checks the type field and key names, not just that cert-like data exists somewhere in the Secret. Same base-level protection as any other Secret applies (base64 is encoding, not encryption — etcd encryption-at-rest and tight RBAC on `secrets` read access both still matter, per Q166).

Certificate Management

Q212 How certificates are generated.

```
openssl genrsa -out key.pem 2048
openssl req -new -key key.pem -out csr.pem -subj "/CN=app.example.com" \
  -addext "subjectAltName=DNS:app.example.com"
# CSR submitted to a CA, which returns a signed certificate
```

The fundamental sequence, worth being able to state precisely regardless of which tooling automates it: (1) generate a **private key** (kept secret, never transmitted); (2) create a **CSR** (Certificate Signing Request) — contains the public key plus identity info (CN, SAN, org), signed with the private key to prove possession without revealing it; (3) submit the CSR to a CA, which validates the request (domain control for public CAs, internal policy for a private CA) and returns a **signed certificate** binding that public key to that identity. cert-manager automates this entire sequence — generating the key, building the CSR, submitting via ACME or an internal CA integration, and storing the result — but understanding the raw OpenSSL-level steps underneath is what separates "I run a command" from actually understanding what's happening.

Q213 How certificates expire.

```
openssl x509 -in cert.pem -noout -dates
# notBefore=Jun  1 00:00:00 2026 GMT
# notAfter=Aug 30 00:00:00 2026 GMT
```

Every certificate carries a validity window (`notBefore` / `notAfter`) baked into it at issuance time — there's no separate "expiry service" checking a database; any TLS client simply compares the current time against these two fields as part of standard chain validation, and rejects the handshake outright the instant `notAfter` passes, with zero grace period. This is exactly why every "expired certificate" scenario throughout this section is a hard, instant failure rather than a gradual degradation.

Modern industry direction is toward deliberately *shorter* validity windows, not longer ones — public CA/browser policy has progressively capped maximum cert lifetime down from years to 398 days currently, with further shortening actively discussed industry-wide, and Let's Encrypt's 90-day default was itself an intentional choice to force automation. The reasoning: a shorter-lived cert limits how long a compromised key remains dangerous if a revocation is ever missed or delayed, at the cost of requiring truly reliable automated renewal — which is precisely the problem cert-manager exists to solve.

Q214 How to rotate certificates.

The concrete mechanical steps underlying rotation, worth being able to describe precisely beyond just "cert-manager handles it": a **new** key+cert pair is generated (not the same private key re-signed — a proper rotation generates a fresh key too, limiting exposure if the old key was ever silently compromised); the new pair is written into the same Kubernetes Secret (cert-manager updates `tls.crt` / `tls.key` in place); and — the step people most often get wrong — anything holding the old cert in memory needs to actually **reload** it, since updating a Secret doesn't retroactively push new bytes into an already-running process's memory.

```
# NGINX Ingress watches mounted Secret files and reloads automatically
# Custom apps loading a cert once at startup need an explicit reload mechanism, e.g.:
kubectl rollout restart deployment/my-app -n my-app
```

Most Ingress controllers watch their mounted TLS Secrets and reload automatically on change — but a custom application reading a cert file once at startup and caching it in memory will keep serving the *old* (soon-to-expire) cert indefinitely until restarted, even though the Secret itself was updated correctly. Always verify the actual reload behavior of whatever's terminating TLS, not just that cert-manager successfully renewed the underlying Secret.

Q215 How cert-manager works internally.

```
kubectl get certificate,certificaterequest,order,challenge -n my-app
```

Worth describing the actual internal resource chain, since it's a good demonstration of understanding the controller pattern, not just "it's magic that renews certs": a `Certificate` resource (the thing you author, declaring desired end-state) creates a `CertificateRequest` (a specific CSR, generated fresh, tied to one attempt) — for ACME issuers specifically, that in turn creates an `Order` (the ACME protocol's tracked issuance request) which creates one `Challenge` per domain needing validation (the actual proof-of-control step, HTTP-01 or DNS-01). Each layer is its own controller reconciling its own resource type, watching the layer below it, which is exactly the same Kubernetes-native "small controllers composing into a larger workflow" pattern used throughout the ecosystem.

Practically useful for debugging: when a `Certificate` is stuck `Ready: False`, walk *down* this chain (`kubectl describe` each level) to find exactly which layer is stuck and why, rather than guessing from the top-level Certificate's often-terse status alone.

Q216 ACME challenge.

The proof-of-control step every ACME-based issuance requires before a CA will sign a certificate — the CA needs cryptographic evidence you actually control the domain being requested, not just that you asked for it. The client (cert-manager) generates a token, publishes it somewhere only the true domain owner could plausibly place it (an HTTP path, or a DNS TXT record — the two challenge types, each covered next), and the CA's servers independently fetch/query that location from the outside to verify the token matches what it issued. Only after this validation succeeds does the CA proceed to actually sign and return the certificate.

```
kubectl describe challenge -n my-app
# Reason: Waiting for HTTP-01 challenge propagation: ... 404
```

A stuck or failing `Challenge` resource is the most common point of failure in any ACME issuance — almost always because the validation path/record isn't actually reachable from the public internet (a firewall, a NAT issue, or DNS propagation delay), which the Challenge's own status/events describe directly.

Q217 DNS challenge (DNS-01).

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
spec:
  acme:
    solvers:
      - dns01:
          route53: {region: us-east-1, hostedZoneID: Z1234567890}
```

Proves domain control by requiring a specific **TXT record** (`_acme-challenge.app.example.com`) to exist with a value the CA specifies — cert-manager creates this record automatically via a DNS provider integration (Route53, Cloudflare, etc., requiring credentials with DNS-edit permission for that zone), the CA queries it, and cert-manager removes it once validated. The two properties that make this the required choice for wildcard certificates (Q209) and the preferred choice for anything not yet publicly reachable: it doesn't require the domain to be internet-reachable over HTTP at all (works for purely internal services, or infrastructure not yet fully provisioned), and it can prove control over an *entire* subdomain space at once (needed for wildcards) rather than one specific hostname. Trade-off: DNS propagation delay means validation isn't instant, and it requires granting cert-manager real DNS-write credentials, a meaningful permission to scope carefully (limit to exactly the zone(s) needed, never broader).

Q218 HTTP challenge (HTTP-01).

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
spec:
  acme:
    solvers:
      - http01: {ingress: {class: nginx}}
# cert-manager temporarily serves:
# http://app.example.com/.well-known/acme-challenge/<token>
```

Proves control by requiring a specific token be servable at a well-known HTTP path on the domain itself — cert-manager automatically creates a temporary Ingress rule (or reuses the existing one) routing that specific path to a small solver pod it spins up just for the challenge, the CA fetches it over plain HTTP (port 80, even for what will become an HTTPS cert — the validation itself doesn't need TLS), and the temporary routing is torn down once validated.

Simpler to set up than DNS-01 (no DNS provider credentials needed) and validates near-instantly (no DNS propagation wait), but has two real constraints worth naming: the domain must already be genuinely reachable over HTTP from the public internet at the time of validation (doesn't work for internal-only services, or anything behind auth), and it can only validate **one specific hostname per challenge** — it fundamentally cannot prove control over a wildcard, which is exactly why DNS-01 is required for that case instead.

Q219 ClusterIssuer vs Issuer.

```
kubectl get issuer -A          # namespaced — visible/usable only within its own namespace
kubectl get clusterissuer      # cluster-scoped — usable by Certificates in any namespace
```

Purely a **scope** distinction, same shape as Role vs ClusterRole: an `Issuer` is namespaced — only `Certificate` resources in that same namespace can reference it — while a `ClusterIssuer` is cluster-scoped and referenceable from any namespace. In practice, most organizations standardize on a small number of `ClusterIssuer`s (one for Let's Encrypt staging, one for production, perhaps one for an internal CA) shared across every team's namespaces, rather than every team configuring and maintaining its own duplicate `Issuer` — simpler to operate and audit centrally, and avoids each team needing its own copy of DNS-provider or ACME-account credentials. A namespaced `Issuer` becomes the right choice specifically when different namespaces genuinely need different CA configurations or credentials that shouldn't be shared cluster-wide (e.g. multi-tenant clusters where tenants must not share ACME account state).

Q220 Renewal process.

```
spec:
  duration: 2160h     # 90d total validity
  renewBefore: 360h   # begin renewal 15d before notAfter
```

cert-manager runs a continuous reconciliation loop (not a cron job triggered externally) checking every `Certificate`'s actual expiry against its `renewBefore` threshold — once within that window, it transparently repeats the entire issuance flow (new CertificateRequest → Order → Challenge, same chain as initial issuance) to obtain a fresh cert *before* the old one expires, then atomically updates the same Secret in place. Because the new cert is obtained fully before the old one's expiry and the Secret update is atomic, there's no gap where the Secret is empty or invalid — the main remaining risk is purely on the consumer side (Q214's point: does whatever's reading that Secret actually reload the new content promptly).

`renewBefore` should be set with enough margin to survive a transient failure and retry — 15 days ahead of a 90-day cert (as shown) gives ACME rate-limit issues, a brief DNS provider outage, or a solver-pod scheduling delay real room to resolve and retry automatically before the actual deadline matters.

Q221 Certificate monitoring.

```
# cert-manager exposes Prometheus metrics directly:
certmanager_certificate_expiration_timestamp_seconds
certmanager_certificate_ready_status
```

The closing principle of this entire section, worth stating explicitly as a summary: every expired-certificate incident described throughout this handbook (API server, kubelet, etcd, Ingress) is fundamentally a **monitoring gap**, not an unavoidable event — certificates expire on a known, queryable schedule; there's no excuse for discovering it via an outage rather than an alert. cert-manager's own Prometheus metrics (shown above) let you alert directly on `certmanager_certificate_expiration_timestamp_seconds` approaching now, and on `Ready: False` persisting (a renewal that's stuck failing, silently, well before it becomes an actual expiry). For certs cert-manager doesn't manage (control-plane certs on kubeadm clusters, third-party services), external blackbox probes (checking a live endpoint's cert expiry directly via TLS handshake, e.g. via a Prometheus blackbox exporter or a dedicated cert-expiry-checking tool) close the remaining gap. A mature platform team treats "30 days before any certificate anywhere expires" as a standing, automatically-alerting SLO, not a manual calendar reminder.

PART VI

Trick Questions

28 questions · conceptual gotchas that separate real understanding from memorized commands

Q222 Pod is Running but the app is unavailable — why?

The trap: `Running` is a **container-runtime-level** status — it means the process inside the container hasn't exited, nothing more. It says nothing about whether the application inside has finished initializing, is deadlocked, is stuck waiting on a dependency, or is silently serving errors. This is exactly why readiness probes exist as a *separate* signal from the phase itself — a pod can be `Running` with `Ready: False` indefinitely, correctly excluded from Service endpoints the whole time.

```
kubectl get pod <pod> -o jsonpath='{.status.conditions[?(@.type=="Ready")].status}'
kubectl exec -it <pod> -- curl -v localhost:<port>/healthz
```

If `Ready` is `True` and it's still unavailable, the failure is likely downstream of the pod entirely — check the Service's endpoints actually list this pod, then NetworkPolicy, then the Ingress/LB layer, working outward from the pod rather than assuming the pod itself is the problem just because it's "Running."

Q223 Readiness vs Liveness — what's the actual difference?

The trap: people describe both as "health checks" and treat them interchangeably, missing that they trigger **completely different actions** on failure. Liveness failing → kubelet **kills and restarts** the container. Readiness failing → the pod is **removed from Service endpoints** — the container keeps running untouched, it just stops receiving traffic.

The consequence of conflating them (a real, common misconfiguration): using the *same* check for both, especially one that tests a downstream dependency (database connectivity), means a temporary DB blip **restarts every pod** in the deployment simultaneously via liveness failures, rather than just quietly pulling them from rotation via readiness — potentially turning a brief database hiccup into a full application outage through unnecessary, synchronized restarts. Correct pattern: liveness checks only "is this process fundamentally stuck/deadlocked" (cheap, self-contained); readiness checks "can I currently serve traffic well" (can include dependency checks, since removing from rotation is a safe, low-cost response to a downstream issue).

Q224 Why not use the `:latest` tag?

The trap: it seems convenient ("always run the newest version"), but it breaks the fundamental guarantee Kubernetes rollouts depend on — that a given *tag* maps to a fixed, known set of bytes. With `:latest`, the same tag can point to entirely different images at different times, which breaks in ways that aren't obvious until they bite: `imagePullPolicy: IfNotPresent` means a node that already cached an older "`:latest`" won't pull the new one at all, so different nodes silently run different actual code under the identical tag; a rollback (`kubectl rollout undo`) becomes meaningless, since the previous ReplicaSet's manifest still just says `:latest` too — there's no way to specify "the version before this one" when the tag never changed; and reproducing "what was actually running last Tuesday" for an incident postmortem becomes impossible without external records.

Correct practice: immutable tags — the git commit SHA, or a semantic version bound to a specific digest — so a tag always, permanently, means one exact set of bytes.

Q225 Deployment vs StatefulSet — when does it actually matter?

The trap: people say "StatefulSet is for stateful apps" and leave it there, missing the actual mechanical guarantees that matter. A Deployment's pods are fungible — any replica can be replaced by any other, in any order, with a randomly-suffixed name, and none has a fixed identity or storage tied specifically to it. A StatefulSet gives each replica a **stable, predictable identity** (`app-0`, `app-1`, ...), a stable DNS name via a headless Service, and (if using `volumeClaimTemplates`) its *own* PersistentVolumeClaim that follows that specific ordinal wherever it's rescheduled — `app-1` always gets back "its" volume, not a random one.

The trick-question core: many "stateful" apps (a stateless API reading from an external managed database) need *none* of this and should just be a Deployment — the deciding question isn't "does my app have state" but "does each specific replica need its own durable identity and storage that must survive and reattach across rescheduling" (a database node, a Kafka broker) — if not, a Deployment with PVCs shared via a normal (non-per-replica) claim is simpler and correct.

Q226 DaemonSet vs Deployment — why can't a Deployment just do this?

The trap: "just set `replicas` to the number of nodes" doesn't work, and understanding *why* is the actual test. A Deployment's replica count is a fixed number with no awareness of the node topology at all — if a node is added or removed, the Deployment doesn't automatically follow; the scheduler just places (or doesn't place) pods according to normal scheduling constraints, with no guarantee of "exactly one per node."

A DaemonSet's entire purpose is different: it guarantees exactly one copy runs on **every node matching its selector**, automatically, growing and shrinking as nodes join and leave the cluster — no replica count to manage at all. This is the correct (and really, only correct) primitive for genuinely per-node infrastructure: the CNI plugin (`aws-node`), `kube-proxy`, log-shipping agents (Fluent Bit), and node-level monitoring agents all need exactly this "one instance per node, always" guarantee, which a Deployment structurally cannot provide even at the "right" replica count for today's node total.

Q227 ClusterIP vs NodePort — what's the real trade-off?

The trap: both "expose a Service," so people treat picking one as arbitrary. `ClusterIP` (the default) is reachable **only from inside the cluster** — correct for anything that's purely an internal dependency (a backend API another service calls, a database). `NodePort` opens the exact same port on **every node's own IP** in the cluster, cluster-wide, whether or not that node is actually running the target pod — kube-proxy forwards the request to a node that does have a matching pod if the one you hit doesn't.

The trick-question sting: NodePort is almost **never the right way to actually expose something externally** in a real production setup, despite technically making a service internet-reachable if the node has a public IP — it bypasses all the things a real Ingress/LoadBalancer gives you (TLS termination, path-based routing, a single stable external IP instead of every node's IP, health-check-aware routing). NodePort exists mainly as the underlying mechanism *Load Balancer*-type Services and Ingress controllers build on top of internally, not as a customer-facing exposure method on its own.

Q228 When would you actually use HostNetwork?

```
spec:
  hostNetwork: true
```

The trap: it sounds like a simple performance optimization ("skip the overlay network overhead"), and interviewers are testing whether you know the real cost. With `hostNetwork: true`, the pod shares the **node's own network namespace** entirely — no pod IP of its own, and any port it binds is bound directly on the node's IP, meaning **only one pod on that node** can ever bind that port (a second instance scheduled to the same node conflicts outright, unlike normal pods which each get their own IP and can all use the same container port simultaneously).

Legitimate uses are narrow and specific: CNI plugins and `kube-proxy` themselves genuinely need host-level network access to do their job (they *are* the networking layer, so they can't rely on the pod network they're implementing); extremely latency-sensitive workloads sometimes accept the trade-off deliberately. For anything else — and this is the trick — it's a red flag in a design review, not a reasonable default optimization, given the port-conflict constraint and the reduced network isolation from the host itself.

Q229 Can two Pods share the same IP address?

The trap answer is a flat "no" — the correct answer is "not normally, but yes under specific, nameable circumstances," and naming those circumstances is the actual test. Under the standard Kubernetes networking model, every pod gets its own unique IP, full stop. But: pods using `hostNetwork: true` (previous question) all share the **node's** IP, meaning multiple such pods on the same node do technically have the same IP (which is exactly the source of the port-conflict issue above); and containers *within* the same pod always share one IP by definition (that's the actual definition of what makes them "in the same pod" — a shared network namespace) — which is worth mentioning since it reframes the question as slightly ambiguous ("two Pods" vs "two containers") in a way worth clarifying with the interviewer.

The deeper point interviewers are fishing for: understanding that the "one IP per pod" rule is a Kubernetes networking *model* convention (which the CNI plugin is responsible for implementing), not a law of physics — and knowing exactly where that model's exceptions are.

Q230 Why doesn't deleting a Pod delete the Deployment?

```
kubectl delete pod <pod>   # a new one appears almost immediately — why?
```

The trap is really the inverse framing — the interesting question isn't "why doesn't deleting the pod delete the Deployment" (a Pod is never the owner of anything above it) but "why does a new pod immediately reappear," which reveals whether you understand the **ownership/reconciliation hierarchy**: Deployment → ReplicaSet → Pod, each layer expressed via `ownerReferences` pointing *upward*, never downward. Deleting a Pod only removes that one object; the ReplicaSet controller's reconciliation loop immediately notices "observed replicas (now one fewer) ≠ desired replicas" and creates a replacement to close the gap — it has no concept of "someone deleted this on purpose, leave it be."

The only way to actually reduce running pod count is to change the *desired state* the higher-level controller is reconciling toward — scaling the Deployment down, or deleting the Deployment/ReplicaSet itself (which cascades downward via `ownerReferences` and garbage collection, deleting every Pod it owns). This ownership-and-reconciliation model is the single most foundational Kubernetes concept, and this question is really testing whether you can articulate it precisely rather than just knowing "a new pod comes back."

Q231 Why do ReplicaSets even exist if Deployments manage replicas?

```
kubectl get rs -l app=my-app
# my-app-7d4b9c8f5   0   0   0   # old ReplicaSet, scaled to zero, kept for history
# my-app-8f6a2d1c3    3   3   3   # current ReplicaSet
```

The trap: it looks like a redundant layer, since you almost never interact with a ReplicaSet directly. The actual reason: **a Deployment's real job is managing rollouts across multiple ReplicaSet versions**, not just running pods — every time you change the pod template (a new image, an env var), the Deployment creates a *brand new* ReplicaSet for that version and shifts replica counts from the old ReplicaSet to the new one according to the rollout strategy (maxSurge/maxUnavailable). The old ReplicaSet is scaled to zero but kept around (up to `revisionHistoryLimit`) specifically so `kubectl rollout undo` has something concrete to scale back up to — rollback is really "reverse the traffic shift between two existing ReplicaSets," not "recreate pods from scratch."

So the layering answer is: ReplicaSet's actual job is "maintain N identical pods of one exact spec"; Deployment's job is "orchestrate a transition between two ReplicaSets, each pinned to a different spec" — genuinely different responsibilities, not a redundant wrapper.

Q232 kube-proxy iptables mode vs IPVS mode.

```
kubectl -n kube-system get cm kube-proxy -o yaml | grep mode
```

The trap: assuming they're just two interchangeable implementations with no real trade-off. **iptables mode** (the long-time default) implements Service routing as a chain of sequentially-evaluated firewall rules — conceptually simple, but rule evaluation is **O(n)** in the number of Services, meaning lookup time genuinely degrades as a cluster grows into thousands of Services, since the kernel walks the chain rule-by-rule. **IPVS mode** uses the kernel's IP Virtual Server subsystem, backed by proper hash-table lookups — effectively O(1) regardless of Service count, plus a wider choice of load-balancing algorithms (round-robin, least-connection, etc., versus iptables' effectively-random rule matching).

The practical guidance worth stating: iptables mode is genuinely fine for small-to-medium clusters and remains extremely common in the wild; IPVS becomes the clearly better choice specifically at large scale (thousands of Services), where the O(n) vs O(1) difference stops being theoretical and starts showing up as measurable added latency per connection.

Q233 Why doesn't updating a ConfigMap automatically update my running app?

The trap: it depends entirely on *how* the ConfigMap is consumed, and this is exactly the nuance interviewers probe for. If mounted as a **volume**, the kubelet *does* eventually sync the updated file content into the container's filesystem (with a sync delay, historically up to ~1 minute, tunable) — but if consumed as **environment variables** (`envFrom` / `valueFrom.configMapKeyRef`), env vars are resolved **once, at container start**, full stop — there is no mechanism to push a changed value into an already-running process's environment, since that's simply not how process environments work at the OS level.

```
kubectl rollout restart deployment/my-app -n my-app
```

Second half of the trap, even for volume-mounted ConfigMaps: even when the *file on disk* updates, the application itself typically doesn't re-read that file unless it was specifically written to watch for changes (inotify-based reload logic) — most applications read config once at startup and cache it in memory regardless of what's happening on disk afterward. The reliable, universally-correct answer for either case: trigger a rollout restart, forcing new pods that read the current ConfigMap value at their own fresh startup.

Q234 Why doesn't my HPA scale even though CPU usage looks high?

```
kubectl describe hpa <name>
kubectl top pod
kubectl get pods -l app=my-app -o jsonpath='{.items[*].spec.containers[*].resources.requests.cpu}'
```

The trap almost everyone falls into: HPA's default CPU metric is a **percentage of the requested CPU**, not absolute usage — if no `resources.requests.cpu` is set on the container at all, the HPA has no denominator to calculate a percentage against and **cannot scale on CPU at all**, often failing silently/unhelpfully rather than with an obvious error (check `kubectl describe hpa`'s conditions for `FailedGetResourceMetric` specifically). Second common trap: the metrics-server itself isn't installed/healthy (HPA depends on it entirely for the metrics API), which produces the identical symptom of "HPA just isn't reacting" with no scaling activity.

Beyond those two, also check: `maxReplicas` already reached (silently capped, not actually stuck — easy to overlook when staring at a flat replica count); and the metric's actual *average across all pods* — a few very hot pods averaged against many idle ones can stay under the scaling threshold overall even though any single pod looks maxed out, since HPA scales on the aggregate, not the hottest individual pod.

Q235 What happens if etcd data is lost?

The trap-within-a-trap: people answer at the wrong layer. The correct framing to lead with: **etcd is the entire cluster's state**, full stop — not "an important database," but literally the only place the desired-state and current-state of every object in the cluster is recorded. Losing it isn't "an outage," it's losing the cluster's memory of what it's even supposed to be running.

The nuance worth stating precisely: the *running workloads themselves keep running* in the immediate aftermath (containers already scheduled on nodes don't just vanish) — but the control plane can no longer do **anything**: no new scheduling, no reconciliation if a pod crashes (nothing restarts it, since the controller that would react has nothing to read), no `kubectl` command of any kind succeeds. Effectively, the cluster becomes a frozen photograph of whatever was already running, silently unable to heal itself from that point forward, degrading node by node as anything inevitably crashes with no controller left to notice or replace it.

```
etcdctl snapshot save /backup/etcd-snapshot.db
etcdctl snapshot restore /backup/etcd-snapshot.db --data-dir /var/lib/etcd-restored
```

This is precisely why etcd backups (regular, tested `snapshot save`, ideally shipped off-node/off-cluster) are one of the single highest-priority disaster-recovery items for any self-managed cluster — on EKS, AWS manages etcd's durability entirely, which is itself worth naming as a real, concrete benefit of managed control planes when this question comes up.

Q236 Why does a Pod only ever schedule on one node, even with multiple replicas requested?

The trap is usually in the phrasing itself — a single *Pod* object is, by definition, one instance placed on exactly one node; it's the **replica count** (via a Deployment/ReplicaSet) that creates multiple distinct Pod objects, each independently scheduled, potentially to different nodes. If someone observes "all my replicas landed on the same node" unexpectedly, that's a genuinely different, real question worth answering too: no `podAntiAffinity` configured to spread them, or a taint/label constraint that happens to only match one node, or simply too few nodes/insufficient resources on the others for the scheduler to place replicas elsewhere even if it wanted to.

```
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector: {matchLabels: {app: my-app}}
          topologyKey: kubernetes.io/hostname
```

The fix, and the real point of this question: explicit `podAntiAffinity` (or the simpler `topologySpreadConstraints`, generally the more modern, more flexible recommendation) to actively instruct the scheduler to spread replicas across nodes/AZs for resilience — without it, the scheduler optimizes purely for resource fit, with no inherent notion of "spread these apart for availability" unless explicitly told to.

Q237 Why does `kubectl` still work if kube-proxy is down?

The trap: assuming `kube-proxy` is somehow "the network," involved in every cluster interaction. It isn't — `kubectl` talks directly to the **API server** over its own HTTPS endpoint, a completely separate code path that has nothing to do with `kube-proxy` at all. `kube-proxy`'s actual job is narrower and specific: programming the iptables/IPVS rules that implement **Service**-level virtual IP routing — i.e., "traffic to this ClusterIP gets forwarded to one of these actual pod IPs." `kubectl`, the scheduler, the controller-manager, and direct pod-to-pod traffic (via the CNI, not kube-proxy) all function completely independently of it.

What actually breaks when `kube-proxy` is down: **only Service-mediated traffic** — a client hitting a ClusterIP/NodePort gets nothing, since the routing rules kube-proxy maintains are gone or stale, while direct pod-IP-to-pod-IP traffic (bypassing any Service) keeps working fine, since that path never went through kube-proxy in the first place. This distinction — that kube-proxy is a Service-routing component, not a general networking dependency — is exactly what separates a real understanding of cluster networking from a surface-level one.

Q238 What happens if CoreDNS crashes?

The trap: assuming it's an instant, total outage identical to losing the API server. The more precise answer: **already-established connections keep working** (they don't re-resolve DNS mid-connection), and any application that **cached** a prior successful DNS lookup keeps functioning against that cached IP for as long as its TTL/cache allows. What breaks immediately is anything attempting a *new* DNS lookup — a pod starting up for the first time and trying to resolve another Service by name, or a client whose DNS cache just expired and needs a fresh lookup.

```
kubectl -n kube-system get pods -l k8s-app=kube-dns
kubectl -n kube-system logs -l k8s-app=kube-dns --tail=50
kubectl run dns-test --image=busybox:1.28 --rm -it -- nslookup kubernetes.default
```

The failure mode compounds over time rather than hitting all at once: as more caches naturally expire and more pods restart/scale (each needing fresh lookups), an increasing fraction of the cluster loses the ability to resolve internal Service names, looking like a slow-building, confusing partial outage rather than a clean, instant, obviously-DNS-shaped failure — which is exactly why CoreDNS needs its own dedicated `PodDisruptionBudget` and multiple replicas spread across nodes, treated with the same seriousness as any other genuinely single-point-of-failure cluster component.

Q239 What happens if the Scheduler stops running?

The trap: assuming everything stalls immediately. In fact, the blast radius is narrower and more specific than people expect: **every pod already scheduled and running is completely unaffected** — the scheduler's only job is deciding *which node a new, currently-unscheduled pod should go to*; it has no ongoing runtime role once that decision is made and recorded. What actually breaks: any *new* pod (from a fresh Deployment, a scale-up, a rescheduled pod after a node failure) gets created but sits permanently in `Pending`, since nothing is left to assign it a node — it exists as an API object with an empty `.spec.nodeName` indefinitely.

```
kubectl get pods --field-selector=status.phase=Pending
kubectl -n kube-system get pods -l component=kube-scheduler
```

This is the same "existing state frozen, no new reconciliation" pattern as the etcd-loss and API-server-down scenarios, just scoped narrowly to scheduling decisions specifically rather than the whole control plane — a good answer draws that parallel explicitly, showing you understand it as one instance of a general pattern (each control-plane component owns a specific slice of reconciliation, and losing one component degrades only that slice) rather than memorizing this as an isolated fact.

Q240 Can Pods communicate with each other without kube-proxy?

The trap: yes, and understanding *why* reveals whether you actually understand the two separate networking layers Kubernetes composes. **Direct pod-to-pod IP communication** is handled entirely by the **CNI plugin** (VPC CNI, Calico, Cilium, etc.) — every pod gets a real, routable IP the moment it's created, and any pod can reach any other pod's IP directly across the flat pod network the CNI establishes, with zero involvement from kube-proxy at any point in that path.

`kube-proxy`'s entire job is layered *on top* of that base pod-network reachability — it only handles the extra step of "Service ClusterIP → pick one of the matching pod IPs," a convenience/abstraction layer (stable virtual IP instead of tracking individual pod IPs that come and go) rather than the foundational connectivity mechanism itself. So a client using a Service name/ClusterIP genuinely needs kube-proxy's routing rules to work; a client that already knows a specific pod's IP directly (unusual in normal application code, but true of things like StatefulSet peer discovery via a headless Service's DNS, which returns pod IPs directly) doesn't need kube-proxy at all for that specific connection.

Q241 Why does CrashLoopBackOff keep increasing even though I already fixed the issue?

```
kubectl delete pod <pod>   # forces a fresh pod, resetting the backoff timer
kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].restartCount}'
```

The trap: assuming Kubernetes somehow "knows" you fixed the underlying problem and should immediately retry at full speed. It doesn't — the backoff delay (10s, 20s, 40s... capped at 5 minutes) is tracked **per-pod, based purely on restart count**, completely independent of whether the actual root cause is still present. If you fixed the issue by, say, editing a ConfigMap or fixing an external dependency *without* the pod itself being replaced, the **same pod object** retains its accumulated restart count and continues waiting out whatever backoff interval it had already escalated to — it doesn't reset just because the next attempt would now succeed.

This is exactly why "I fixed it but it's still not coming back for another 3 minutes" is a common, confusing observation rather than a sign anything is actually still broken — either wait out the existing backoff, or (the standard practical fix) delete the pod outright to force a fresh one with its restart count reset to zero, immediately testing whether the fix actually worked rather than waiting on an arbitrary escalating timer.

Q242 Why is a Pod stuck Pending when the cluster clearly has free resources?

The trap: "free resources" as observed by `kubectl top nodes` or a dashboard is **actual usage**, but the scheduler decides placement based on **requested** resources (the sum of every pod's `resources.requests` already scheduled to a node), which can be far higher than what's actually being used at any given moment — a node can look 30% utilized in real terms while its requests already sum to 100% of allocatable capacity, making it genuinely full from the scheduler's point of view even though there's plenty of real headroom being wasted.

```
kubectl describe pod <pod>   # check the Events section specifically
kubectl describe nodes | grep -A5 "Allocated resources"
```

Beyond the requests-vs-usage gap, the Pod's own `Events` almost always states the exact reason directly (don't guess when it's right there): `Insufficient cpu` / `memory` (the requests-based mismatch above), a `nodeSelector`/affinity rule that no node actually satisfies, a taint with no matching toleration, or (on EKS specifically) the Cluster Autoscaler being slow/stuck to add a new node when everything is genuinely full — each produces the same surface symptom (stuck Pending) with a completely different fix, which is exactly why reading the actual Events output first beats reasoning from assumptions.

Q243 Why does a NodePort Service open its port on every node, even nodes with no matching Pod?

The trap: it looks like a bug or waste at first glance ("why would it open a port on a node running none of these pods?") — but it's the entire point of the design, and explaining *why* is the actual test. A client connecting to a NodePort has no way to know in advance which specific node(s) happen to be running a matching pod right now (that set changes constantly with scaling, rescheduling, node replacement) — so the guarantee has to be "you can hit **any** node's IP on this port and reach the Service," with kube-proxy transparently forwarding the connection to a node that actually has a matching pod if the one you hit doesn't.

```
iptables-save | grep <nodeport>
# shows the DNAT/forwarding rule on every node, not just ones running the pod
```

Without this "open everywhere, forward as needed" behavior, external clients (or a load balancer sitting in front of the NodePort, which is exactly how cloud LoadBalancer-type Services are actually implemented under the hood) would need to constantly track which specific nodes currently have a matching pod — brittle and operationally painful. Opening it uniformly on every node and letting kube-proxy handle the internal forwarding is what makes "point a load balancer at all your nodes' IPs" a viable, simple pattern at all.

Q244 NodeSelector vs Affinity vs Taints/Tolerations — aren't these all the same thing?

The trap: they all "control where pods land," so people lump them together — but they solve genuinely different placement problems, and conflating them is a real, common design mistake. **NodeSelector**: the simplest form — an exact-match label requirement, no partial matching, no "prefer but don't require," no expressions at all. **Node Affinity**: a strictly more expressive superset — supports `In` / `NotIn` / `Exists` operators, and critically distinguishes `required` (hard constraint, same as nodeSelector but richer) from `preferred` (a soft, weighted hint the scheduler tries to honor but won't block scheduling over). **Pod Affinity/Anti-Affinity**: places pods relative to *other pods*, not node labels at all (e.g. "run near/away from pods with this label").

The one people most often get backwards: **Taints and Tolerations work in the opposite direction** from all of the above — a taint on a *node* repels pods by default, and a pod needs a matching *toleration* to be allowed to schedule there anyway; it's a node-side "keep out unless permitted" mechanism, not a pod-side "I want to go here" preference. This is exactly why dedicating nodes to a specific workload correctly requires **both**: a taint (repel everything else) plus a toleration on the intended pods (permit them specifically) plus, often, affinity too (actually attract them there, since tolerating a taint only means "allowed," not "preferred") — using only one of the three usually doesn't achieve true node dedication.

Q245 Why use IRSA instead of just giving the node's IAM role broad permissions?

The trap: "it's simpler to just give the node role everything it might need" is a real, common shortcut — the question tests whether you can articulate exactly what it costs. Without IRSA, **every pod on that node inherits the node's full instance-profile permissions**, since standard AWS credential resolution has no concept of "which pod is asking" — only "which EC2 instance." That means the least-privileged pod on a node is only as protected as the *most*-privileged workload sharing that node's role, and a single compromised pod (an RCE vulnerability, a malicious dependency) gets every permission every workload on that node collectively needs, combined — a far larger blast radius than that one pod's actual job requires.

```
# Without IRSA: every pod on the node can do this, whether it should or not
kubectl exec -it <any-pod-on-node> -- aws s3 ls   # inherits full node role
```

IRSA's actual value, stated precisely: it scopes IAM permissions down to the **pod/ServiceAccount** level instead of the node level, so a compromised pod is limited to exactly what its own workload legitimately needs, not everything every neighbor on that node needs too — the textbook least-privilege argument, made concrete with a real, common architectural shortcut people take specifically because IRSA setup involves more upfront configuration than "just widen the node role."

Q246 Why bother with NetworkPolicy if Security Groups already control traffic?

The trap: they sound redundant since both "control which traffic is allowed," but they operate at fundamentally different granularity and layers, and knowing exactly where each stops is the point. **Security Groups** (with the default VPC CNI model, where pods share the node's SG unless using per-pod SGs) operate at the **node** level — they can't natively distinguish "traffic to pod A on this node" from "traffic to pod B on the same node," since from the SG's perspective it's all just traffic to that node's ENI. **NetworkPolicy** operates at the **pod** level, using label selectors — it can express "only pods labeled `role=frontend` may reach pods labeled `role=backend`, on this exact port," regardless of which nodes either happens to be scheduled on right now, and regardless of how pods get rescheduled over time.

Concretely, the gap this closes: two entirely unrelated applications' pods, co-located on the same node purely by scheduling coincidence, are indistinguishable to a node-level Security Group but can be fully isolated from each other by NetworkPolicy — which matters a great deal in a genuinely multi-tenant cluster, or under a zero-trust posture where "same node" should never imply "trusted to talk to each other." The correct mental model: SGs secure the perimeter (VPC boundary, what's allowed in/out of the cluster's nodes at all); NetworkPolicy secures the interior (which workloads may talk to which other workloads once already inside) — genuinely complementary layers, not a redundant pair.

Q247 What actually happens during an EKS version upgrade?

The trap: assuming "upgrade the cluster" is one atomic action — it's actually **two entirely separate upgrades** that AWS deliberately keeps decoupled, and conflating them is the most common misunderstanding this question is testing for. **Control plane upgrade**: triggered via `aws eks update-cluster-version`, entirely AWS-managed, updates the API server/etcd/controller-manager/scheduler with no action needed on your nodes at all — but it can only ever move **one minor version at a time** (1.29→1.30, never 1.29→1.31 directly), a hard constraint worth stating precisely since it directly shapes upgrade planning for anyone multiple versions behind.

```
aws eks update-cluster-version --name <cluster> --kubernetes-version 1.31
aws eks update-nodegroup-version --cluster-name <cluster> --nodegroup-name <ng>
```

Node upgrade: a completely separate, later step you trigger yourself (`update-nodegroup-version` for managed node groups) — nodes keep running their old kubelet version until you do this, and EKS enforces a defined **version skew policy** (kubelet can run up to a few minor versions behind the control plane, but not indefinitely) meaning you can't upgrade the control plane and simply leave nodes on an old version forever. The sequencing that matters: control plane first, always, then nodes after — nodes newer than the control plane isn't a supported configuration at all.

Q248 How do you upgrade Kubernetes with genuinely zero downtime?

The trap: treating this as a single technique rather than the composition of several independent safety mechanisms already covered elsewhere in this handbook, applied together specifically to the upgrade scenario. Sequence worth stating explicitly: control plane upgrade first (AWS-managed on EKS, and inherently zero-downtime for your workloads since it doesn't touch running pods at all); then nodes, using **surge upgrades** or a **blue/green node group swap** — create new nodes on the new version *alongside* the old ones (never upgrade-in-place on existing nodes), then drain the old ones one at a time (or in small batches), letting the scheduler place displaced pods onto the new nodes as capacity allows.

```
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data --grace-period=60
kubectl get pdb -A   # verify PDBs won't block the drain entirely
```

The specific mechanisms that make the drain itself non-disruptive, all covered earlier in this handbook and worth connecting together here explicitly: readiness probes ensuring traffic never reaches a not-yet-ready replacement pod, `PodDisruptionBudgets` preventing the drain from evicting so many replicas at once that availability drops below a defined minimum, and `maxUnavailable` / `maxSurge` on the Deployment governing how aggressively pods get rescheduled during the churn. The core insight worth stating as a summary: a zero-downtime cluster upgrade isn't a special new capability, it's simply the normal zero-downtime deployment machinery (Q149) applied to node replacement instead of application rollout.

Q249 How do you recover an accidentally deleted namespace?

The trap: people reach straight for "restore from an etcd backup," which is disproportionate, cluster-wide, and risky (rolling back *every* object cluster-wide to a snapshot time, not just the one namespace) when a far more targeted fix is usually available and should be tried first. Deleting a namespace triggers **cascading garbage collection** of everything inside it (via `ownerReferences`) — but that cascade takes real time proportional to how much was in it, and the namespace often sits in `Terminating` for a while, meaning immediate action can sometimes still catch objects before they're actually gone.

```
kubectl get all -n <deleted-ns>   # act fast — check if anything is still Terminating, not yet gone
velero restore create --from-backup <backup-name> --include-namespaces <ns>
```

The properly-designed recovery path, worth naming as the "right" answer rather than a full etcd restore: a dedicated backup tool like **Velero**, which takes namespace-scoped (or full-cluster) backups of Kubernetes object definitions *and* can coordinate PV snapshots, letting you restore precisely the one deleted namespace's resources without touching anything else in the cluster. This question is really testing whether the candidate's instinct is "nuke and restore everything from etcd" (blunt, risky, slow) versus "do we have namespace-scoped backup tooling for exactly this" (targeted, fast, safe) — and, for GitOps-managed namespaces specifically, whether they'd think to just let Argo CD/Flux re-reconcile the namespace back into existence from Git as an even simpler recovery path when the manifests are already declaratively tracked.

PART VII

Advanced Staff / Principal SRE Scenarios

10 questions · open-ended system design & incident scenarios expected at Staff/Principal/AVP level

Advanced Scenarios

Q250 **Design a highly-available EKS cluster spanning multiple Availability Zones.**

Structure this as a layer-by-layer design walkthrough — at this level, interviewers are grading whether you can reason about every layer independently rather than reciting "spread nodes across AZs" as a single sufficient answer.

Control plane: EKS already runs the control plane across multiple AZs automatically — state this as a given, but note that your own account's **API request rate and any self-managed components** (webhooks, CRDs with heavy reconciliation) still need to scale sensibly against it; there's nothing to architect here beyond ensuring subnets across at least 3 AZs are provided at cluster creation (a decision made once, at creation time, and not easily changed later).

Node groups: spread managed node groups across at least 3 AZs, sized so that losing any single AZ still leaves enough capacity for full production load in the remaining AZs (not just "spread evenly," but "survive losing exactly one" as the explicit design target) — this means running at >50% extra capacity across the remaining AZs specifically for this failure mode, not just round-robin distribution. Combine with `topologySpreadConstraints` at the pod level so the scheduler actively spreads replicas across AZs rather than merely allowing it.

Storage: this is the layer most people under-design. EBS volumes are AZ-locked (Q136) — a stateful workload using EBS can't simply "fail over" across AZs; either accept that constraint explicitly for workloads that can tolerate it (rescheduled to the same AZ only), or use EFS/S3 for anything that genuinely needs cross-AZ data access, or run application-level replication (a database's own multi-AZ replica topology) rather than relying on the storage layer to solve it.

Networking: one NAT Gateway *per AZ* (never a single shared one, per Q124), and load balancers (ALB) provisioned across subnets in every AZ in use so an AZ failure doesn't take down ingress capacity along with it.

Validated by: the design isn't complete until you've actually tested an AZ-failure game day — cordoning/draining every node in one AZ and confirming the application stays healthy, not just assuming the architecture diagram implies it will.

Q251 The API server's request latency has spiked to 10 seconds cluster-wide. Walk through your response.

Lead with the single highest-probability root cause before broadening — at Staff/Principal level, the answer should demonstrate a prioritized diagnostic order, not an unordered list of everything that could possibly matter.

```
# 1. etcd health/latency first — far and away the most common root cause
ETCDCTL_API=3 etcdctl endpoint status --write-out=table
ETCDCTL_API=3 etcdctl endpoint health
kubectl get --raw /metrics | grep etcd_request_duration
```

1) etcd first, always: the API server's read/write latency is fundamentally bound by etcd's own latency — disk I/O contention (etcd is extremely sensitive to disk write latency specifically), a leader election churning repeatedly, or database size approaching etcd's practical limits (large clusters with excessive Events/ConfigMap churn are a classic, self-inflicted cause) are the first things to rule in or out.

2) Request volume and throttling: check API server metrics for request rate and any `429` throttling, which points to either a genuine traffic increase (a bad client retry-storming, a controller stuck in a hot reconcile loop hammering the API) or API Priority and Fairness limits being hit.

3) Admission webhooks: a slow or failing `ValidatingWebhookConfiguration` / `MutatingWebhookConfiguration` adds its own latency to *every* matching request synchronously — check webhook `timeoutSeconds` and whether a specific webhook's backend is itself degraded, since this is an easy thing to overlook when focused purely on the API server/etcd.

4) Scale of the control plane itself: on EKS specifically, the control plane auto-scales based on load, but a very sudden spike (a large batch job creating thousands of objects at once, a controller in a reconcile storm) can outpace that scaling briefly — correlate the timing with any recent large-scale operation.

Communicate broadly and early (this affects every team simultaneously) while narrowing the cause, and once resolved, add the specific metric that would have caught it earlier (etcd latency percentiles, API server request duration by verb/resource) as a standing dashboard/alert rather than closing the incident with only a point fix.

Q252 You've had an accidental etcd data loss in production. Walk through recovery.

Worth stating upfront, precisely: on EKS, etcd is entirely AWS-managed — this exact scenario isn't something an EKS customer can cause or directly recover from, since there's no customer-facing etcd access at all. Frame the answer around a **self-managed** cluster (kubeadm, or EKS Anywhere/on-prem), which is clearly what's actually being tested here, while explicitly naming that distinction — it shows precise knowledge of the shared-responsibility boundary rather than glossing over it.

```
ETCDCTL_API=3 etcdctl snapshot status /backup/etcd-snapshot.db --write-out=table
ETCDCTL_API=3 etcdctl snapshot restore /backup/etcd-snapshot.db \
  --name <member-name> --initial-cluster <cluster-str> \
  --initial-advertise-peer-urls https://<ip>:2380 --data-dir /var/lib/etcd-restored
```

Immediate: stop the API server (and anything else writing to etcd) first — continuing to accept writes against a partially-lost/inconsistent etcd risks compounding the damage or creating a split-brain state across members.

Assess scope: is this total data loss (all members affected) or a single member's disk failure (recoverable by simply re-joining a fresh member to the still-healthy majority, no restore needed at all — the far more common real-world version of "etcd data loss," worth explicitly distinguishing from a full-cluster loss)? A healthy etcd cluster tolerates the loss of a minority of members without any data loss at all, by design (Raft quorum) — true full data loss is comparatively rare and usually implies a backup/snapshot restore was already the plan, not an improvisation.

Restore: from the most recent validated snapshot (`snapshot restore`, shown above) onto a fresh data directory, reconstructing the etcd cluster from that single-member restored state, then rejoining remaining/replacement members to it. Point out explicitly: any writes between the snapshot and the loss are gone — this is real, acknowledged data loss, and part of the answer is being honest about that rather than implying a snapshot restore is lossless.

Verify before reopening traffic: restart the API server against the restored etcd, verify core objects (Nodes, key Deployments) look sane, and only then allow traffic/reconciliation to resume. Close the loop afterward: this incident is really a referendum on backup cadence and testing — the actual fix isn't just "restore," it's tightening snapshot frequency, verifying restores are periodically *tested* (not just taken and assumed to work), and shipping backups off-node/off-cluster so a single infrastructure failure can't take out both etcd and its backups together.

Q253 You're seeing intermittent network failures affecting only some Pods. How do you diagnose this?

The framing that matters most at this level: intermittent + only-some-pods means it's almost never "the network is broken" in a simple sense — the actual work is **finding what dimension the affected pods share** that the unaffected ones don't, and every step below is aimed at narrowing that dimension.

```
kubectl get pods -o wide | sort -k7   # group by node — same node(s)?
kubectl get pods -o json | jq -r '.items[] | select(.status.phase=="Running") | .spec.nodeName' | sort | uniq -c
```

Same node(s)? If every affected pod lands on a small subset of nodes, suspect node-level causes: that node's `aws-node` (VPC CNI) pod unhealthy, ENI/IP exhaustion intermittently hitting a capacity ceiling specifically on that node, or a NIC/driver issue at the EC2 instance level.

Same workload/version? If it's one specific Deployment regardless of node, look at the application itself — connection pool exhaustion, an aggressive client-side retry/timeout setting that surfaces as "network failure" but is actually the app giving up too early under load, or a recent version bump that changed connection-handling behavior.

Same destination/path? If failures cluster around calls to one specific downstream (cross-AZ, cross-VPC, an external dependency), suspect that specific path: security group rule, NACL, cross-AZ NAT Gateway bandwidth ceiling, or conntrack table exhaustion on a specific node under connection churn (`conntrack -S` showing high `insert_failed` is the concrete signal here, and a frequently-overlooked one).

Correlate with time/events: plot failures against deployment timestamps, scaling events, and node-replacement events — "intermittent" that correlates precisely with scale-up moments often points to IP/ENI allocation racing scheduling, resolving itself once new IPs are warmed, rather than a persistent fault.

Direct evidence: once narrowed, get inside an affected pod during a failure window (not after) — `tcpdump` on the pod's interface, or a lightweight ephemeral debug container attached via `kubectl debug`, to see the actual failure mode (connection refused vs. timeout vs. reset) rather than inferring it secondhand from application logs, since each failure mode points to a different layer.

Q254 You need to upgrade a production EKS cluster running thousands of workloads with zero downtime. Walk through your plan.

The core mechanics are the same as any zero-downtime upgrade (Q248) — the Staff/Principal-level part of this answer is everything that scale specifically demands beyond those mechanics: capacity planning, staging, automation, and rollback readiness at a volume where manual, one-at-a-time thinking breaks down.

Pre-flight, well before touch anything: audit for deprecated/removed API versions across every workload targeting the new version (a version-skew scan, not a hopeful assumption), verify every critical Deployment has a `PodDisruptionBudget` set sensibly (not `maxUnavailable: 0` cluster-wide, which would simply block the upgrade from draining anything), and confirm cluster-autoscaler/Karpenter capacity limits are raised enough to support genuine surge capacity for a cluster this size — thousands of workloads rescheduling needs real headroom, not the day-to-day buffer.

Control plane: upgrade first (one minor version, AWS-managed, no workload impact) — but don't treat "no impact" as "no verification"; watch API server latency and error rates through this step regardless, since some admission webhooks or custom controllers occasionally have version-specific issues that only surface once traffic resumes hitting the new control plane.

Nodes, staged, not all at once: stand up a new node group on the target version *alongside* the old one, then migrate in explicit waves — start with a small, low-risk slice (a canary node group, or specific low-priority namespaces) and validate before continuing, rather than draining thousands of pods across the whole fleet in one operation. Automate the drain sequence itself with a defined concurrency limit (drain N nodes at a time, wait for stability, continue) instead of a single unbounded `kubectl drain` loop — at this scale, an unbounded drain can overwhelm the scheduler and networking layer simultaneously trying to reschedule everything at once.

Observability throughout: a live dashboard tracking pod-reschedule success rate, error-rate/latency deltas per namespace, and node-join health, so a regression in one wave is caught before proceeding to the next — the whole point of staging is having a natural pause point to catch exactly this. **Rollback readiness:** keep the old node group intact (not deleted) until the new version has fully baked in production for a defined period, so reverting is "shift traffic back," not "rebuild from scratch" if something surfaces late. Budget real calendar time for this — at thousands-of-workloads scale, this is realistically a multi-day, carefully-paced operation, not a single maintenance window, and communicating that timeline honestly upfront is itself part of doing this well.

Q255 One microservice's latency has increased noticeably right after a deployment. Walk through your investigation.

The framing worth establishing immediately: a latency regression tied precisely to a deployment timestamp is one of the *more* tractable production problems, precisely because you have an exact before/after boundary and a known diff (the deployment's changeset) — the investigation should exploit that, not treat it as generic latency debugging from scratch.

Confirm correlation, not just coincidence: overlay the latency graph with the exact rollout timestamp (Argo CD/deployment history gives this precisely) — did it step-change right at that moment, or was it already trending upward beforehand and the deployment is a red herring? This single check saves significant misdirected effort.

```
kubectl rollout history deployment/<svc>
kubectl diff -f <(helm template ./chart -f values-prod.yaml) --prune -l app=<svc>
```

What actually changed: code diff (a new N+1 query, a changed serialization approach, a new synchronous call to a dependency that used to be async or cached); config/values diff (a resource limit lowered, causing throttling under the same load that was fine before); or infrastructure diff introduced alongside it (a new sidecar added, additional per-request overhead from a mesh proxy or logging agent bundled into the same release).

Where in the stack, precisely: pull the specific latency breakdown — is added time inside the application itself (check APM/tracing spans, if instrumented) or waiting on a downstream dependency it calls? A regression concentrated in one specific downstream call, versus uniformly spread across the whole request, points to two completely different fixes. Check resource-level signals too: did CPU throttling appear post-deploy (a lowered limit, or a new resource-heavier code path) — `container_cpu_cfs_throttled_periods_total` climbing right at the deploy boundary is a very specific, easy-to-miss smoking gun.

Resolve, then close the loop properly: if the fix is a rollback, do it while continuing the investigation in parallel rather than treating rollback as the end of the process — the whole point of catching a regression via deployment correlation is being able to identify the exact offending change afterward, not just reverting and moving on. Add whatever signal would have caught this *before* full rollout next time (a canary step with an automated latency-based gate, Q155) as the actual long-term fix, not just this one incident's resolution.

Q256 You discover a compromised container attempting lateral movement. Walk through your response.

Structure the answer around the standard incident phases (contain → investigate → eradicate → recover → learn), which is itself the signal a senior candidate should lead with, rather than jumping straight to "delete the pod."

Contain first, without destroying evidence: the instinct to immediately delete the pod is actually the wrong first move — it destroys forensic evidence (memory state, process tree, network connections) you need for the investigation. Instead: isolate it at the network layer while it keeps running — apply an emergency `NetworkPolicy` denying all ingress/egress for that specific pod (by label/name), or cordon the node it's on to stop new scheduling there, without touching the pod's own lifecycle yet.

```
kubectl label pod <pod> quarantine=true
# NetworkPolicy selecting quarantine=true, denying all traffic
kubectl cordon <node>
kubectl get pod <pod> -o yaml > incident-pod-snapshot.yaml   # capture full state
kubectl exec <pod> -- ps aux   # if still safely inspectable
```

Assess blast radius immediately, in parallel with containment: what does this pod's ServiceAccount/IRSA role actually permit (Q179/Q180) — if it has broad IAM permissions, assume those may already be compromised and begin credential rotation for anything it could reach, in parallel with the technical investigation, not after it concludes. Check audit logs for anything that ServiceAccount's token actually did during the compromise window — this is exactly why audit logging (Q170) being enabled beforehand matters so much; without it, this step is largely guesswork.

Determine the entry point: a vulnerable dependency (correlate against recent CVE disclosures and this image's SBOM, if you have one — Q181), a supply-chain compromise (an unexpectedly modified base image), or a misconfiguration (an exposed debug endpoint, overly permissive RBAC that let something in from elsewhere). This determines whether the fix is "patch this one image" or "audit every workload using the same compromised base layer."

Eradicate and recover: only once evidence is captured and entry point understood, remove the compromised workload entirely (delete pod, revoke/rotate its IAM role and any Secrets it could access, rebuild the image from a clean, verified base) and redeploy through the normal pipeline — never patch and redeploy the same running compromised instance in place.

Learn: this incident is the concrete argument for runtime detection (Falco, Q164) if it wasn't already in place — a compromised container attempting lateral movement is exactly the behavior-based anomaly runtime tooling exists to catch fast, versus discovering it via a downstream symptom much later.

Q257 Plan a cluster-wide certificate rotation without disrupting any running workloads.

Break this into the distinct certificate populations covered throughout this handbook's TLS section — the Staff-level answer treats "cluster-wide cert rotation" as several coordinated sub-plans, not one uniform operation, since each population has a different blast radius and rotation mechanism.

Control-plane certs (API server, kubelet, etcd — on self-managed clusters; a non-issue on EKS where AWS manages this): plan a maintenance window per control-plane node, renewing via `kubeadm certs renew` and restarting the affected static pods one node at a time in a multi-master setup, never all simultaneously — verifying cluster health (`kubectl get componentstatuses` equivalent checks, API responsiveness) between each node before proceeding to the next, so a bad renewal on one node doesn't compound into a full outage before it's caught.

Application/Ingress TLS certs: if these are already on cert-manager with sane `renewBefore` margins, this is largely a non-event — the real planning work here is auditing that every workload's Ingress *actually is* on cert-manager-managed automatic rotation rather than a legacy manually-issued cert someone set up once years ago and forgot about; that audit, not the rotation mechanics, is usually where the actual risk hides.

```
kubectl get certificate -A -o json | jq -r '.items[] | select(.status.conditions[]?.status != "True") |
.metadata.name'
kubectl get ingress -A -o json | jq -r '.items[] | select(.spec.tls == null) | .metadata.name'   # find gaps
entirely
```

Internal CA / mTLS certs (service mesh, or a self-managed internal CA): if it's leaf-certificate rotation only (the common case, using the existing CA), this can typically happen with zero disruption since it's the same automated mechanism as any other cert-manager-issued cert. If it's a **root CA rotation** specifically (rare, high-stakes, and worth calling out as categorically different) — every client's trust store needs the new root added *before* anything starts presenting certs signed by it, requiring a careful dual-trust period (both old and new roots trusted simultaneously) before cutting over, then a second pass removing trust in the old root only once nothing is signed by it anymore.

Sequencing and validation across all populations: always renew and validate in a lower environment first with the exact same automation that'll run in production; stagger production rotation (don't do etcd, kubelet, and application certs all in the same maintenance window, even if each is individually low-risk, since a bad interaction is far harder to isolate when several things changed simultaneously); and have the previous cert/CA state readily available to roll back to for a defined bake period after each population's rotation completes.

Q258 You're seeing sudden IP exhaustion from the VPC CNI in production. Walk through your troubleshooting and fix.

The word "sudden" is the important qualifier here versus the general IP-exhaustion question earlier (Q120) — a senior-level answer distinguishes a *chronic, slowly-approached* capacity ceiling from a *sudden* exhaustion event, since the latter implies something specific changed or spiked, and finding that trigger is the actual investigation.

```
aws ec2 describe-subnets --subnet-ids <id> --query 'Subnets[].AvailableIpAddressCount'
kubectl get events -A --field-selector reason=FailedCreatePodSandBox
kubectl -n kube-system logs -l k8s-app=aws-node --since=1h | grep -i "insufficient\|no available"
```

Find the trigger, not just the symptom: correlate the exact onset time against recent events — a large scale-up (a traffic spike triggering HPA across many services simultaneously, or a batch job launching many pods at once) consuming IPs faster than the CNI's warm-pool/prefix-delegation logic could keep up; a new workload or namespace onboarded recently at a pod density nobody had capacity-planned for; or a change to `WARM_ENI_TARGET` / `WARM_IP_TARGET` /prefix-delegation settings (intentional or accidental) that reduced the buffer the CNI keeps pre-allocated.

Immediate mitigation (stop the bleeding): if a specific runaway workload is identified (an HPA scaling unbounded, a misconfigured Job creating far more pods than intended), pause/scale that down first — don't reach for infrastructure changes before addressing an active, ongoing cause if one exists. If it's genuine organic growth against real capacity limits, the fastest safe relief is usually adding subnet CIDR capacity (if the VPC design allows) or enabling/expanding prefix delegation on already-provisioned instance types, both of which can often apply without a full node replacement.

Structural fix: if this wasn't already using prefix delegation, this incident is the concrete argument for adopting it now (a single settings change dramatically raising per-node pod/IP density without new subnet capacity) — and pairing it with **capacity alerting** on subnet IP availability specifically (not just node CPU/memory) as a leading indicator, so the next approach to this ceiling is caught days ahead rather than discovered as a sudden production incident. Also worth reviewing: whether custom networking (a secondary CIDR dedicated to pod IPs) is warranted for headroom that scales independently of node/instance IP allocation entirely, for clusters expecting continued rapid growth.

Q259 Explain the exact request path from ALB through Ingress, Service, and kube-proxy to the Pod — including where TLS is terminated.

This question rewards precision — walk it hop by hop, naming exactly what changes (headers, IP, port, encryption state) at each boundary, since that precision is what distinguishes genuine understanding from a hand-wavy "the ALB routes to the pod" answer.

1) Client → ALB: the client's TLS connection terminates at the ALB itself, which holds the certificate (ACM-issued, referenced via `alb.ingress.kubernetes.io/certificate-arn`, Q127/Q130) — **this is the TLS termination point** in the most common configuration. The ALB's listener (443) decrypts here; everything from this point forward, by default, travels as plain HTTP inside the VPC, unless re-encryption is explicitly configured (Q205).

```
alb.ingress.kubernetes.io/target-type: ip   # or instance — changes hop 3 significantly
alb.ingress.kubernetes.io/backend-protocol: HTTP   # or HTTPS if re-encrypting
```

2) ALB → Target Group: the AWS Load Balancer Controller programmed the ALB's listener rules and target group directly from the `Ingress` object's path/host rules at reconciliation time (Q127) — the ALB evaluates the incoming request's host/path against those rules and picks a target group. Note there's no separate "NGINX Ingress controller pod" in this path at all when using the native ALB Controller — the ALB itself *is* the Ingress implementation, unlike an NGINX-based Ingress where the ALB/NLB would forward to NGINX pods first, which then apply the routing logic in a second hop.

3) Target Group → destination (the branch that matters most): in `instance` **target-type mode**, the ALB forwards to a node's IP on the Service's **NodePort** — meaning `kube-proxy` *is* in this path, DNAT'ing the connection from <node-ip>:<nodeport> to a chosen pod's actual IP:port via its iptables/IPVS rules (Q232), potentially forwarding to a pod on a completely different node than the one the ALB happened to hit. In `ip` **target-type mode** (the more common modern default), the ALB forwards *directly to the pod's own IP* — `kube-proxy` is entirely bypassed in this mode, since the ALB itself already knows every pod IP (synced continuously by the controller watching Endpoints) and load-balances across them itself.

4) Arrival at the Pod: the request reaches the container's listening port as plain HTTP (assuming no re-encryption was configured at hop 1) — the application itself never sees or handles TLS in this default configuration; that responsibility began and ended entirely at the ALB.

The single most valuable thing to state explicitly, since it's the detail interviewers are actually listening for: **target-type is the fork in the road that determines whether kube-proxy participates in this path at all** — a distinction that also directly explains why `ip` mode's health checks (Q128) go straight to the pod while `instance` mode's go through the Service/NodePort layer instead.